

深度学习

主讲：刘夏雷、郭春乐、王亚星
南开大学计算机学院

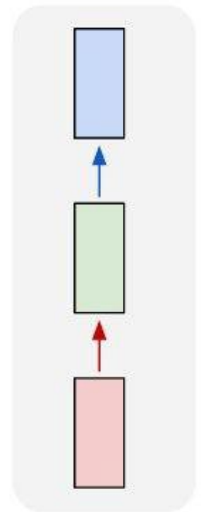
致谢：CS231n课程

提纲

- 深度学习历史发展
- 前馈神经网络
- 卷积神经网络
- 循环神经网络
- 注意力机制与Transformer
- 深度学习应用与研讨

普通(Vanilla)神经网络

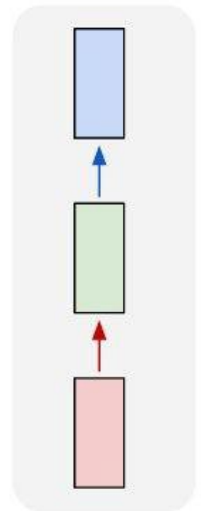
one to one



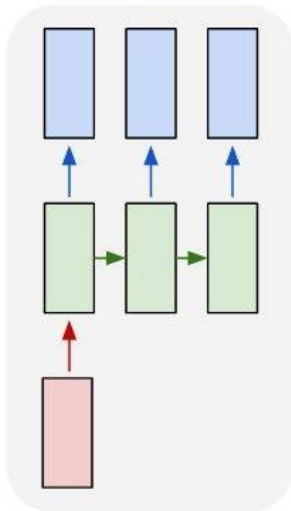
普通神经网络

循环神经网络：处理序列

one to one



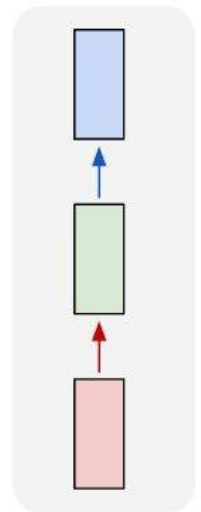
one to many



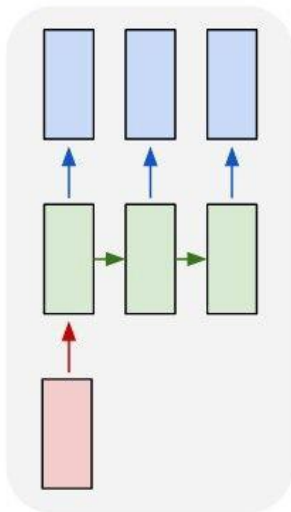
例：图像描述
图像 $\rightarrow$ 词语序列

循环神经网络：处理序列

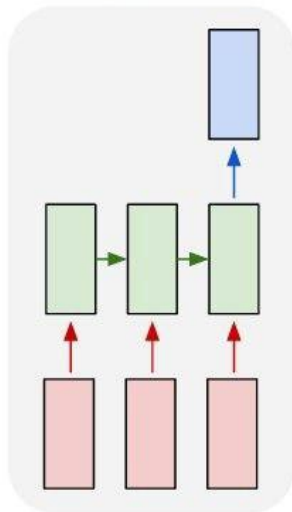
one to one



one to many



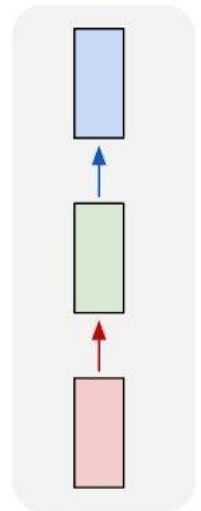
many to one



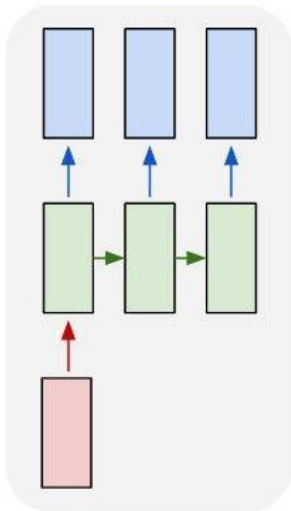
例：动作预测
视频帧序列 → 动作类别

循环神经网络：处理序列

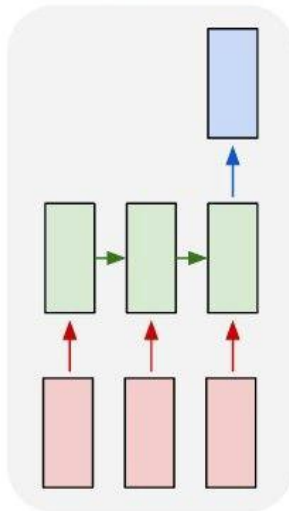
one to one



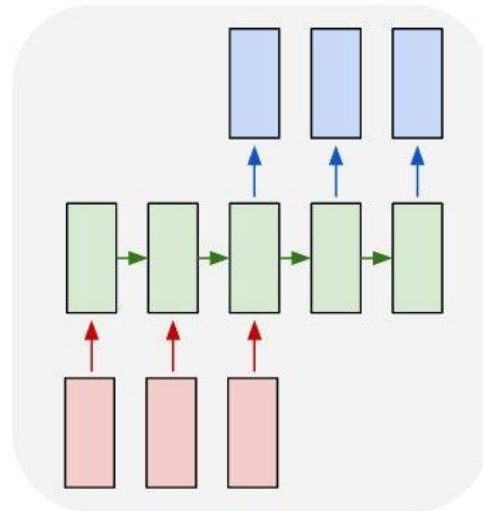
one to many



many to one



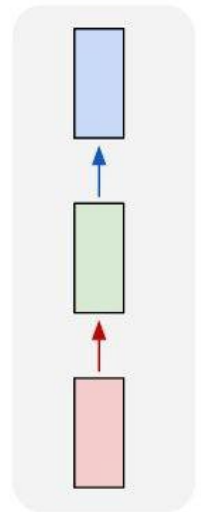
many to many



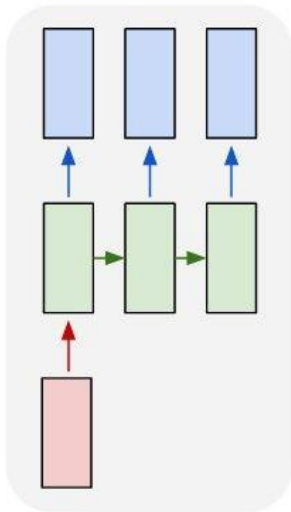
例：视频描述
视频帧序列 → 描述

循环神经网络：处理序列

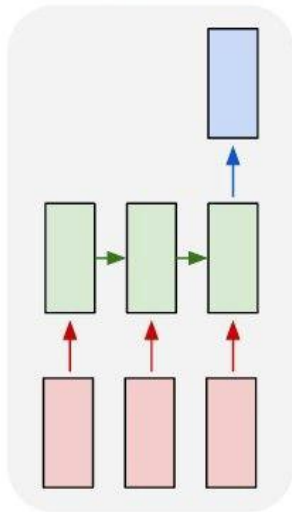
one to one



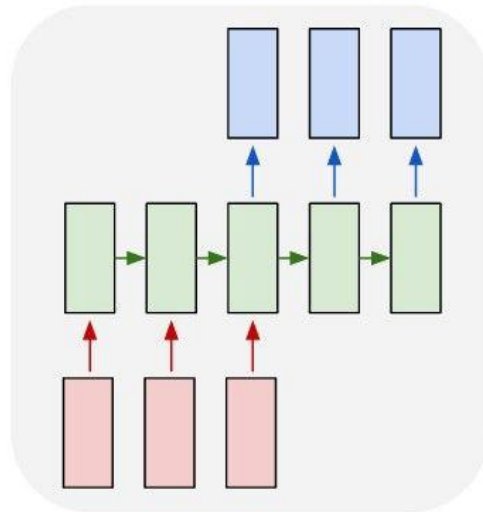
one to many



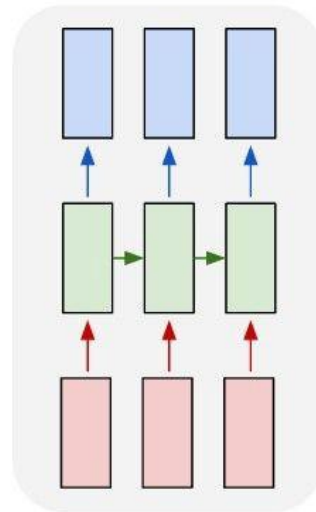
many to one



many to many



many to many



例：帧级别的视频分类

非序列数据的序列化处理

通过一系列“瞥视”来分类图像



Ba, Mnih, and Kavukcuoglu, "Multiple Object Recognition with Visual Attention", ICLR 2015. Gregor et al, "DRAW: A Recurrent Neural Network For Image Generation", ICML 2015

Figure copyright Karol Gregor, Ivo Danihelka, Alex Graves, Danilo Jimenez Rezende, and Daan Wierstra, 2015. Reproduced with permission.

非序列数据的序列化处理

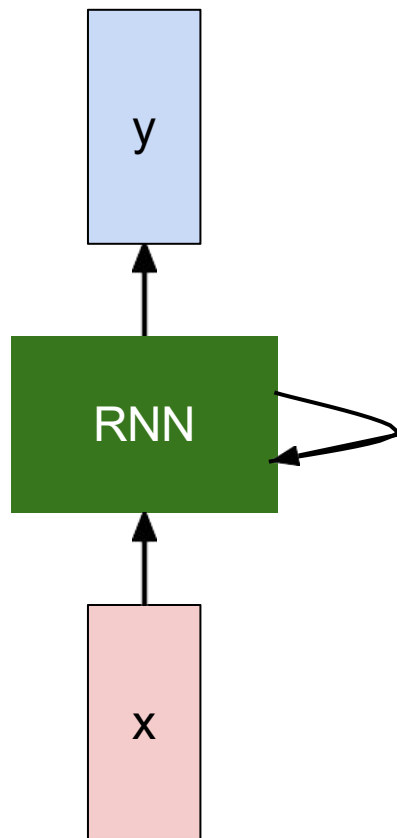
一次一块地生成图像！



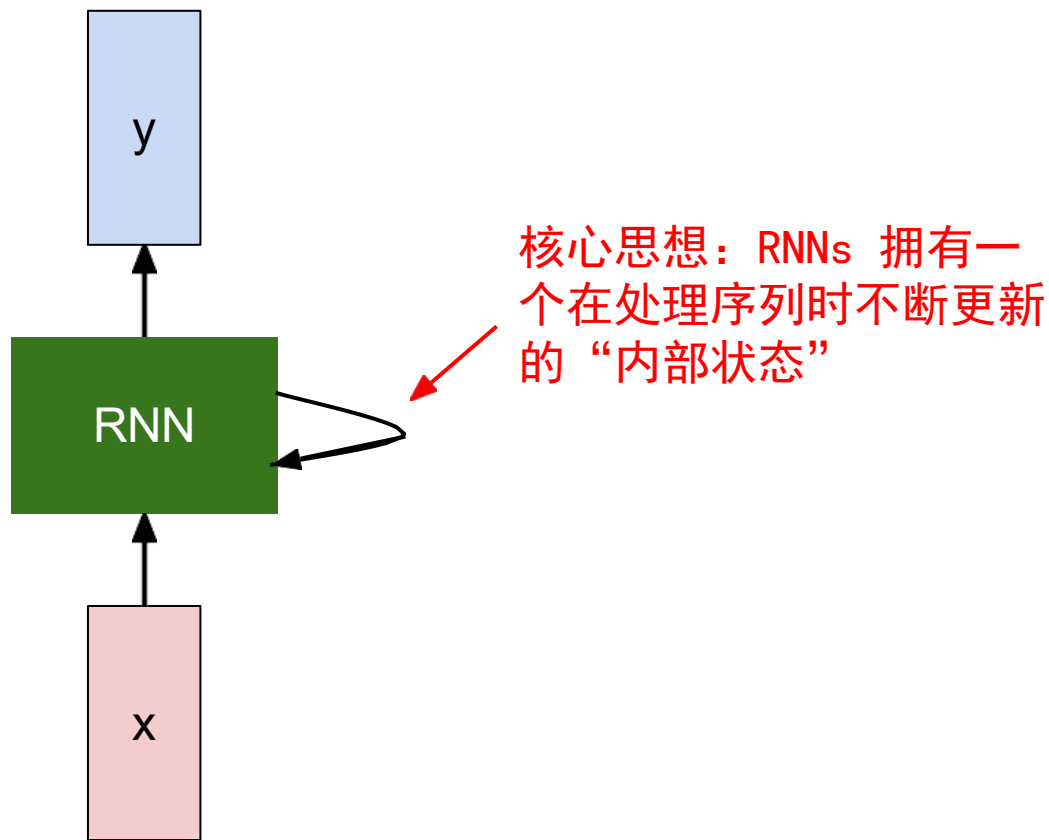
Gregor et al, "DRAW: A Recurrent Neural Network For Image Generation", ICML 2015

Figure copyright Karol Gregor, Ivo Danihelka, Alex Graves, Danilo Jimenez Rezende, and Daan Wierstra, 2015. Reproduced with permission.

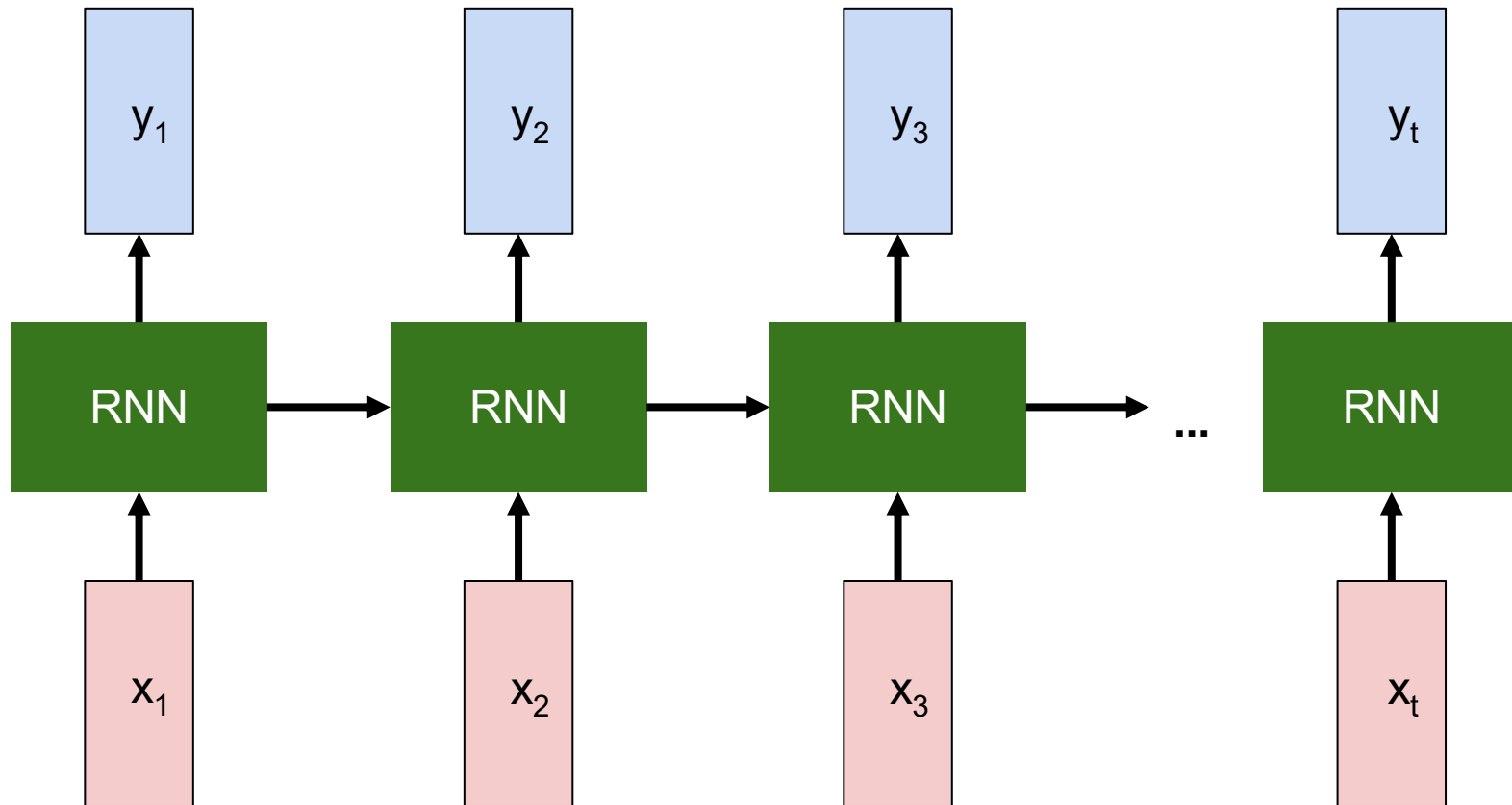
循环神经网络



循环神经网络



展开的 RNN (Unrolled RNN)



RNN隐藏状态更新

我们可以通过在每个时间步应用一个**递推公式**来处理一个向量序列 x :

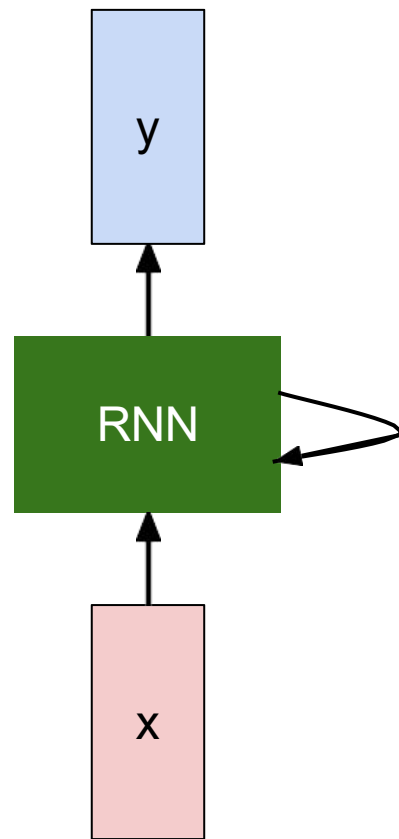
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

新状态

带有参数 W 的
某个函数

旧状态

某一时刻的
输入向量



RNN输出生成

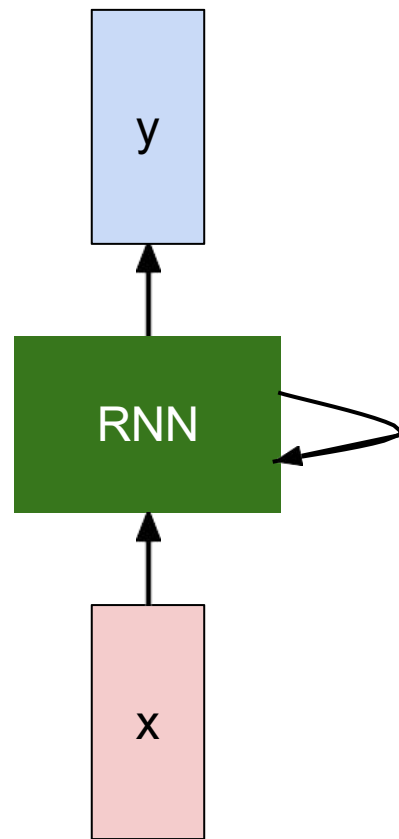
我们可以通过在每个时间步应用一个**递推公式**来处理一个向量序列 x :

$$\boxed{y_t} = \boxed{f_{W_{hy}}}(\boxed{h_t})$$

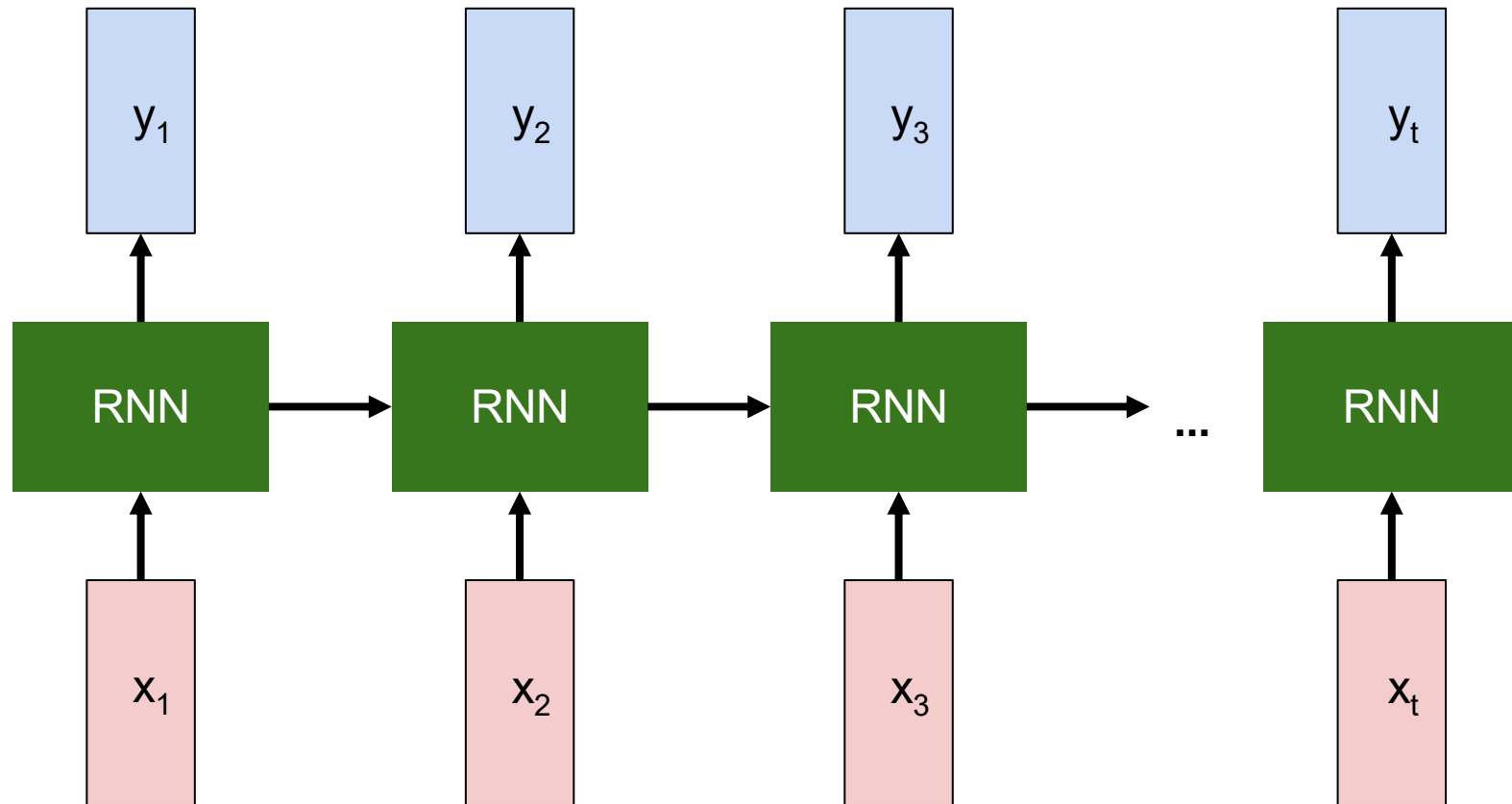
输出

新状态

带有参数 W 的
另一个函数



RNN循环神经网络

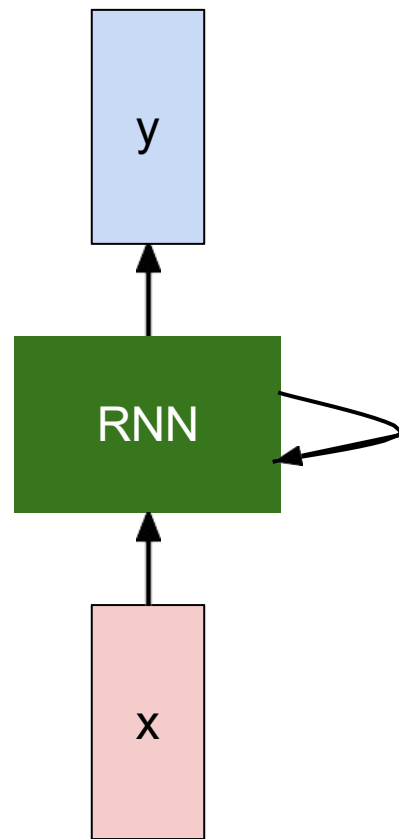


RNN输出生成

我们可以通过在每个时间步应用一个**递推公式**来处理一个向量序列 x :

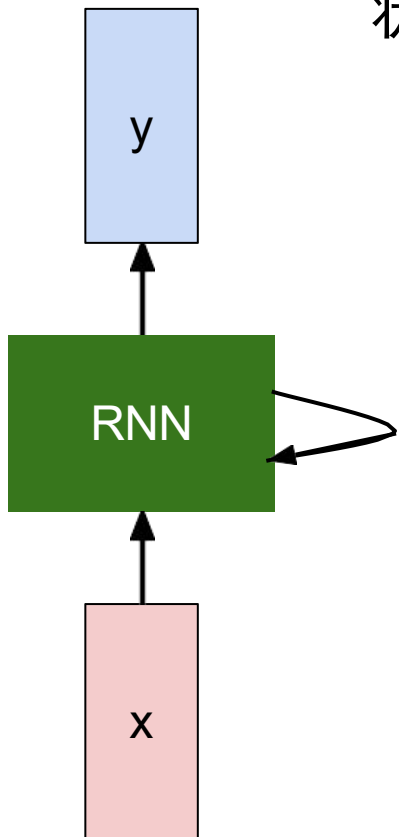
$$h_t = f_W(h_{t-1}, x_t)$$

注意：在每个时间步都使用相同的函数和相同的参数。



普通(Vanilla)循环神经网络

状态由一个单独的“隐藏”向量 h 组成：



$$h_t = f_W(h_{t-1}, x_t)$$

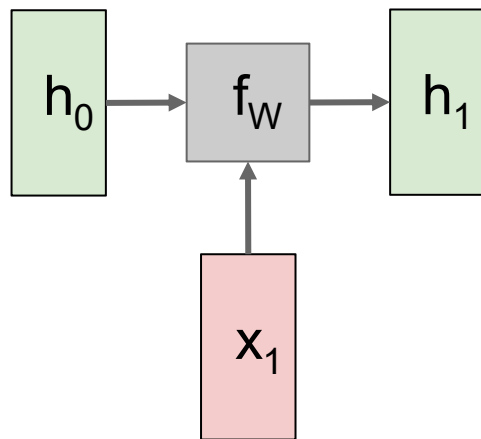


$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

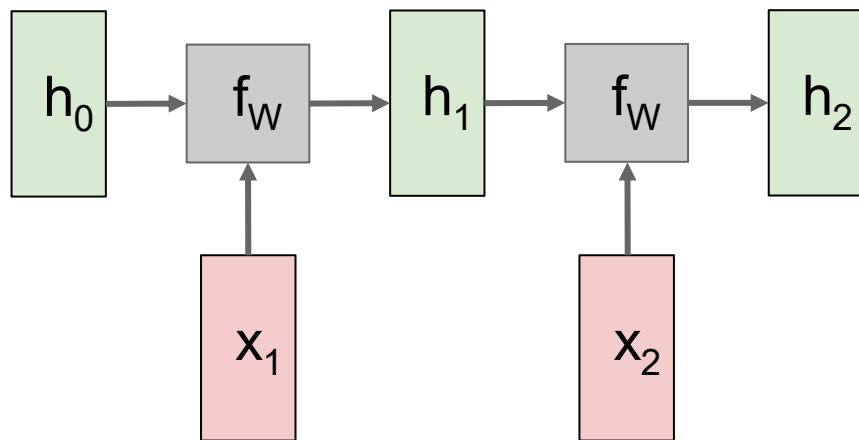
$$y_t = W_{hy}h_t$$

有时被称为“Vanilla RNN”或以 Jeffrey Elman 教授的名字命名的“Elman RNN”

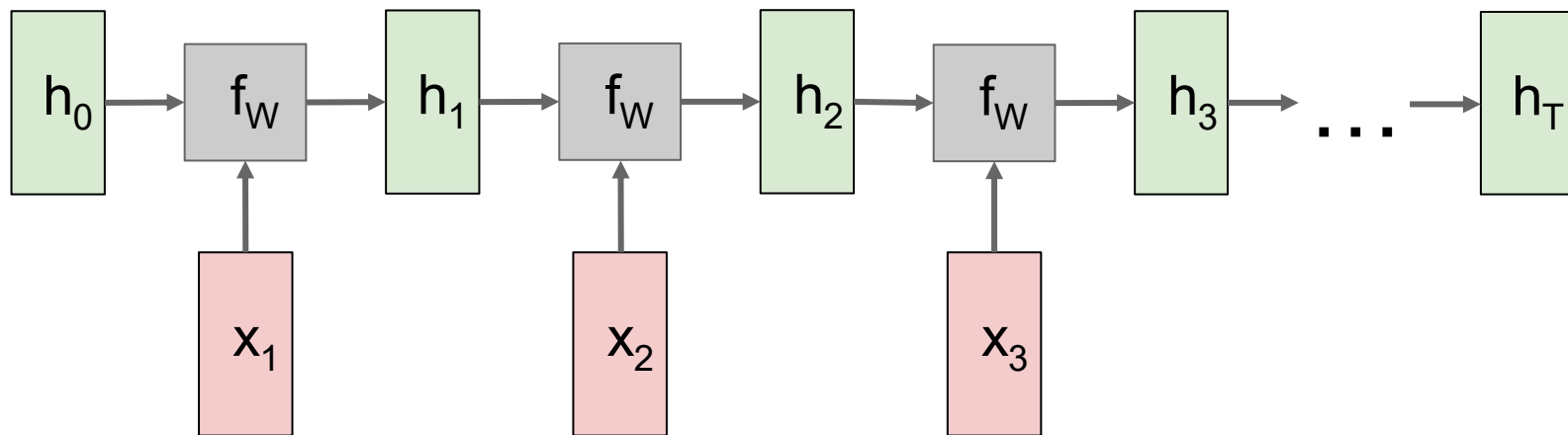
RNN: 计算图



RNN: 计算图

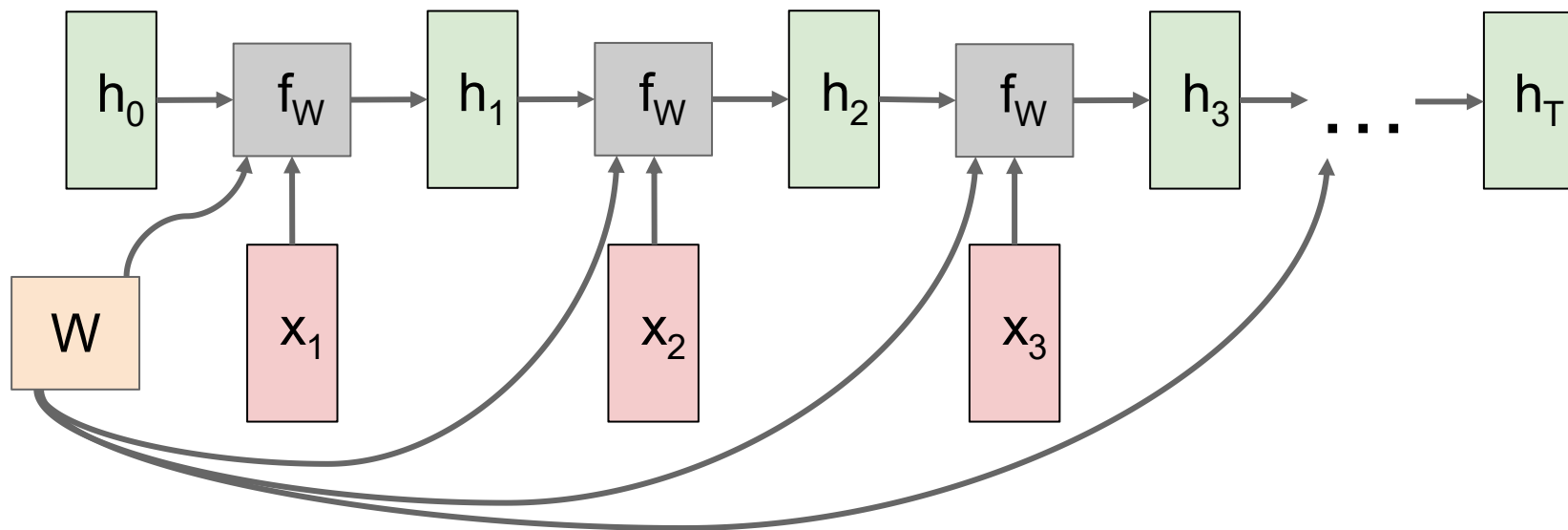


RNN: 计算图

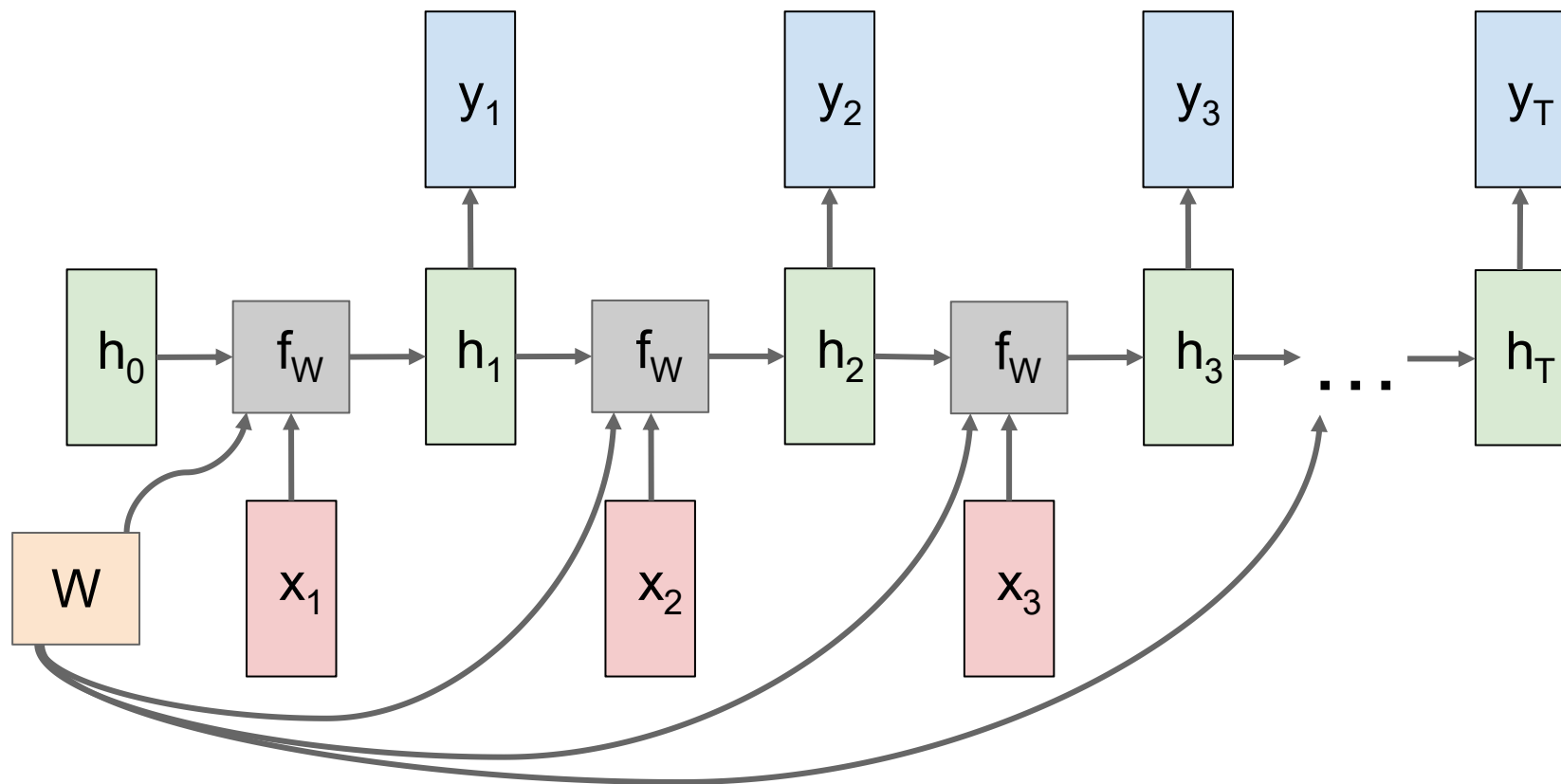


RNN: 计算图

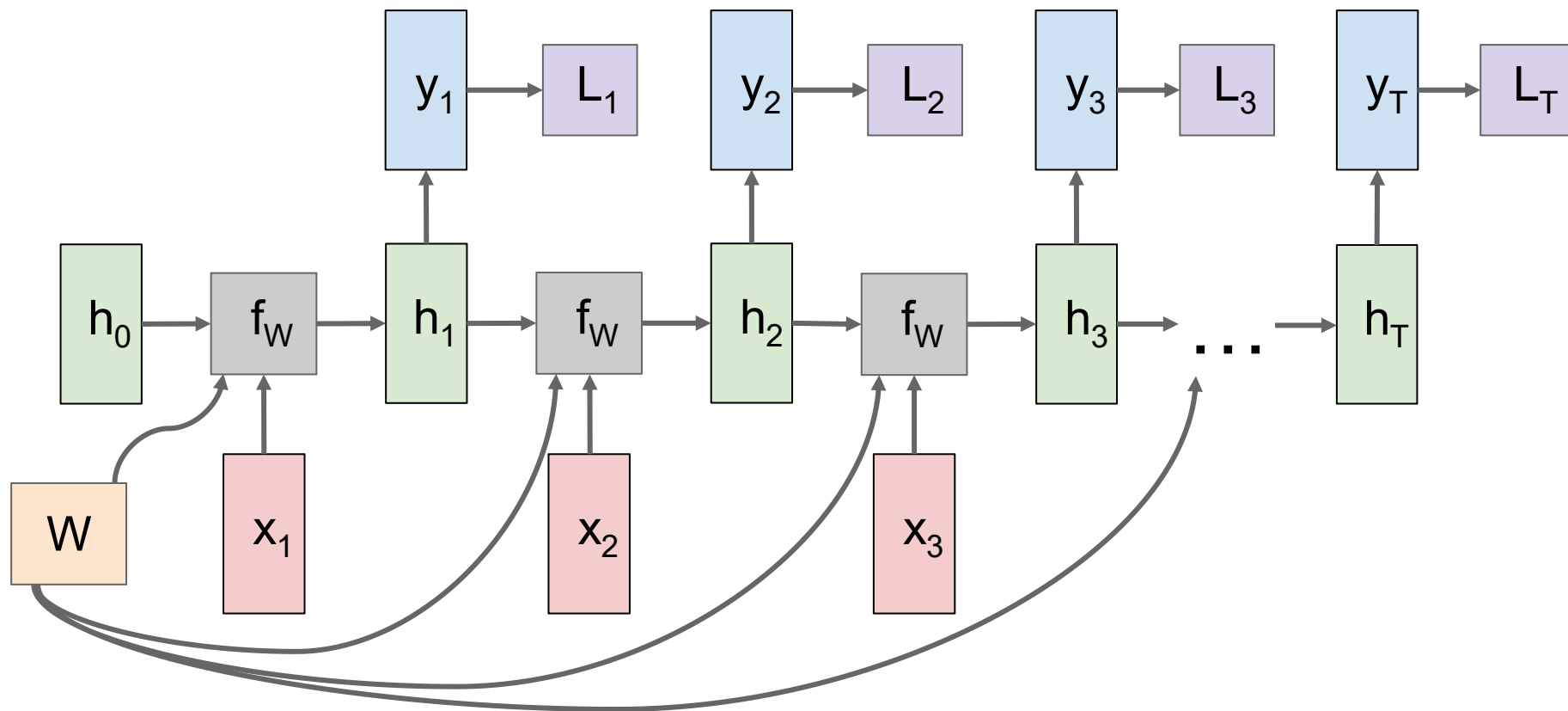
在每个时间步重复使用相同的权重矩阵



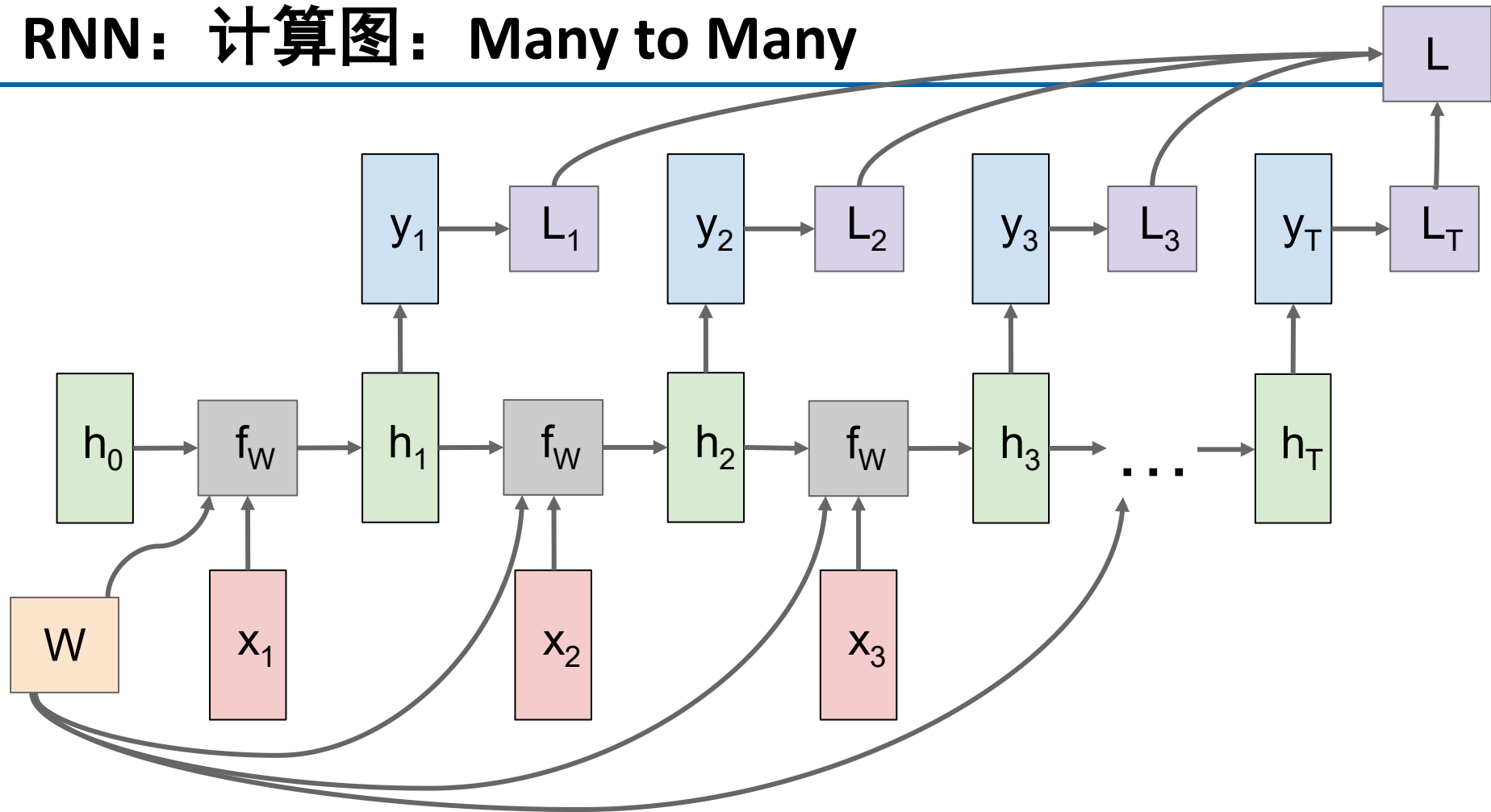
RNN: 计算图: Many to Many



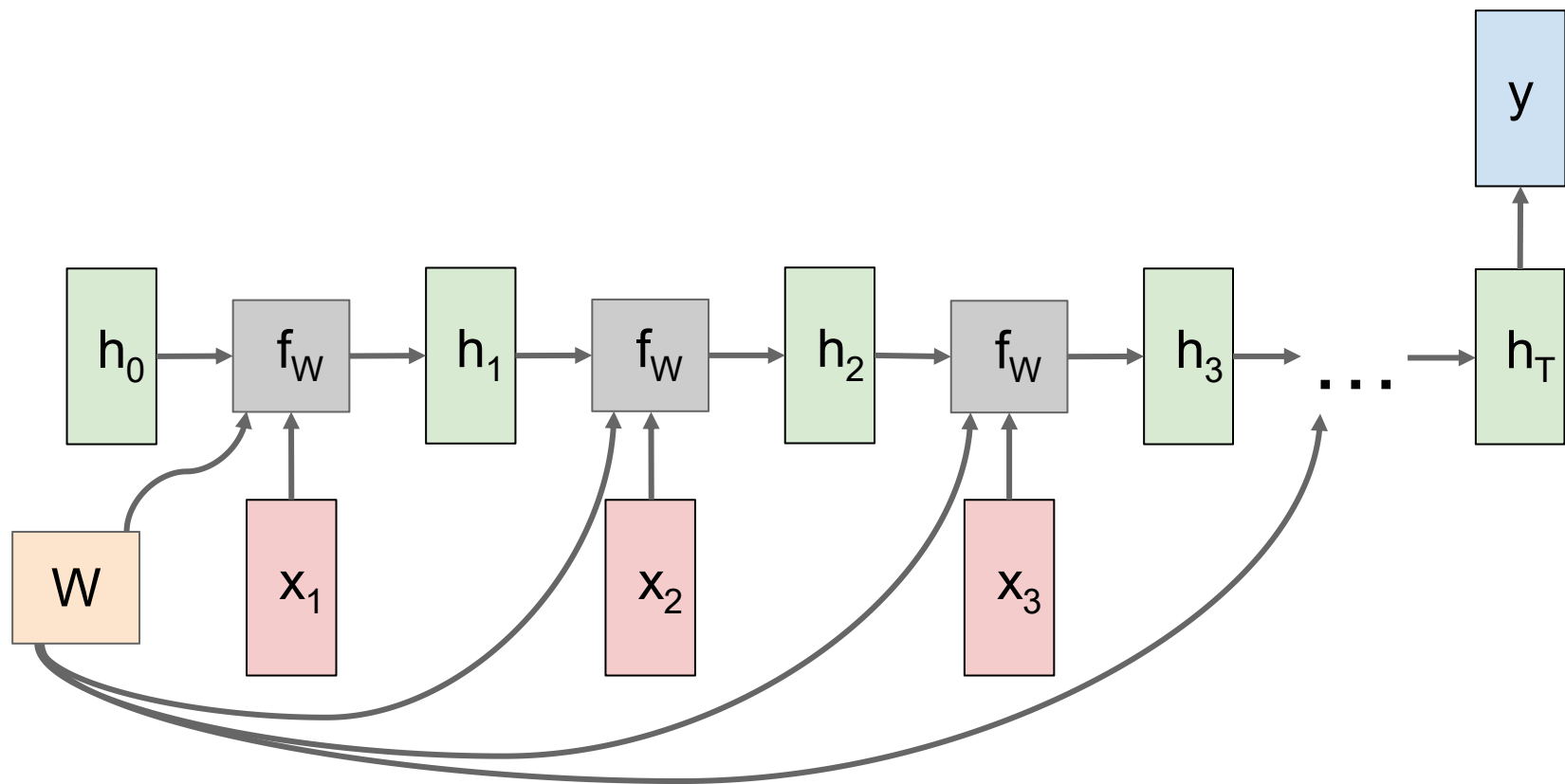
RNN: 计算图: Many to Many



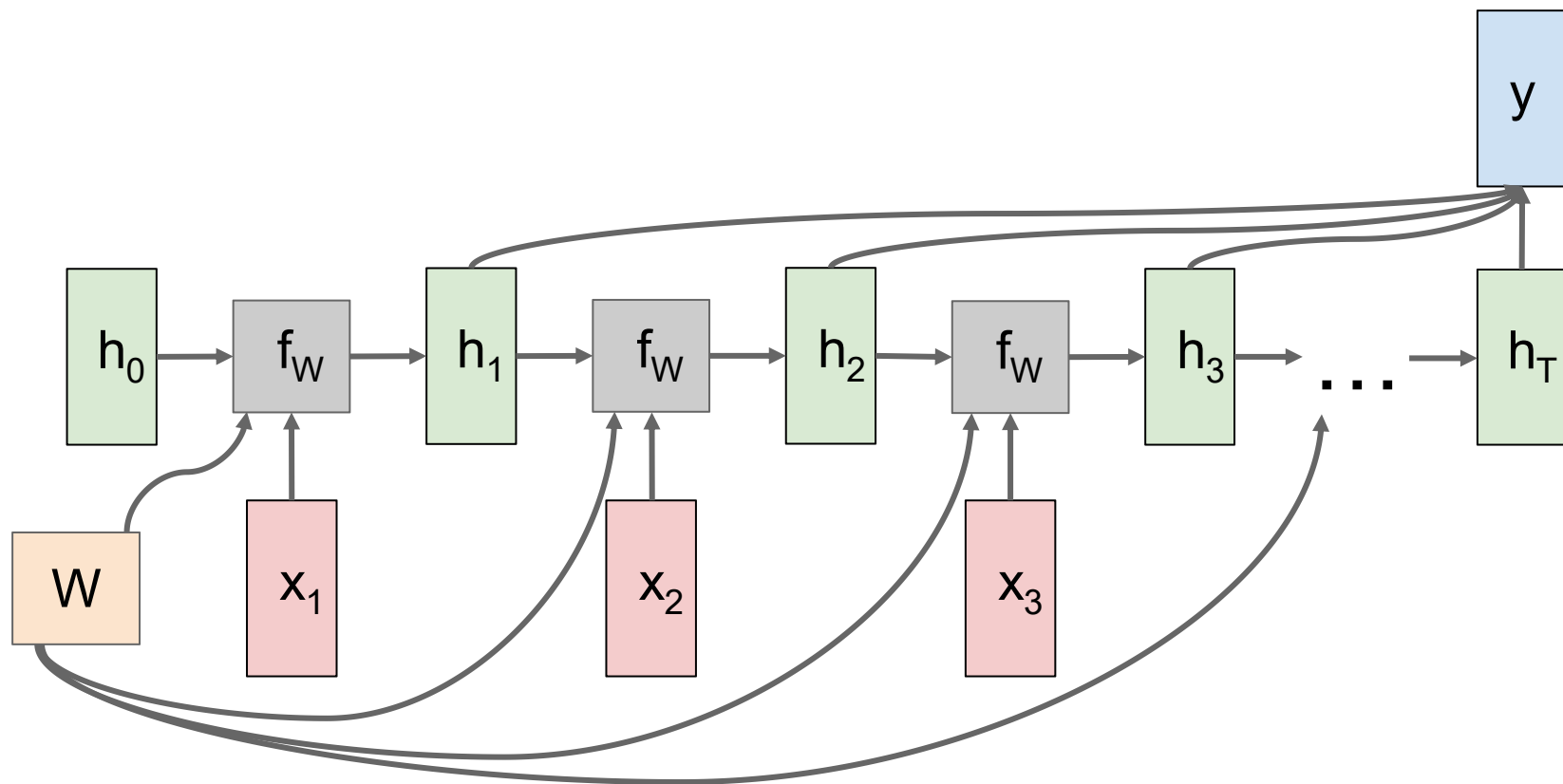
RNN: 计算图: Many to Many



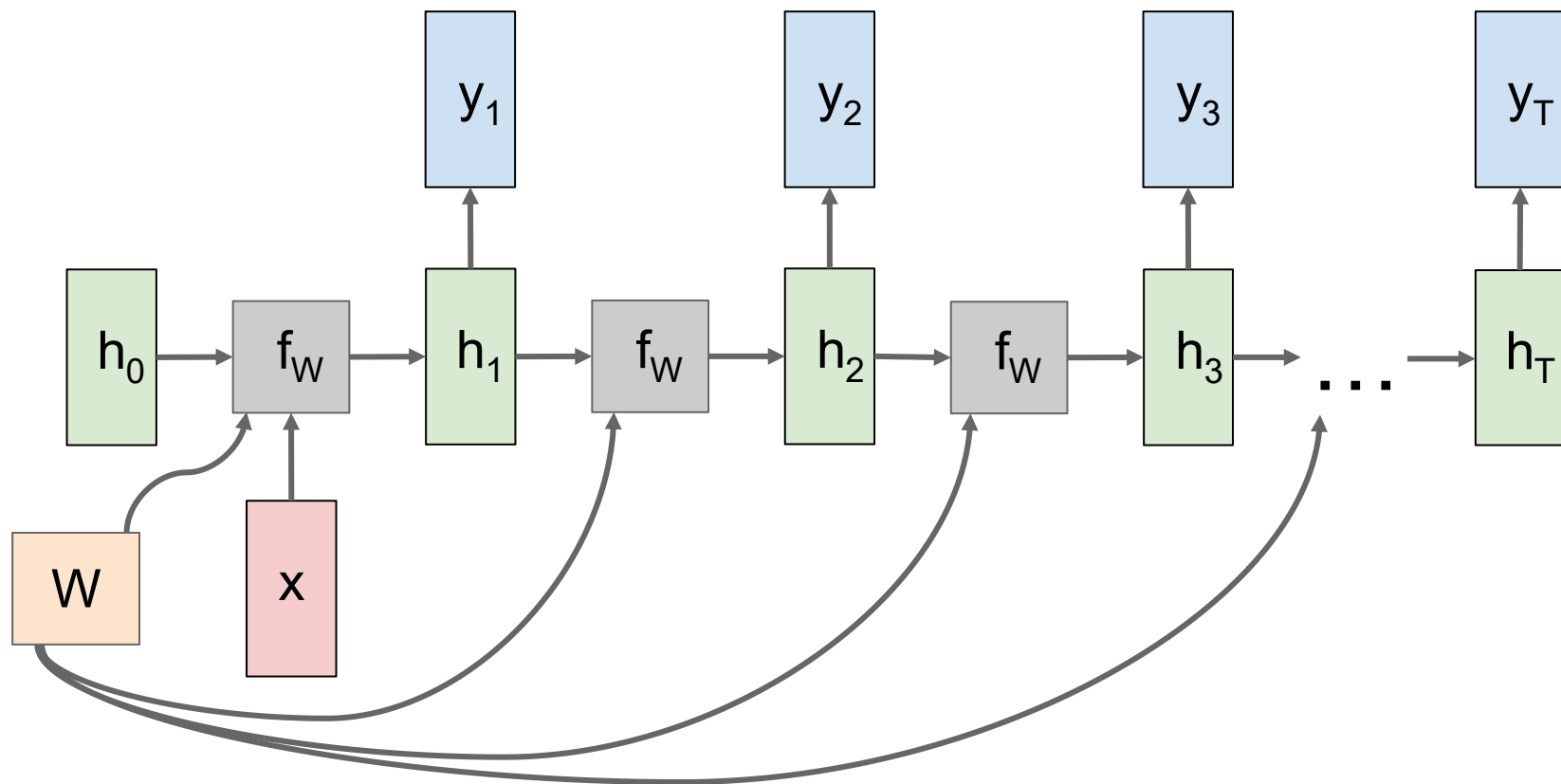
RNN: 计算图: Many to One



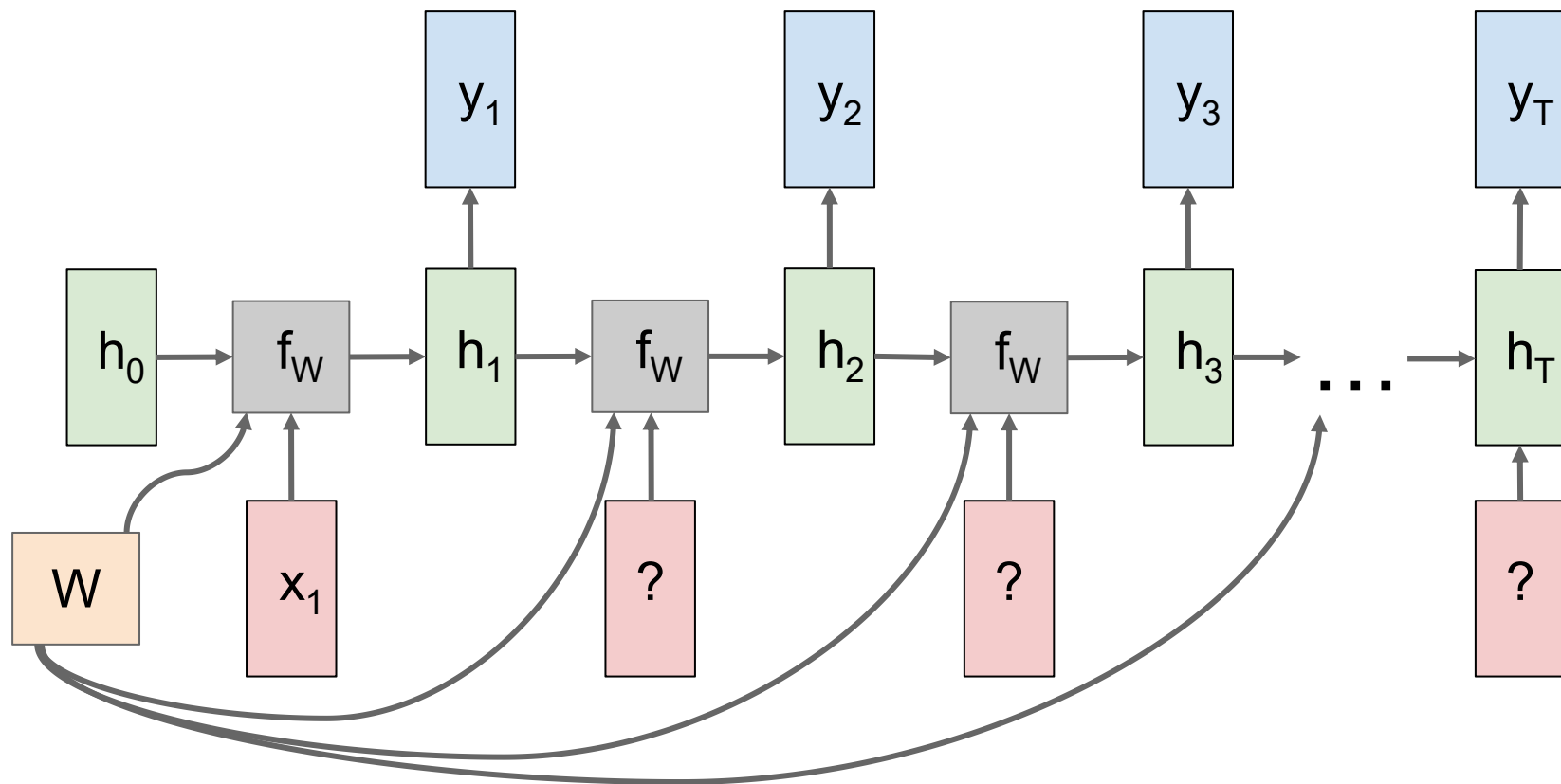
RNN: 计算图: Many to One



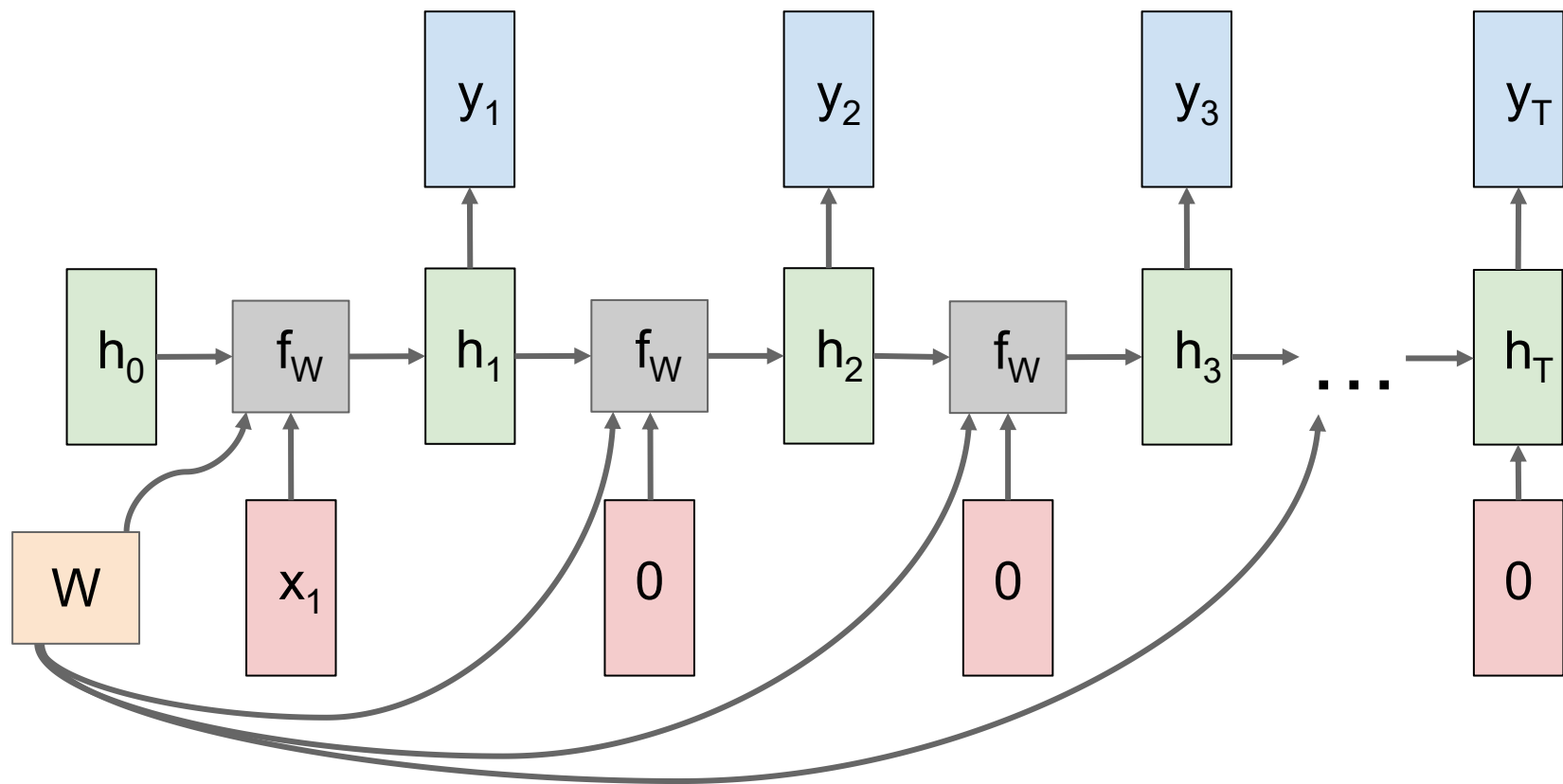
RNN: 计算图: One to Many



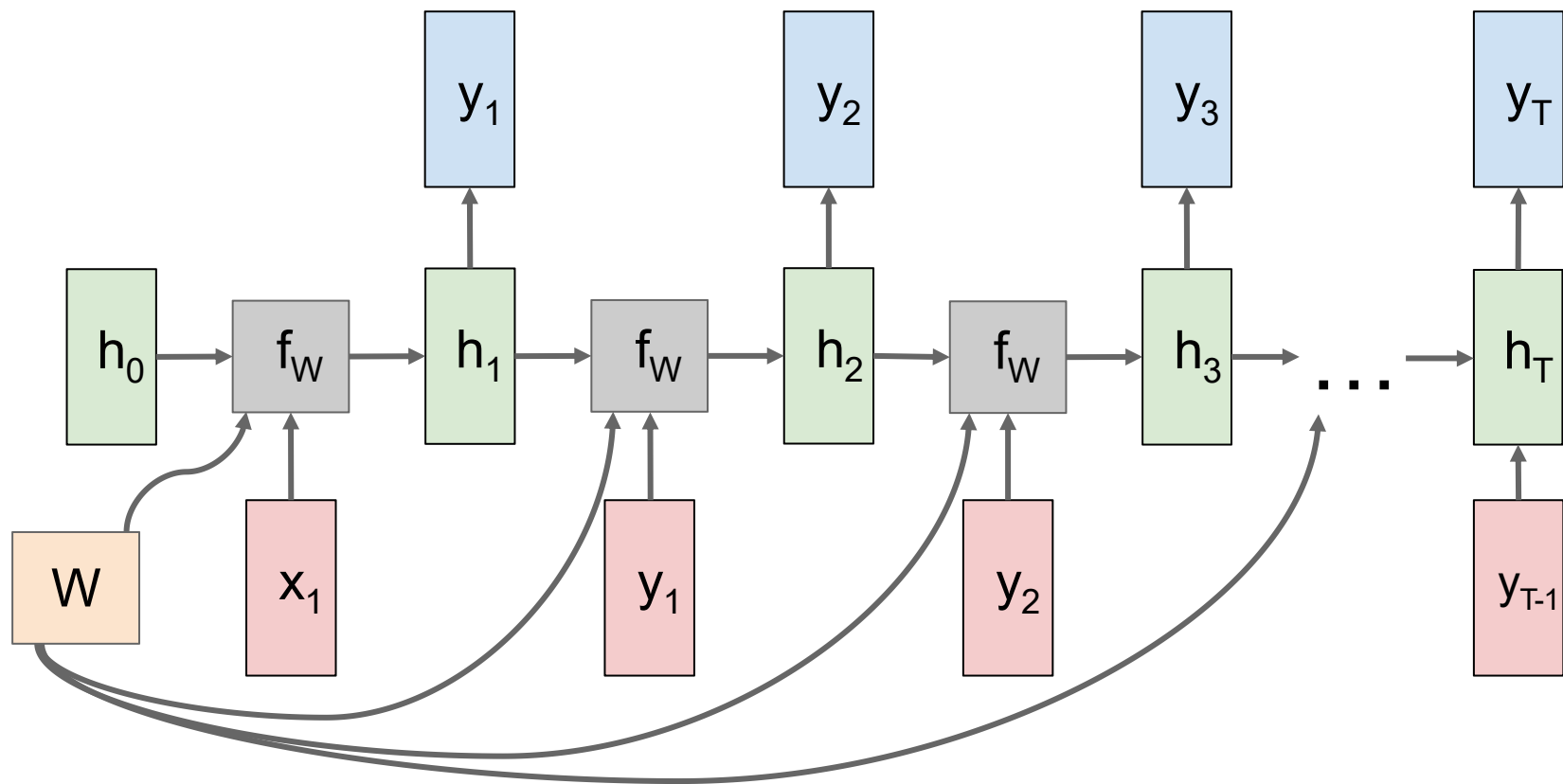
RNN: 计算图: One to Many



RNN: 计算图: One to Many

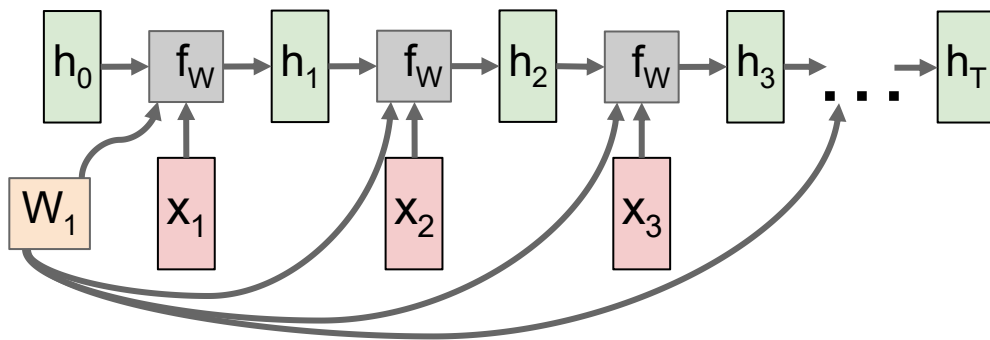


RNN: 计算图: One to Many



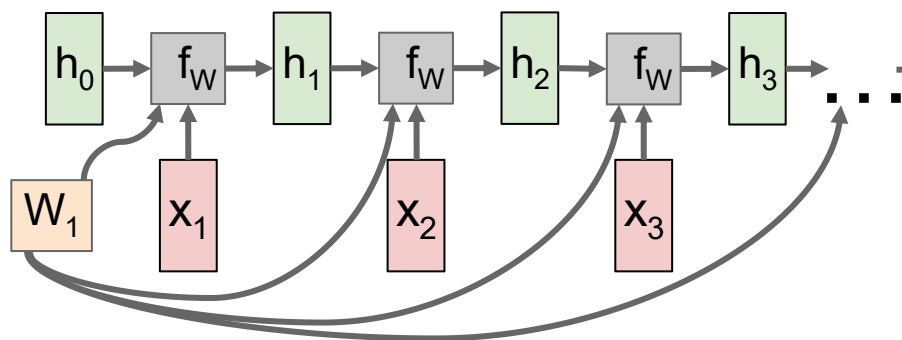
序列到序列: Many to One + One to Many

Many to One: 将输入序列编码成一个单独的向量

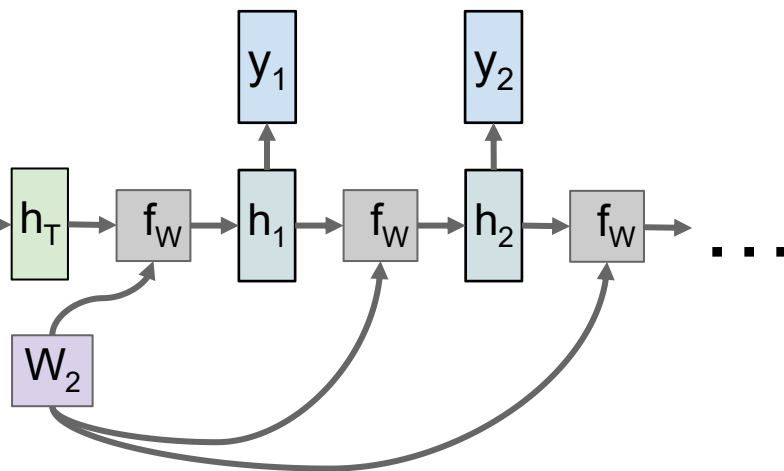


序列到序列: Many to One + One to Many

Many to One: 将输入序列编码成一个单独的向量



One to Many: 从单个输入向量生成输出序列。



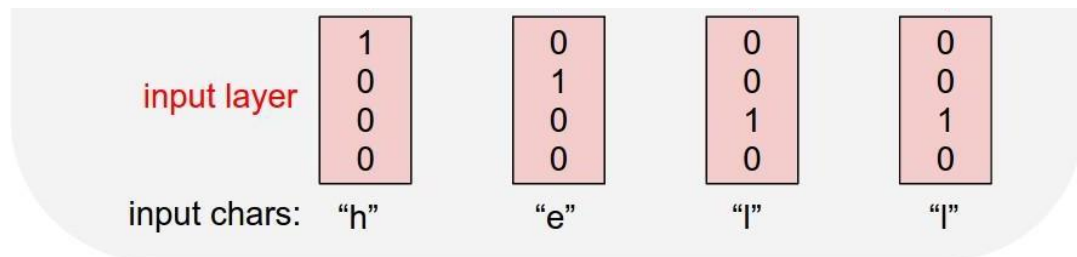
Sutskever et al, "Sequence to Sequence Learning with Neural Networks", NIPS 2014

例子

字符级语言模型

词汇表: [h,e,l,o]

示例训练序列: “hello”



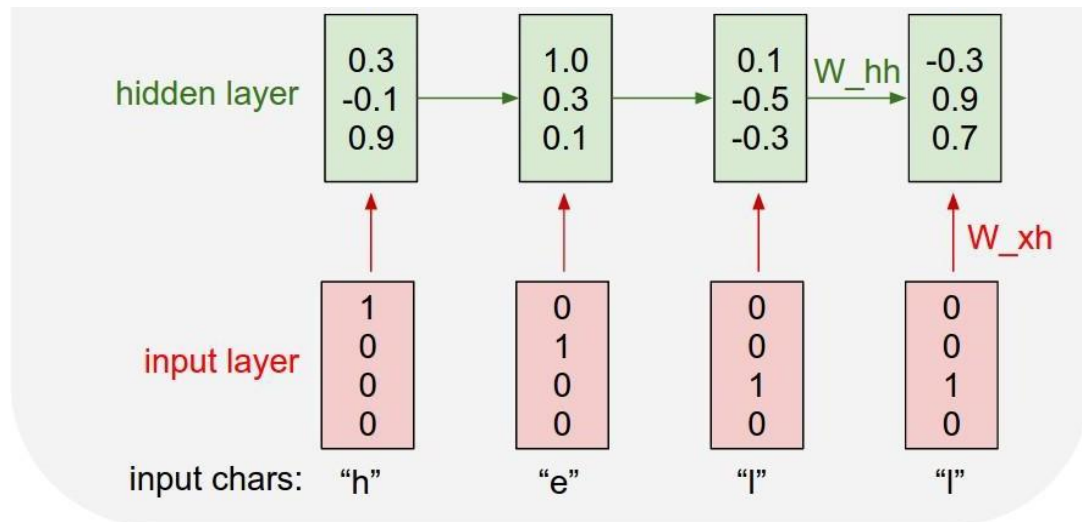
例子

字符级语言模型

词汇表: [h,e,l,o]

示例训练序列: “hello”

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

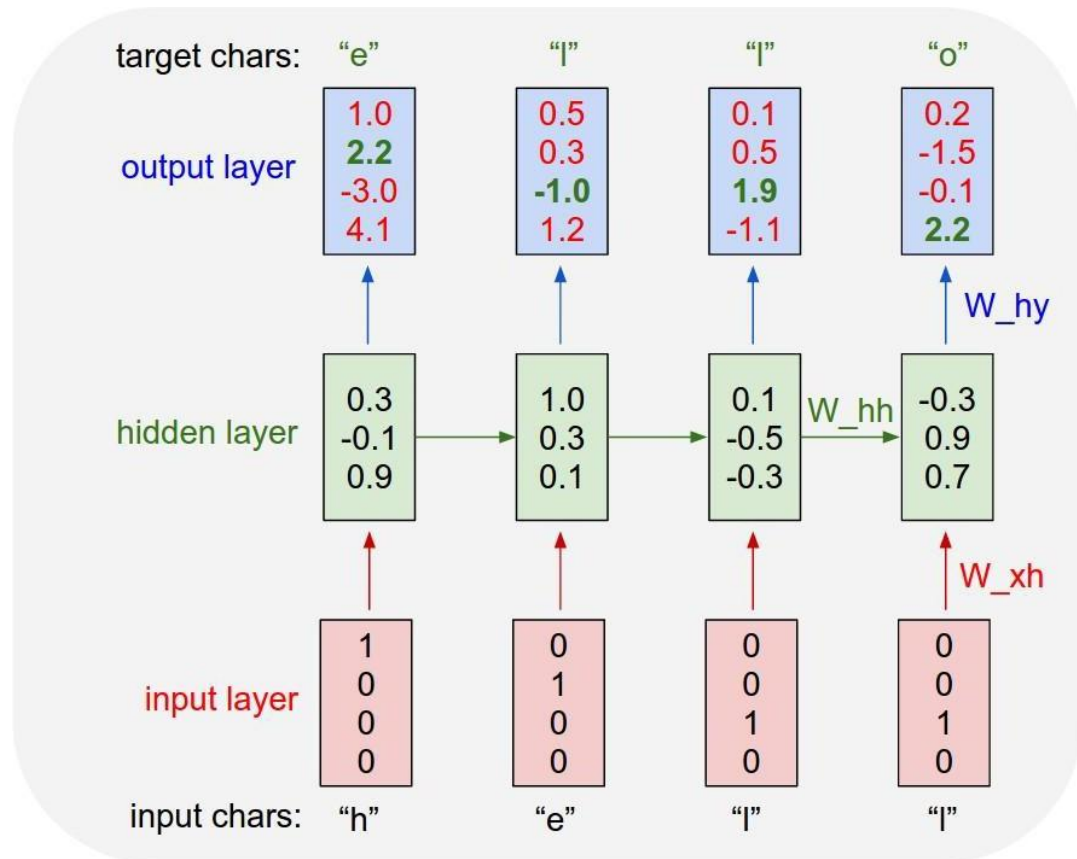


例子

字符级语言模型

词汇表: [h,e,l,o]

示例训练序列: “hello”

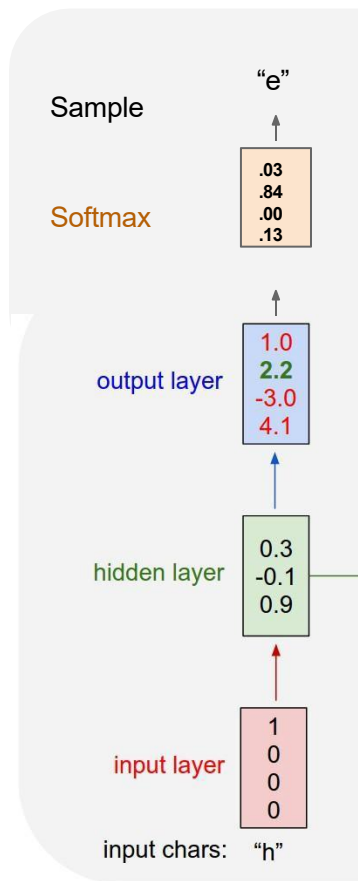


例子

字符级语言模型采样

词汇表: [h,e,l,o]

在测试时，一次采样一个字符，反馈给模型。

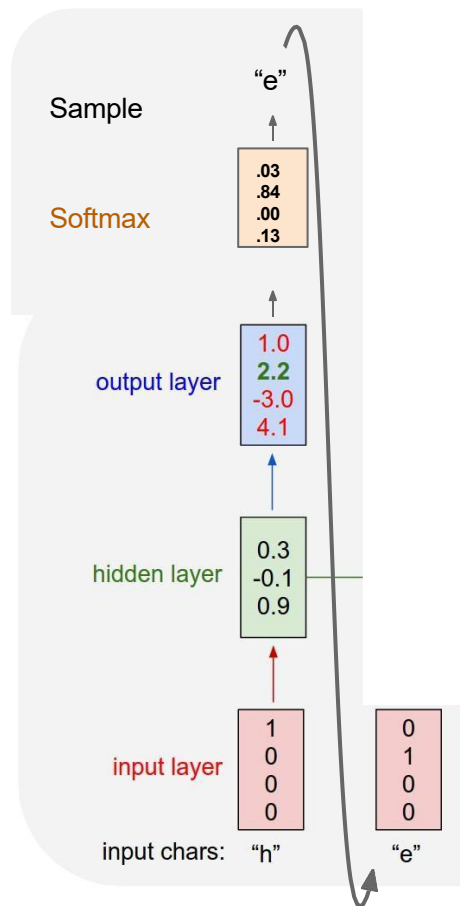


例子

字符级语言模型采样

词汇表: [h,e,l,o]

在测试时，一次采样一个字符，反馈给模型。

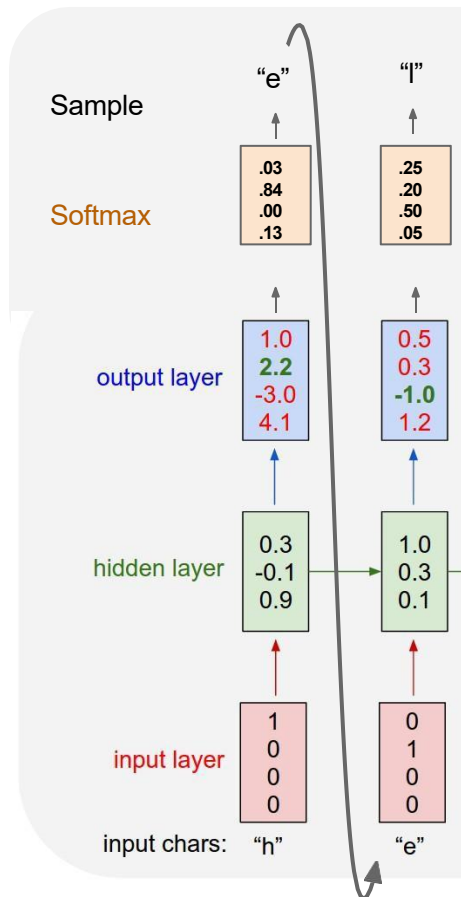


例子

字符级语言模型采样

词汇表: [h,e,l,o]

在测试时，一次采样一个字符，反馈给模型。

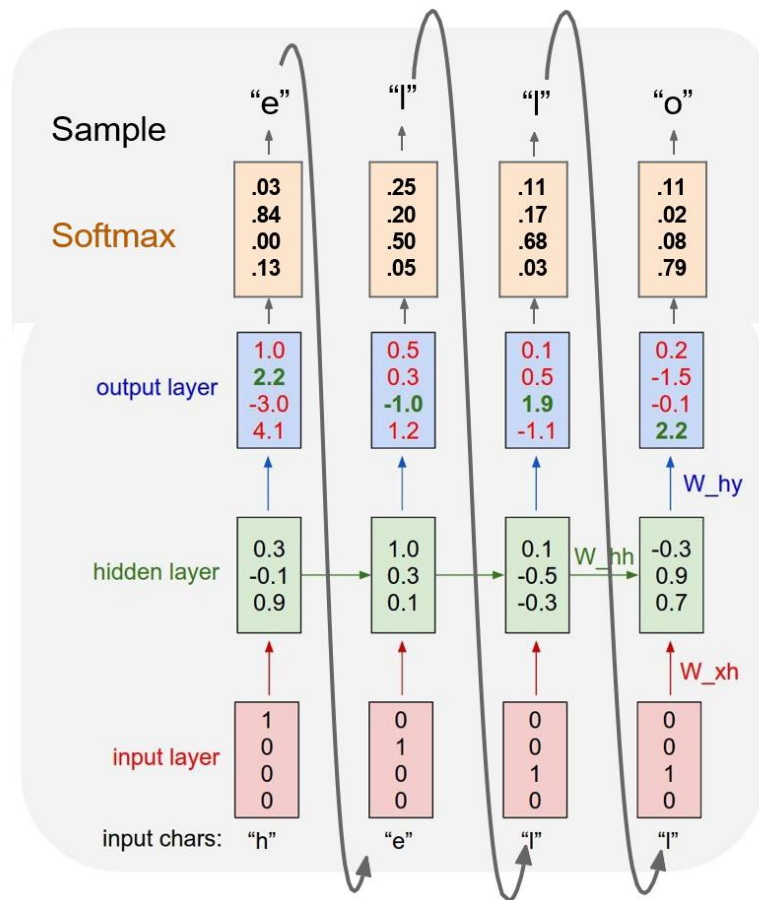


例子

字符级语言模型采样

词汇表: [h,e,l,o]

在测试时，一次采样一个字符，反馈给模型。

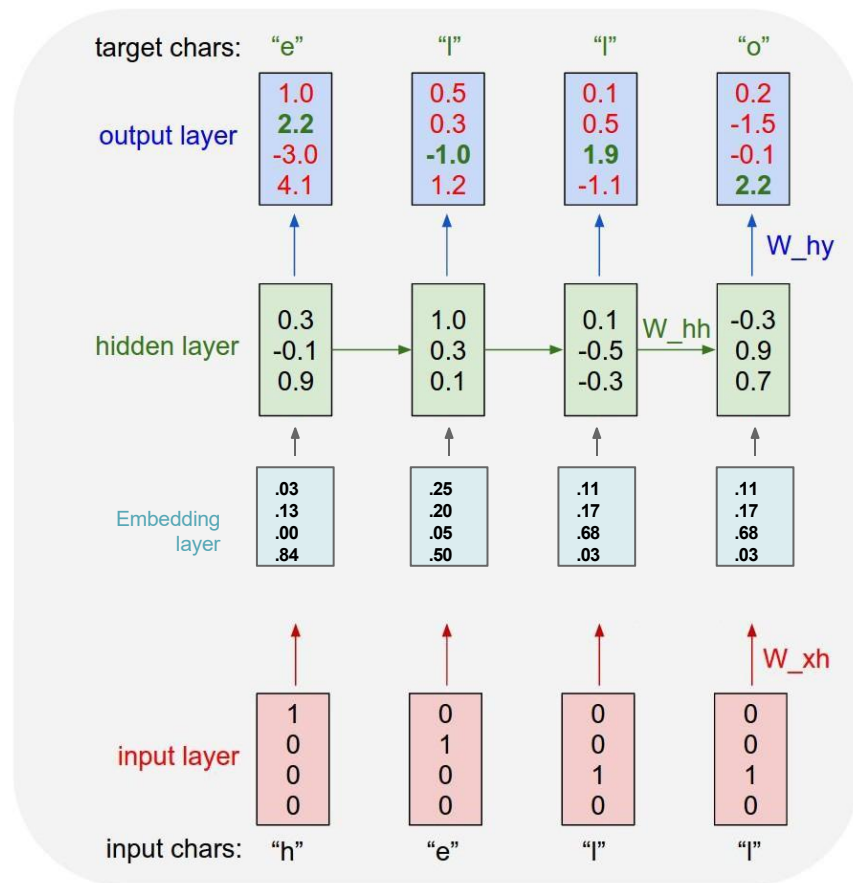


例子

字符级语言模型采样

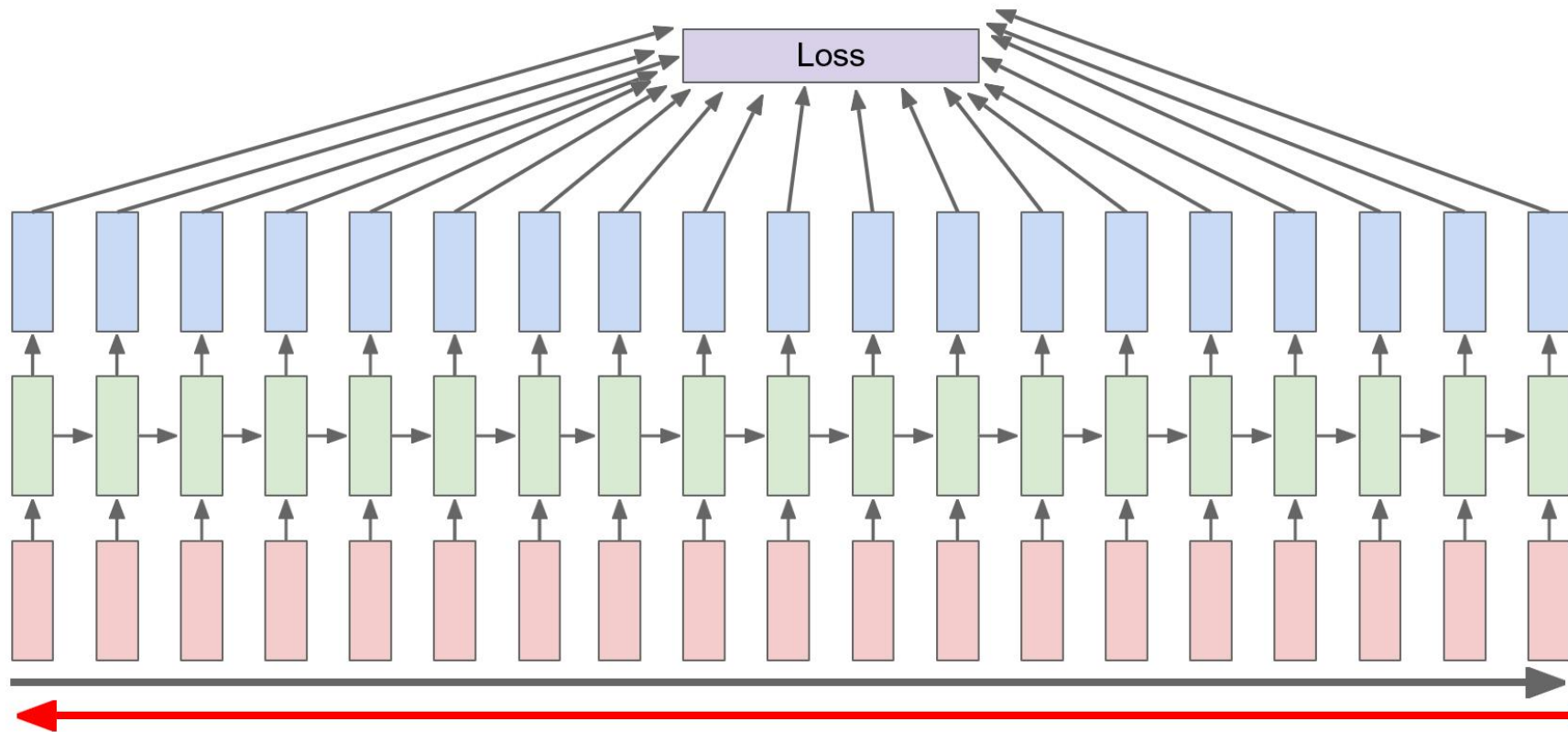
$$\begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \end{bmatrix}$$

一个 one-hot 向量与矩阵相乘，其结果就是提取该矩阵的一列。我们通常在输入层和隐藏层之间放置一个单独的嵌入层。

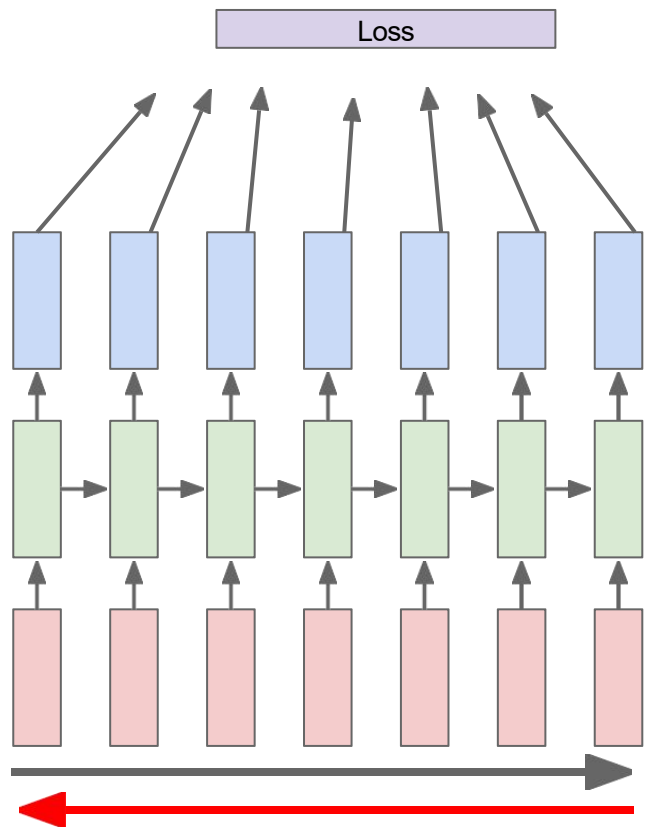


随时间反向传播

前向传播整个序列来计算损失，然后反向传播整个序列来计算梯度。

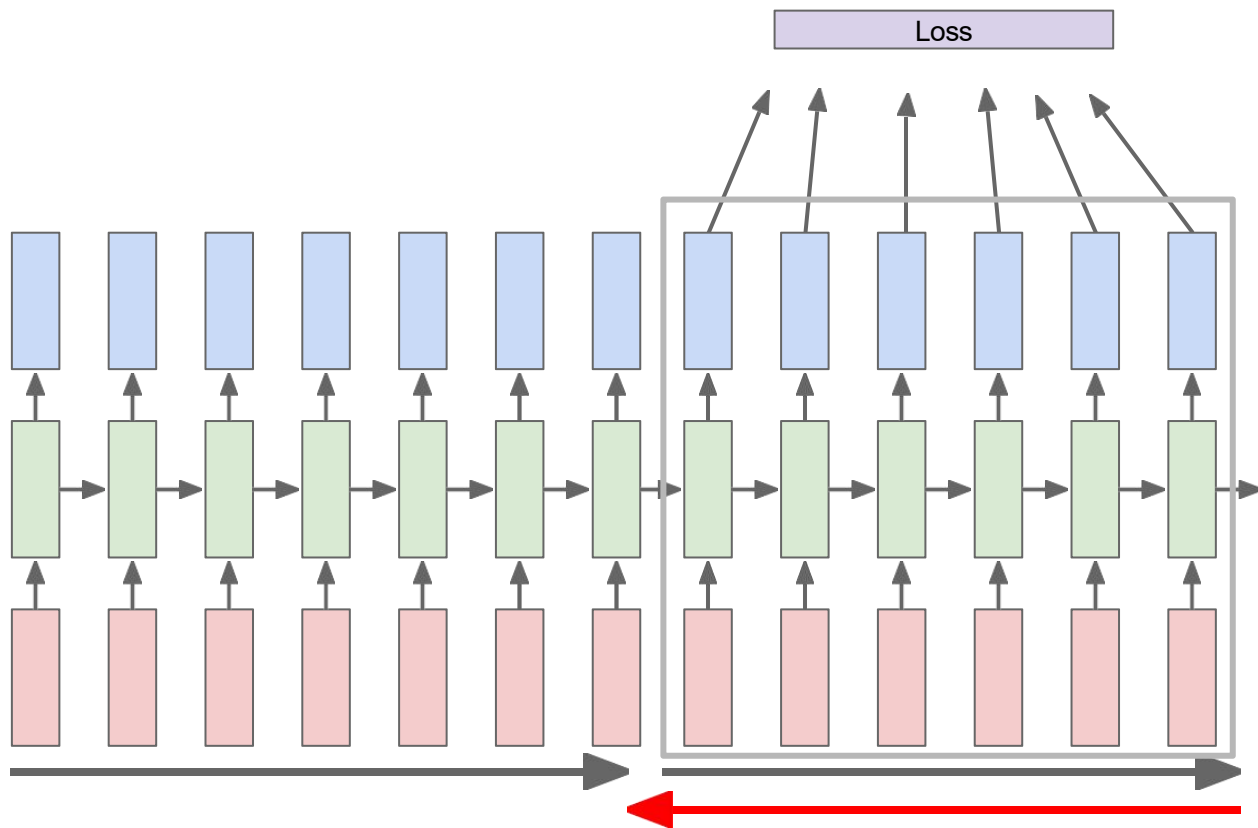


截断的反向传播算法



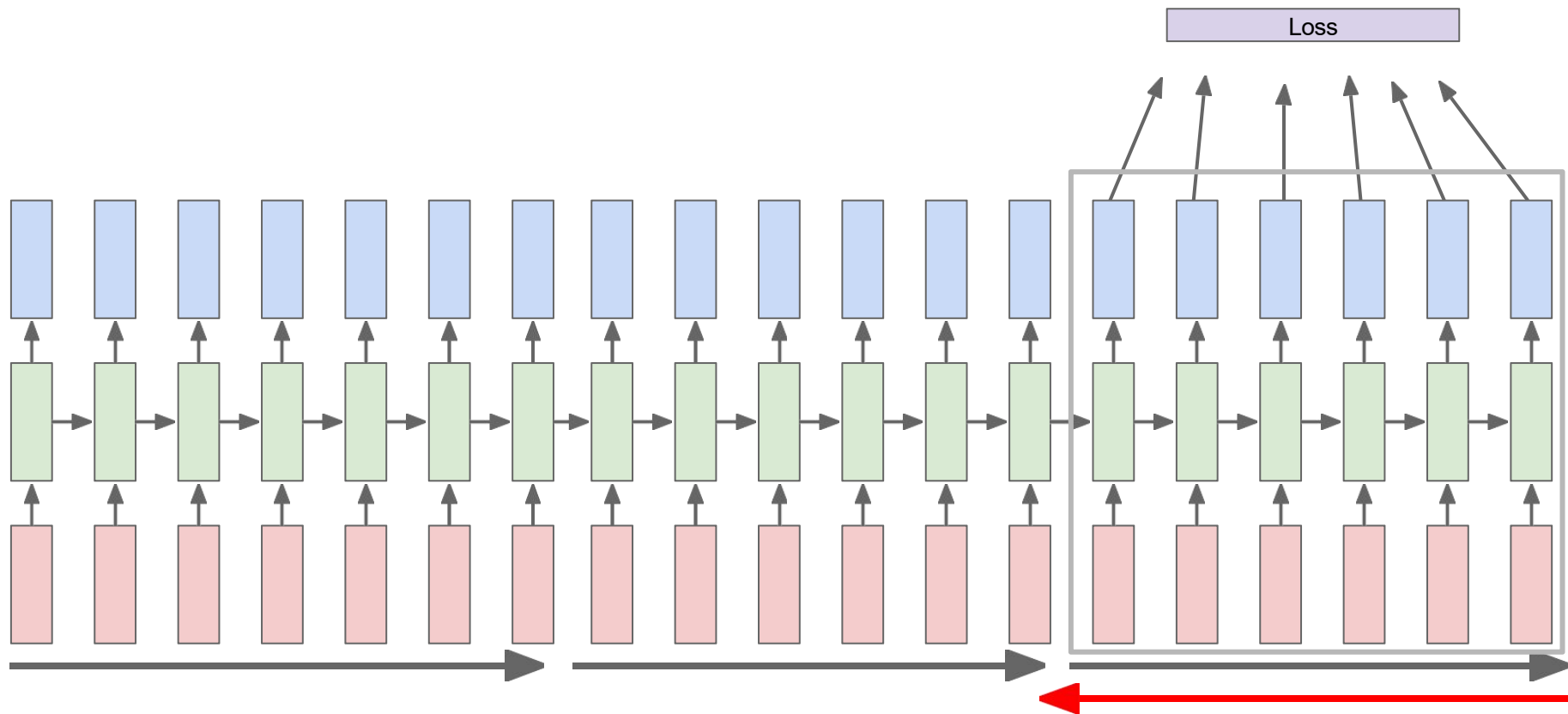
通过序列的若干片段进行前向和反向传播，而不是整个序列。

截断的反向传播算法



在时间上无限地
向前传递隐藏状
态，但只反向传
播有限的步数。

截断的反向传播算法



min-char-rnn.py gist: 112 lines of Python

```
1 """
2 Minimal character-level Vanilla RNN model. Written by Andrej Karpathy (@karpathy)
3 BSD License
4 """
5 import numpy as np
6
7 # data I/O
8 data = open('input.txt', 'r').read() # should be simple plain text file
9 chars = list(set(data))
10 data_size, vocab_size = len(data), len(chars)
11 print 'data has %d characters, %d unique.' % (data_size, vocab_size)
12 char_to_ix = { ch:i for i,ch in enumerate(chars) }
13 ix_to_char = { i:ch for i,ch in enumerate(chars) }
14
15 # hyperparameters
16 hidden_size = 100 # size of hidden layer of neurons
17 seq_length = 25 # number of steps to unroll the RNN for
18 learning_rate = 1e-1
19
20 # model parameters
21 wxh = np.random.randn(hidden_size, vocab_size)*0.01 # input to hidden
22 whh = np.random.randn(hidden_size, hidden_size)*0.01 # hidden to hidden
23 why = np.random.randn(vocab_size, hidden_size)*0.01 # hidden to output
24 bh = np.zeros((hidden_size, 1)) # hidden bias
25 by = np.zeros((vocab_size, 1)) # output bias
26
27 def lossFun(inputs, targets, hprev):
28     """
29     inputs, targets are both list of integers.
30     hprev is Hx1 array of initial hidden state
31     returns the loss, gradients on model parameters, and last hidden state
32     """
33     xs, hs, ys, ps = {}, {}, {}, {}
34     hs[-1] = np.copy(hprev)
35     loss = 0
36     # forward pass
37     for t in xrange(len(inputs)):
38         xs[t] = np.zeros((vocab_size,1)) # encode in 1-of-k representation
39         xs[t][inputs[t]] = 1
40         hs[t] = np.tanh(np.dot(wxh, xs[t]) + np.dot(whh, hs[t-1]) + bh) # hidden state
41         ys[t] = np.dot(why, hs[t]) + by # unnormalized log probabilities for next chars
42         ps[t] = np.exp(ys[t]) / np.sum(np.exp(ys[t])) # probabilities for next chars
43         loss += -np.log(ps[t][targets[t],0]) # softmax (cross-entropy loss)
44     # backward pass: compute gradients going backwards
45     dwxh, dwhh, dwhy = np.zeros_like(wxh), np.zeros_like(whh), np.zeros_like(why)
46     dbh, dby = np.zeros_like(bh), np.zeros_like(by)
47     dhnext = np.zeros_like(hs[0])
48     for t in reversed(xrange(len(inputs))):
49         dy = np.copy(ps[t])
50         dy[targets[t]] -= 1 # backprop into y
51         dwhy += np.dot(dy, hs[t].T)
52         dby += dy
53         dh = np.dot(why.T, dy) + dhnext # backprop into h
54         dhrw = (1 - hs[t] * hs[t]) * dh # backprop through tanh nonlinearity
55         dbh += dhrw
56         dwxh += np.dot(dhrw, xs[t].T)
57         dwhh += np.dot(dhrw, hs[t-1].T)
58         dhnext = np.dot(whh.T, dhrw)
59     for dparam in [dwxh, dwhh, dwhy, dbh, dby]:
60         np.clip(dparam, -5, 5, out=dparam) # clip to mitigate exploding gradients
61     return loss, dwxh, dwhh, dwhy, dbh, dby, hs[len(inputs)-1]
62
63 def sample(h, seed_ix, n):
64     """
65     sample a sequence of integers from the model
66     h is memory state, seed_ix is seed letter for first time step
67     """
68     x = np.zeros((vocab_size, 1))
69     x[seed_ix] = 1
70     ixes = []
71     for t in xrange(n):
72         h = np.tanh(np.dot(wxh, x) + np.dot(whh, h) + bh)
73         y = np.dot(why, h) + by
74         p = np.exp(y) / np.sum(np.exp(y))
75         ix = np.random.choice(range(vocab_size), p=p.ravel())
76         x = np.zeros((vocab_size, 1))
77         x[ix] = 1
78         ixes.append(ix)
79     return ixes
80
81 n, p = 0, 0
82 mxh, mwhh, mwhy = np.zeros_like(wxh), np.zeros_like(whh), np.zeros_like(why)
83 mbh, mby = np.zeros_like(bh), np.zeros_like(by) # memory variables for Adagrad
84 smooth_loss = -np.log(1.0/vocab_size)*seq_length # loss at iteration 0
85 while True:
86     # prepare inputs (we're sweeping from left to right in steps seq_length long)
87     if p+seq_length+1 >= len(data) or n == 0:
88         hprev = np.zeros((hidden_size,1)) # reset RNN memory
89         p = 0 # go from start of data
90     inputs = [char_to_ix[ch] for ch in data[p:p+seq_length]]
91     targets = [char_to_ix[ch] for ch in data[p+1:p+seq_length+1]]
92
93     # sample from the model now and then
94     if n % 100 == 0:
95         sample_ix = sample(hprev, inputs[0], 200)
96         txt = ''.join(ix_to_char[ix] for ix in sample_ix)
97         print '----\n %s \n----' % (txt, )
98
99     # forward seq_length characters through the net and fetch gradient
100     loss, dwxh, dwhh, dwhy, dbh, dby, hprev = lossFun(inputs, targets, hprev)
101     smooth_loss = smooth_loss * 0.999 + loss * 0.001
102     if n % 100 == 0: print 'iter %d, loss: %f' % (n, smooth_loss) # print progress
103
104     # perform parameter update with Adagrad
105     for param, dparam, mem in zip([wxh, whh, why, bh, by],
106                                   [dwxh, dwhh, dwhy, dbh, dby],
107                                   [mxh, mwhh, mwhy, mbh, mby]):
108         mem += dparam * dparam
109         param += -learning_rate * dparam / np.sqrt(mem + 1e-8) # adagrad update
110
111     p += seq_length # move data pointer
112     n += 1 # iteration counter
```

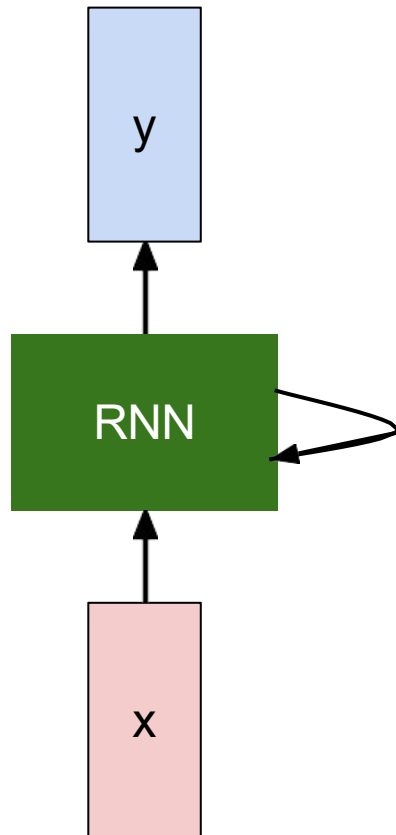
(<https://gist.github.com/karpathy/d4dee566867f8291f086>)

THE SONNETS

by William Shakespeare

From fairest creatures we desire increase,
That thereby beauty's rose might never die,
But as the ripper should by time decease,
His tender heir might bear his memory:
But thou, contracted to thine own bright eyes,
Feed'st thy light's flame with self-substantial fuel,
Making a famine where abundance lies,
Thyself thy foe, to thy sweet self too cruel:
Thou that art now the world's fresh ornament,
And only herald to the gaudy spring,
Within thine own bud buriest thy content,
And tender churl mak'st waste in niggarding:
Pity the world, or else this glutton be,
To eat the world's due, by the grave and thee.

When forty winters shall besiege thy brow,
And dig deep trenches in thy beauty's field,
Thy youth's proud livery so gazed on now,
Will be a tatter'd weed of small worth held:
Then being asked, where all thy beauty lies,
Where all the treasure of thy lusty days;
To say, within thine own deep sunken eyes,
Were an all-eating shame, and thriftless praise.
How much more praise deserv'd thy beauty's use,
If thou couldst answer 'This fair child of mine
Shall sum my count, and make my old excuse,'
Proving his beauty by succession thine!
This were to be new made when thou art old,
And see thy blood warm when thou feel'st it cold.



例

一开始:

tyntd-iafhatawiaoihrdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrqd t o idoe ns,smtt h ne etie h,hregtrs nigtike,aoaenns lng

训练

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuw y fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."

训练

Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and ofter.

训练

"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftened him.
Pierre aking his soul came to the packs and drove up his father-in-law women.

练习题

- **题目：RNN的隐藏状态计算依赖于？**

- A. 仅当前输入
- B. 当前输入和上一时刻隐藏状态
- C. 所有历史输入的平均值
- D. 未来时刻的隐藏状态

- **答案：B**

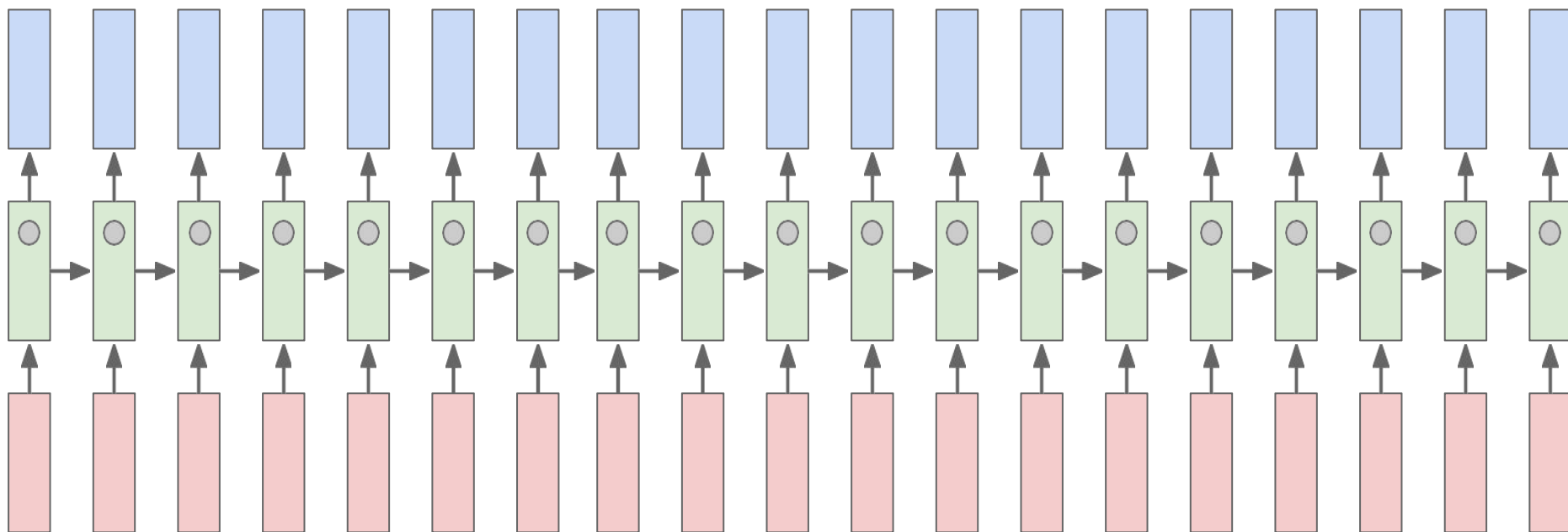
练习题

- **题目：RNN梯度消失问题的主要原因是？**

- A. 激活函数ReLU的死亡神经元
- B. 权重矩阵的连乘导致梯度指数级衰减
- C. 输入数据归一化不足
- D. 批量大小过大

- **答案：B**

寻找可解释的细胞单元



寻找可解释的细胞单元

```
/* Unpack a filter field's string representation from user-space
 * buffer. */
char *audit_unpack_string(void **bufp, size_t *remain, size_t len)
{
    char *str;
    if (!*bufp || (len == 0) || (len > *remain))
        return ERR_PTR(-EINVAL);
    /* Of the currently implemented string fields, PATH_MAX
     * defines the longest valid length.
     */
}
```

Karpathy, Johnson, and Fei-Fei: Visualizing and Understanding Recurrent Networks, ICLR Workshop 2016
Figures copyright Karpathy, Johnson, and Fei-Fei, 2015; reproduced with permission

寻找可解释的细胞单元

"You mean to imply that I have nothing to eat out of.... On the contrary, I can supply you with everything even if you want to give dinner parties," warmly replied Chichagov, who tried by every word he spoke to prove his own rectitude and therefore imagined Kutuzov to be animated by the same desire.

Kutuzov, shrugging his shoulders, replied with his subtle penetrating smile: "I meant merely to say what I said."

引号检测单元

Karpathy, Johnson, and Fei-Fei: Visualizing and Understanding Recurrent Networks, ICLR Workshop 2016
Figures copyright Karpathy, Johnson, and Fei-Fei, 2015; reproduced with permission

寻找可解释的细胞单元

对行内位置敏感的单元:

The sole importance of the crossing of the Berezina lies in the fact that it plainly and indubitably proved the fallacy of all the plans for cutting off the enemy's retreat and the soundness of the only possible line of action--the one Kutuzov and the general mass of the army demanded--namely, simply to follow the enemy up. The French crowd fled at a continually increasing speed and all its energy was directed to reaching its goal. It fled like a wounded animal and it was impossible to block its path. This was shown not so much by the arrangements it made for crossing as by what took place at the bridges. When the bridges broke down, unarmed soldiers, people from Moscow and women with children who were with the French transport, all--carried on by vis inertiae--pressed forward into boats and into the ice-covered water and did not, surrender.

行长度跟踪单元

Karpathy, Johnson, and Fei-Fei: Visualizing and Understanding Recurrent Networks, ICLR Workshop 2016
Figures copyright Karpathy, Johnson, and Fei-Fei, 2015; reproduced with permission

寻找可解释的细胞单元

```
static int __dequeue_signal(struct sigpending *pending, sigset_t *mask,
                           siginfo_t *info)
{
    int sig = next_signal(pending, mask);
    if (sig) {
        if (current->notifier) {
            if (sigismember(current->notifier_mask, sig)) {
                if (!(current->notifier)(current->notifier_data)) {
                    clear_thread_flag(TIF_SIGPENDING);
                    return 0;
                }
            }
        }
        collect_signal(sig, pending, info);
    }
    return sig;
}
```

条件判断单元

Karpathy, Johnson, and Fei-Fei: Visualizing and Understanding Recurrent Networks, ICLR Workshop 2016
Figures copyright Karpathy, Johnson, and Fei-Fei, 2015; reproduced with permission

寻找可解释的细胞单元

在注释和引号内部激活的单元:

```
/* Duplicate LSM field information. The lsm_rule is opaque, so
 * re-initialized. */
static inline int audit_dupe_lsm_field(struct audit_field *df,
                                       struct audit_field *sf)
{
    int ret = 0;
    char *lsm_str;
    /* our own copy of lsm_str */
    lsm_str = kstrdup(sf->lsm_str, GFP_KERNEL);
    if (unlikely(!lsm_str))
        return -ENOMEM;
    df->lsm_str = lsm_str;
    /* our own (refreshed) copy of lsm_rule */
    ret = security_audit_rule_init(df->type, df->op, df->lsm_str,
                                   (void *)&df->lsm_rule);
    /* Keep currently invalid fields around in case they
     * become valid after a policy reload. */
    if (ret == -EINVAL) {
        pr_warn("audit rule for LSM '%s' is invalid\n",
                df->lsm_str);
        ret = 0;
    }
    return ret;
}
```

引号/注释单元

Karpathy, Johnson, and Fei-Fei: Visualizing and Understanding Recurrent Networks, ICLR Workshop 2016
Figures copyright Karpathy, Johnson, and Fei-Fei, 2015; reproduced with permission

寻找可解释的细胞单元

```
#ifdef CONFIG_AUDITSYSCALL
static inline int audit_match_class_bits(int class, u32 *mask)
{
    int i;
    if (classes[class]) {
        for (i = 0; i < AUDIT_BITMASK_SIZE; i++)
            if (mask[i] & classes[class][i])
                return 0;
    }
    return 1;
}
```

代码深度单元

Karpathy, Johnson, and Fei-Fei: Visualizing and Understanding Recurrent Networks, ICLR Workshop 2016
Figures copyright Karpathy, Johnson, and Fei-Fei, 2015; reproduced with permission

RNN的优劣

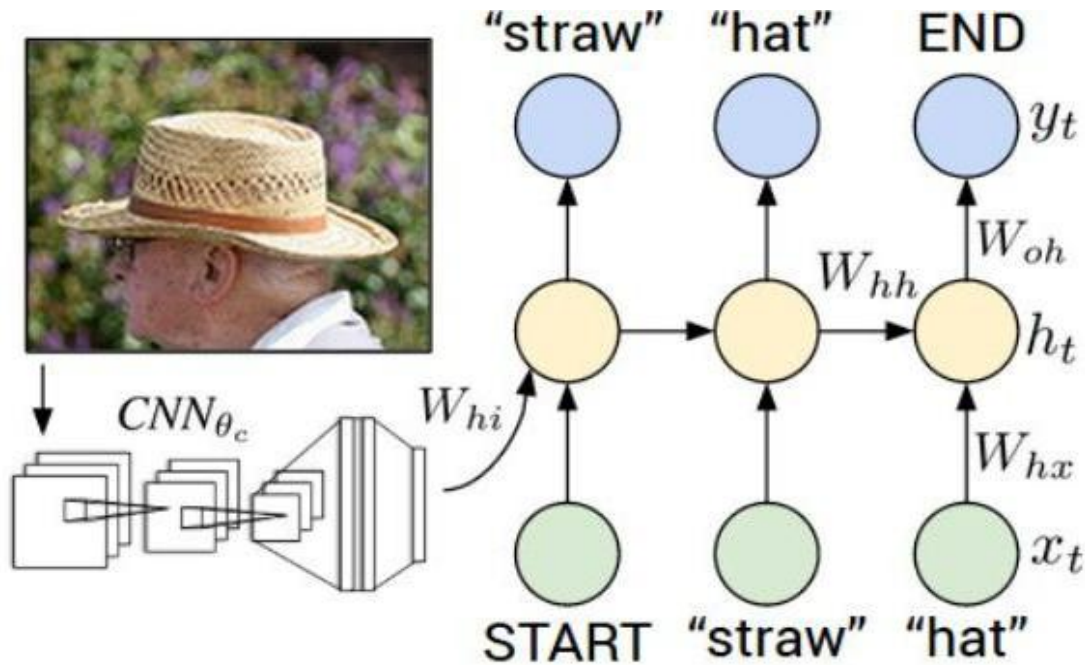
RNN的优点：

- 可以处理任意长度的输入
- 在第 t 步的计算（理论上）可以利用多个之前步骤的信息
- 输入越长，模型大小不增加
- 每个时间步使用相同的权重，保证输入处理方式的一致性

RNN的缺点：

- 递归计算速度较慢
- 实际中很难访问很多步之前的信息

Image Captioning

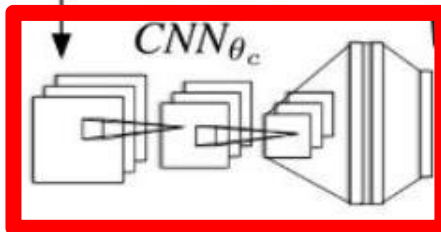


Explain Images with Multimodal Recurrent Neural Networks, Mao et al.
Deep Visual-Semantic Alignments for Generating Image Descriptions, Karpathy and Fei-Fei
Show and Tell: A Neural Image Caption Generator, Vinyals et al.
Long-term Recurrent Convolutional Networks for Visual Recognition and Description, Donahue et al.
Learning a Recurrent Visual Representation for Image Caption Generation, Chen and Zitnick

Figure from Karpathy et al, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015; figure copyright IEEE, 2015. Reproduced for educational purposes.

Image Captioning

RNN



CNN

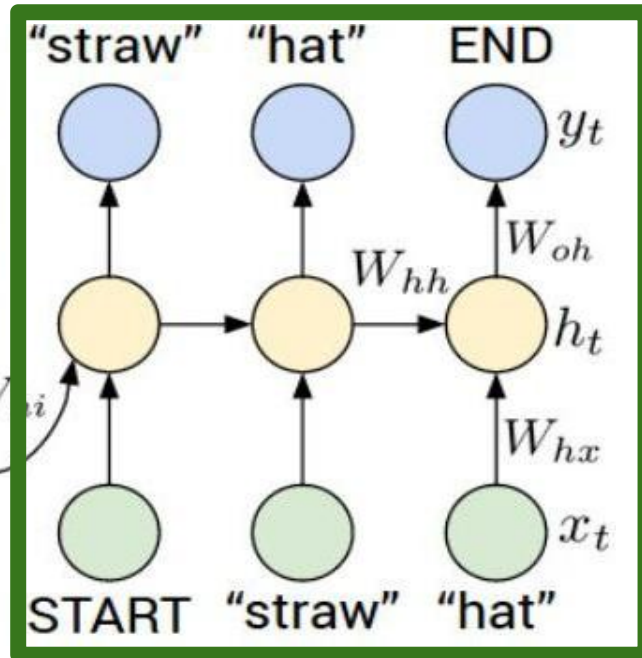
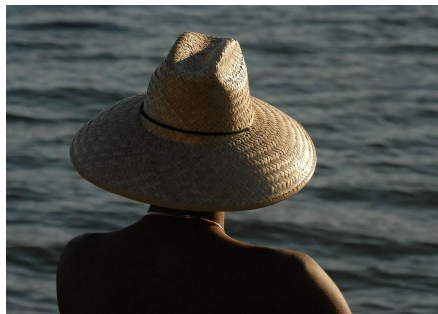


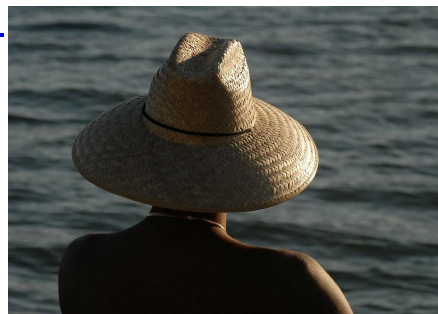
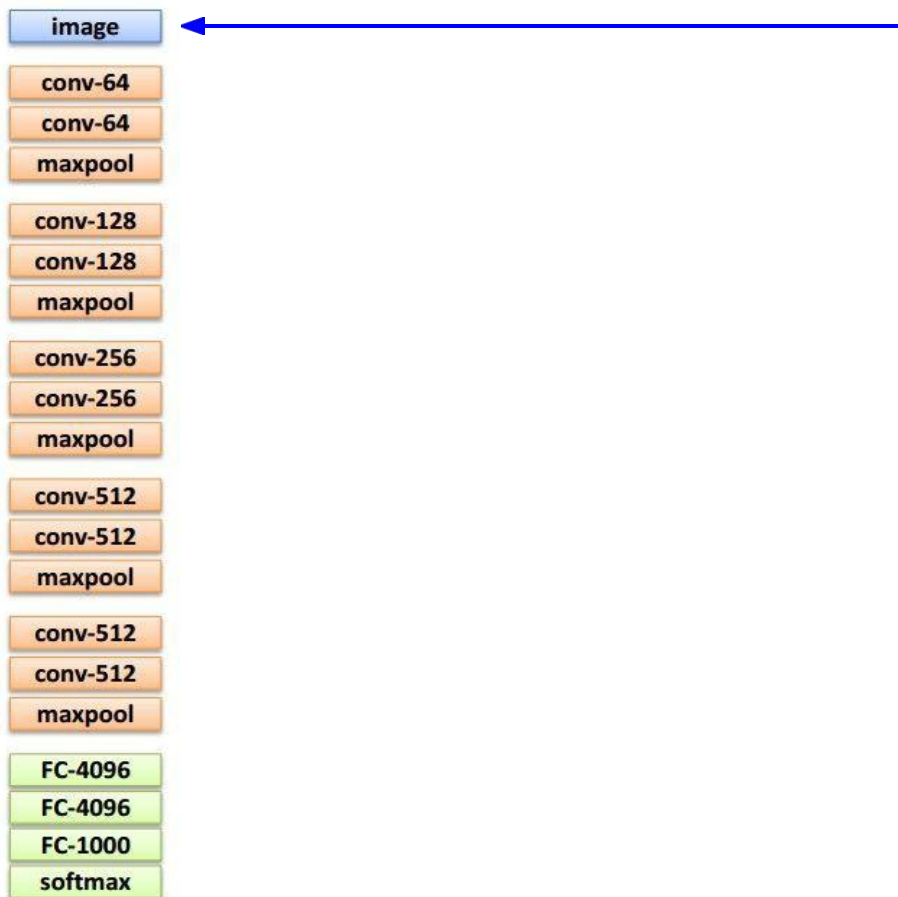
Image Captioning



test image

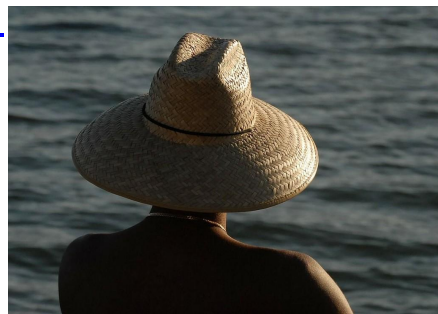
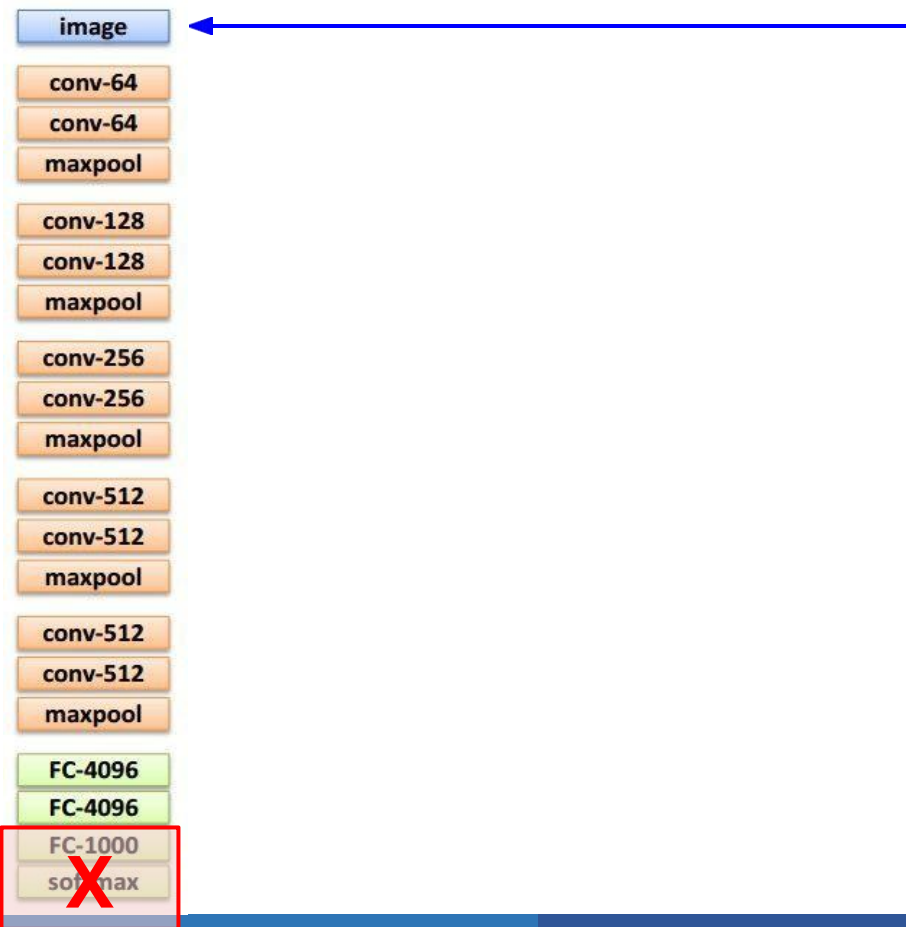
[This image](#) is [CC0 public domain](#)

Image Captioning



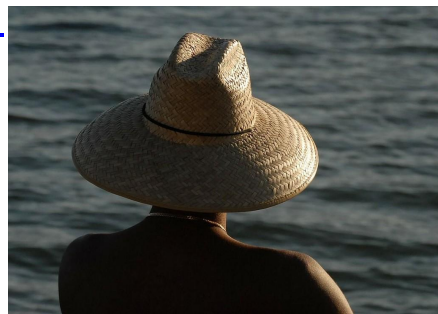
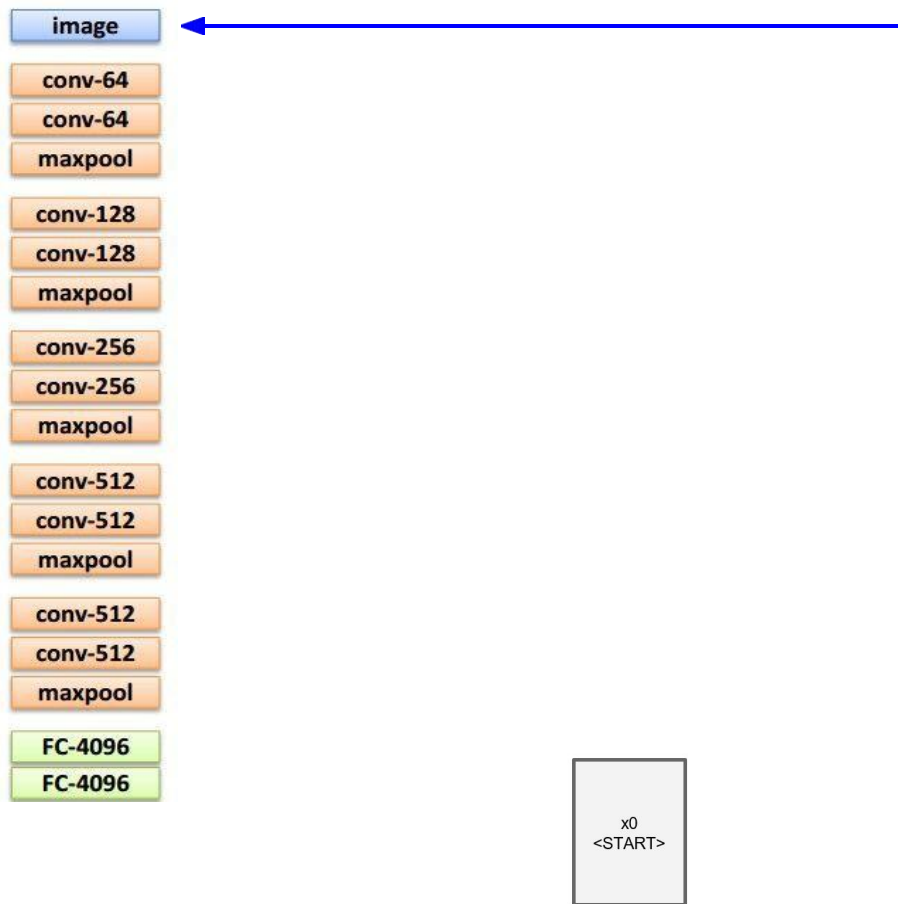
test image

Image Captioning



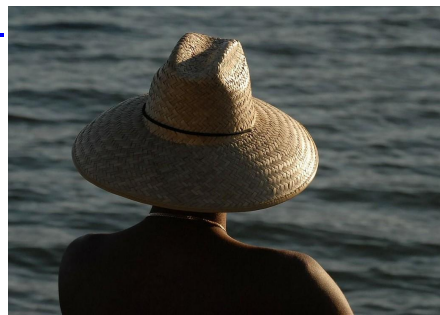
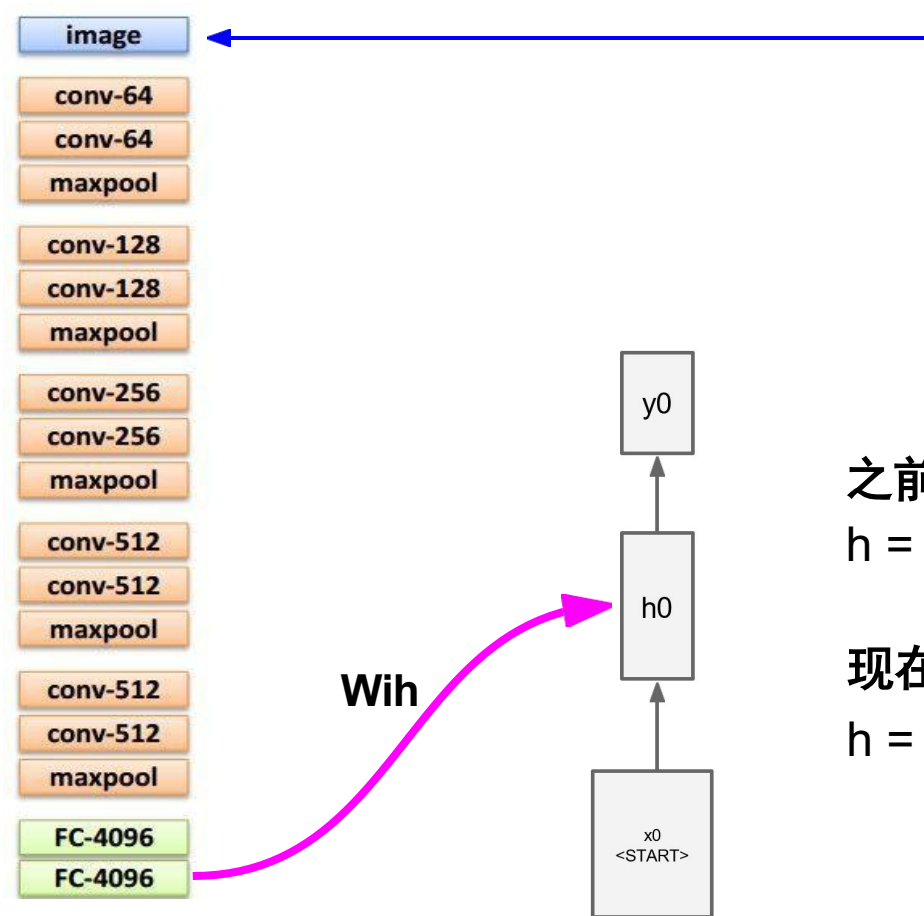
test image

Image Captioning



test image

Image Captioning



test image

之前:

$$h = \tanh(W_{xh} * x + W_{hh} * h)$$

现在:

$$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$$

Image Captioning

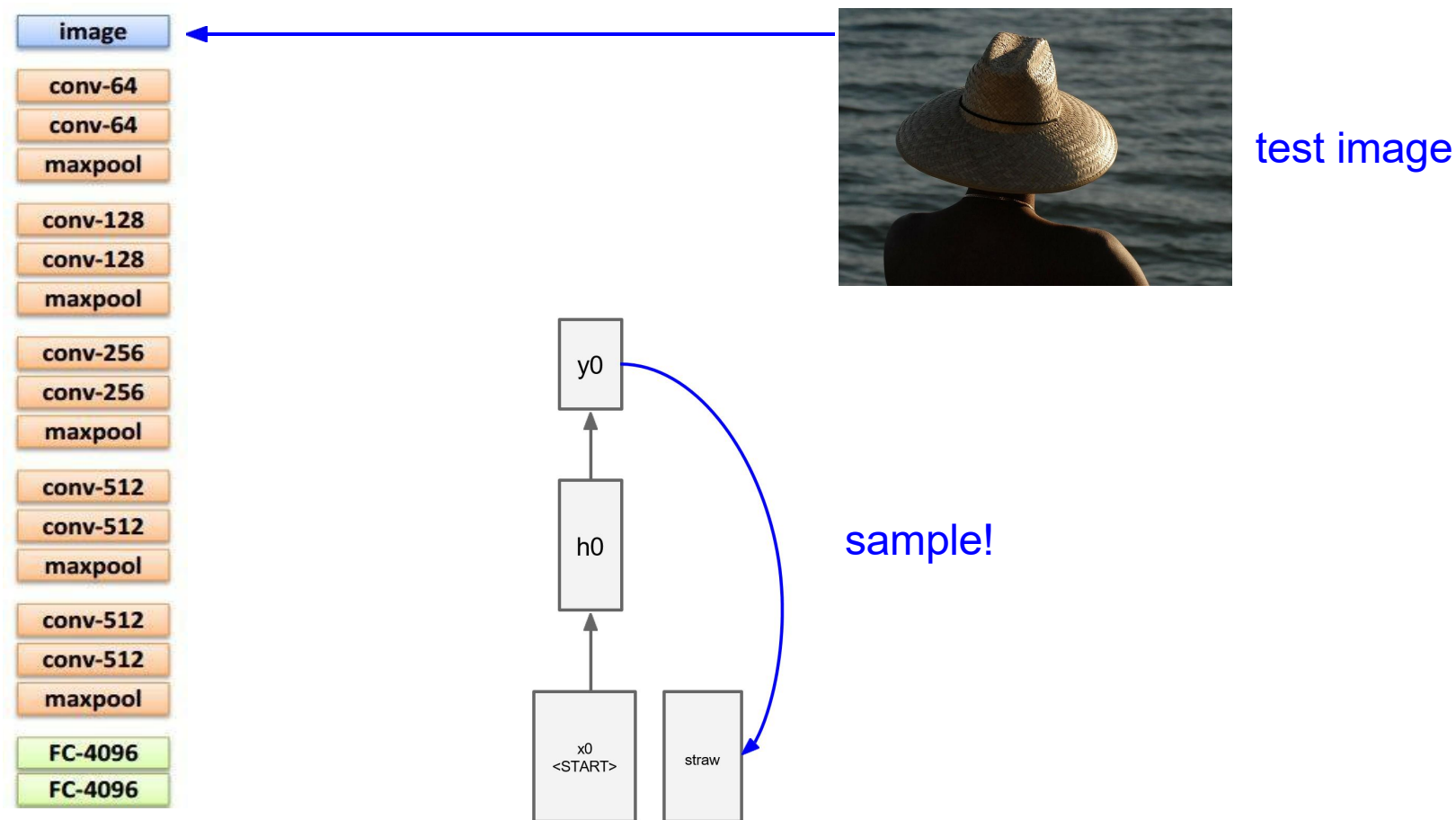
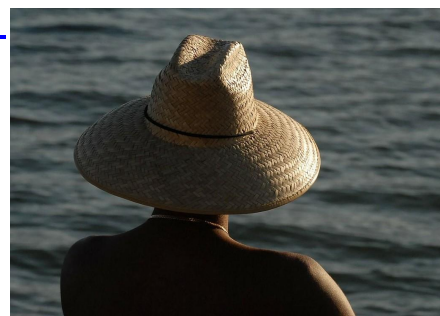
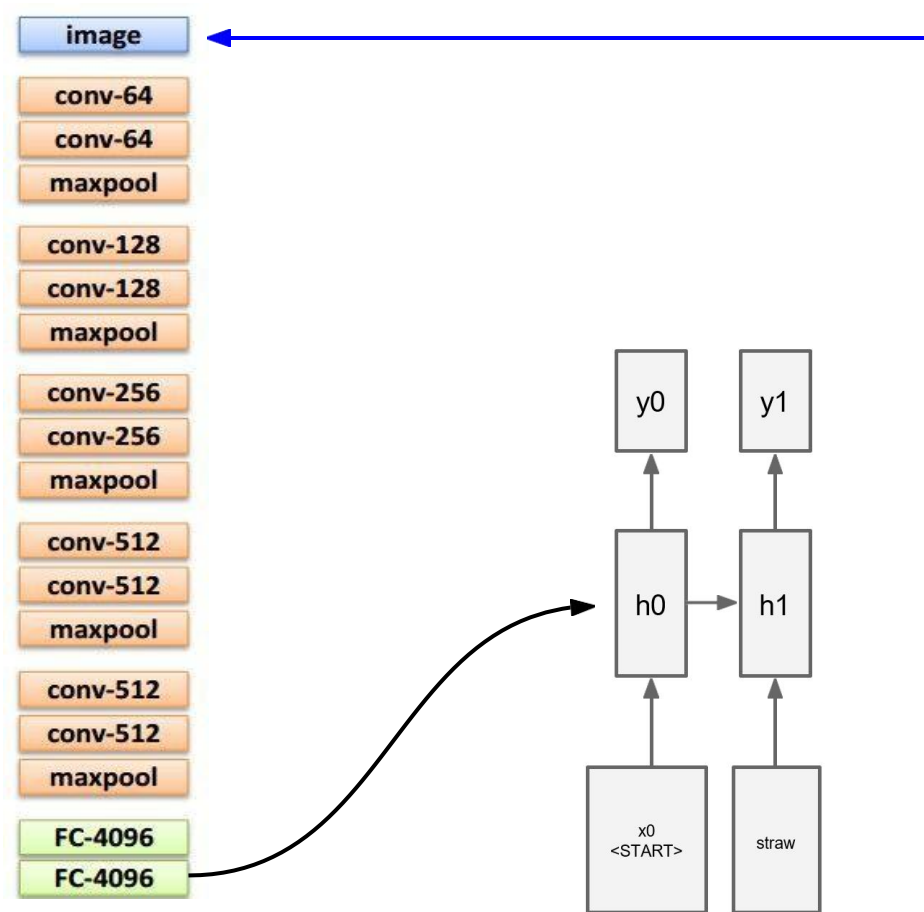


Image Captioning



test image

Image Captioning

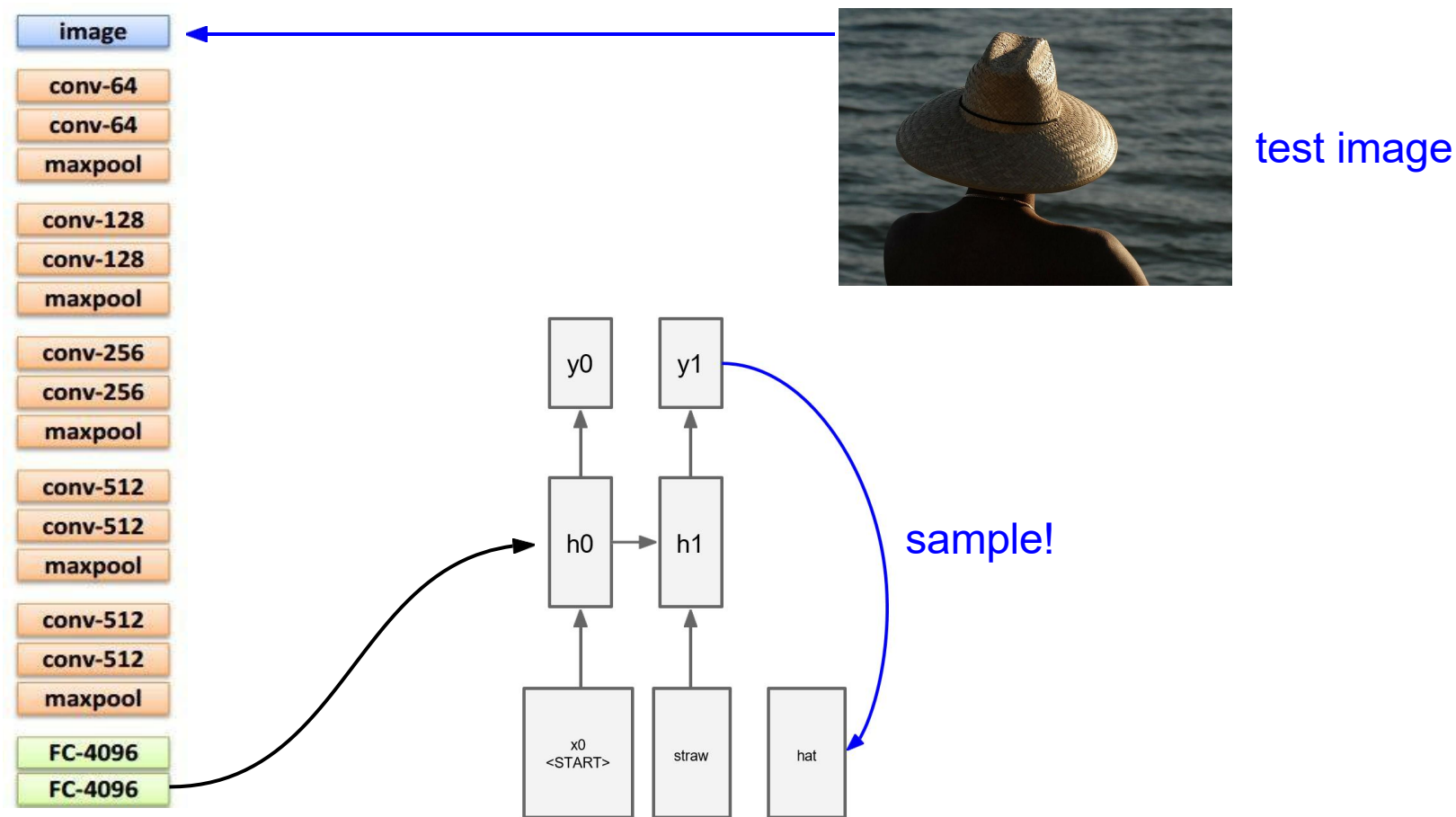


Image Captioning

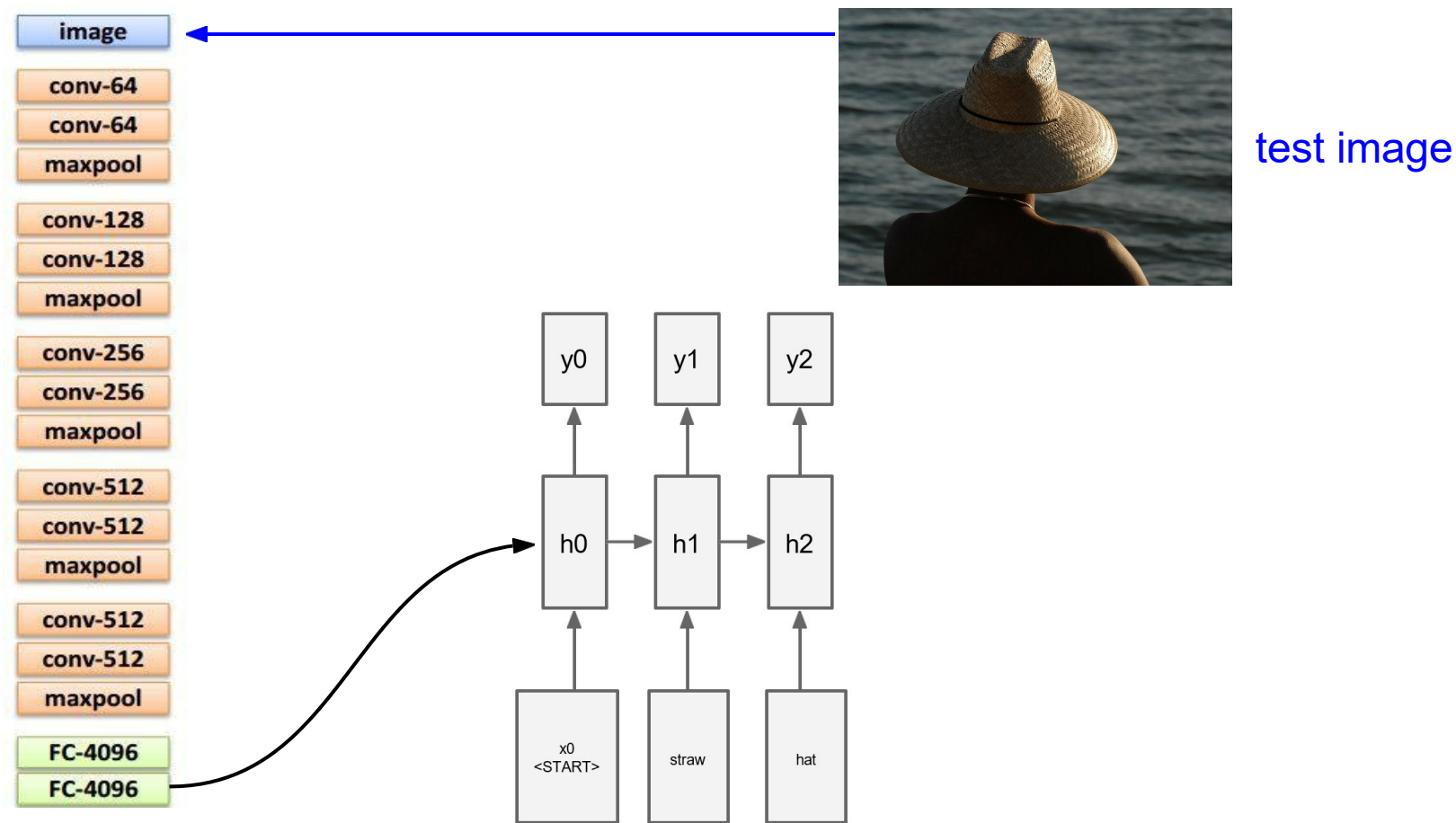


Image Captioning

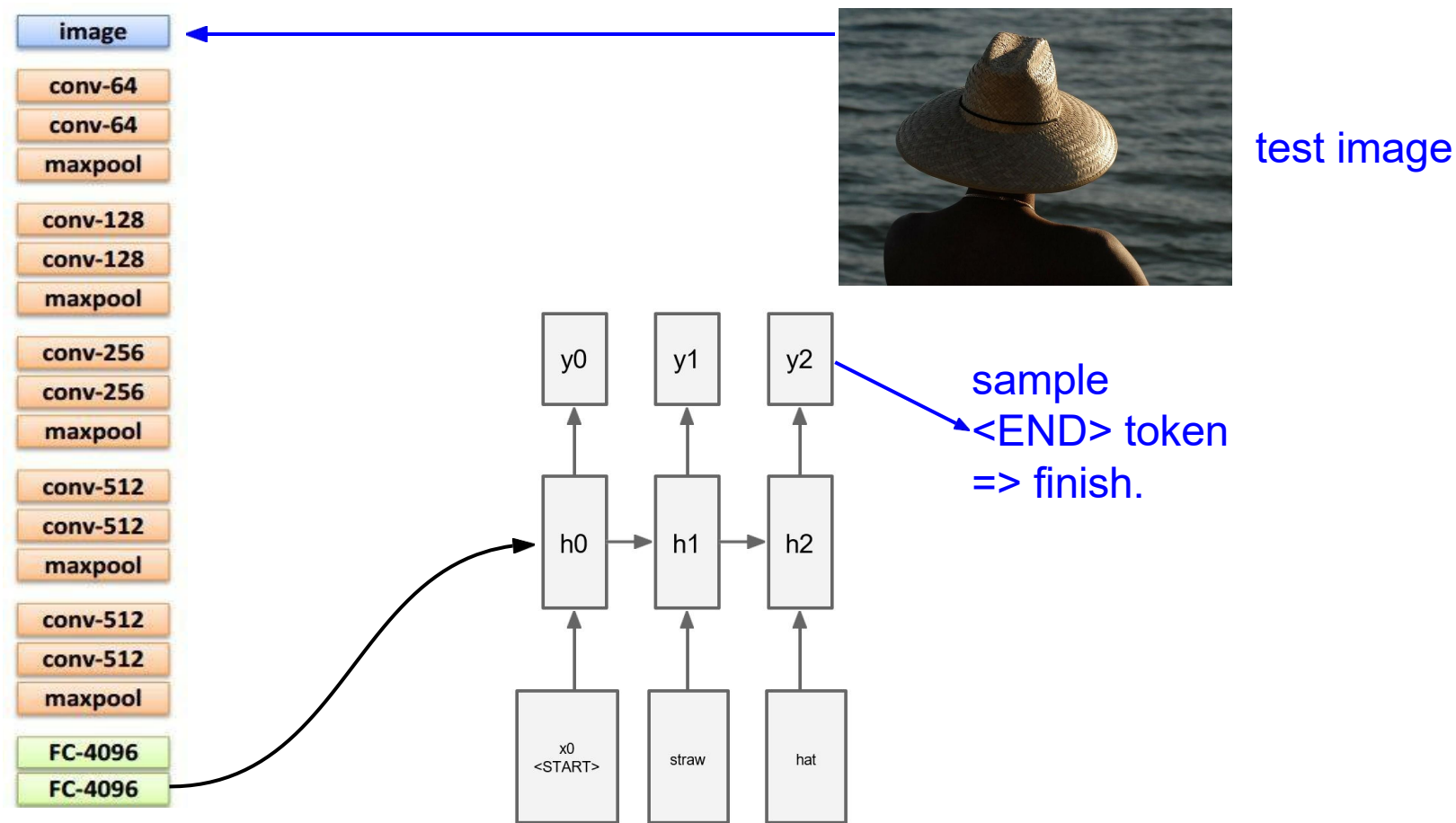


Image Captioning: 示例结果



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



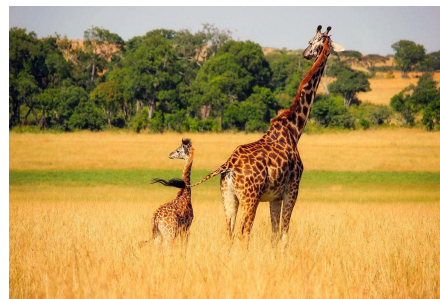
A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



Two giraffes standing in a grassy field

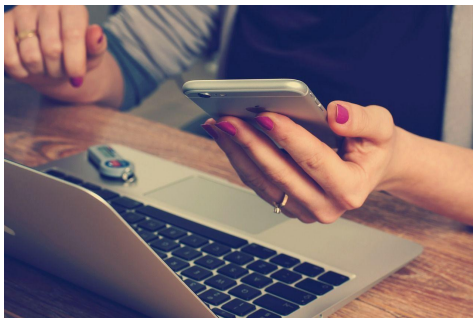


A man riding a dirt bike on a dirt track

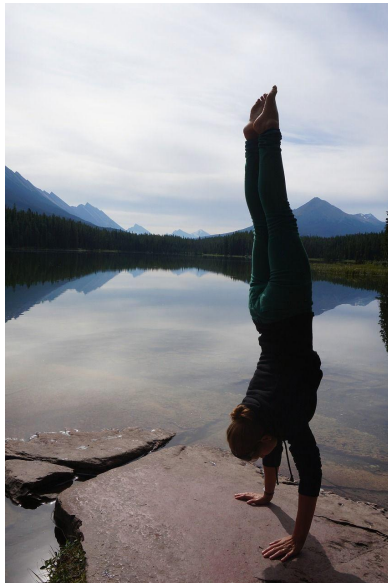
Image Captioning: 失败案例



A woman is holding a cat in her hand



A person holding a computer mouse on a desk



A woman standing on a beach holding a surfboard



A bird is perched on a tree branch



A man in a baseball uniform throwing a ball

视觉问答 (VQA)



Q: What endangered animal is featured on the truck?

A: A bald eagle.

A: A sparrow.

A: A hummingbird.

A: A raven.



Q: Where will the driver go if turning right?

A: Onto 24 1/4 Rd.

A: Onto 25 1/4 Rd.

A: Onto 23 1/4 Rd.

A: Onto Main Street.



Q: When was the picture taken?

A: During a wedding.

A: During a bar mitzvah.

A: During a funeral.

A: During a Sunday church service.



Q: Who is under the umbrella?

A: Two women.

A: A child.

A: An old man.

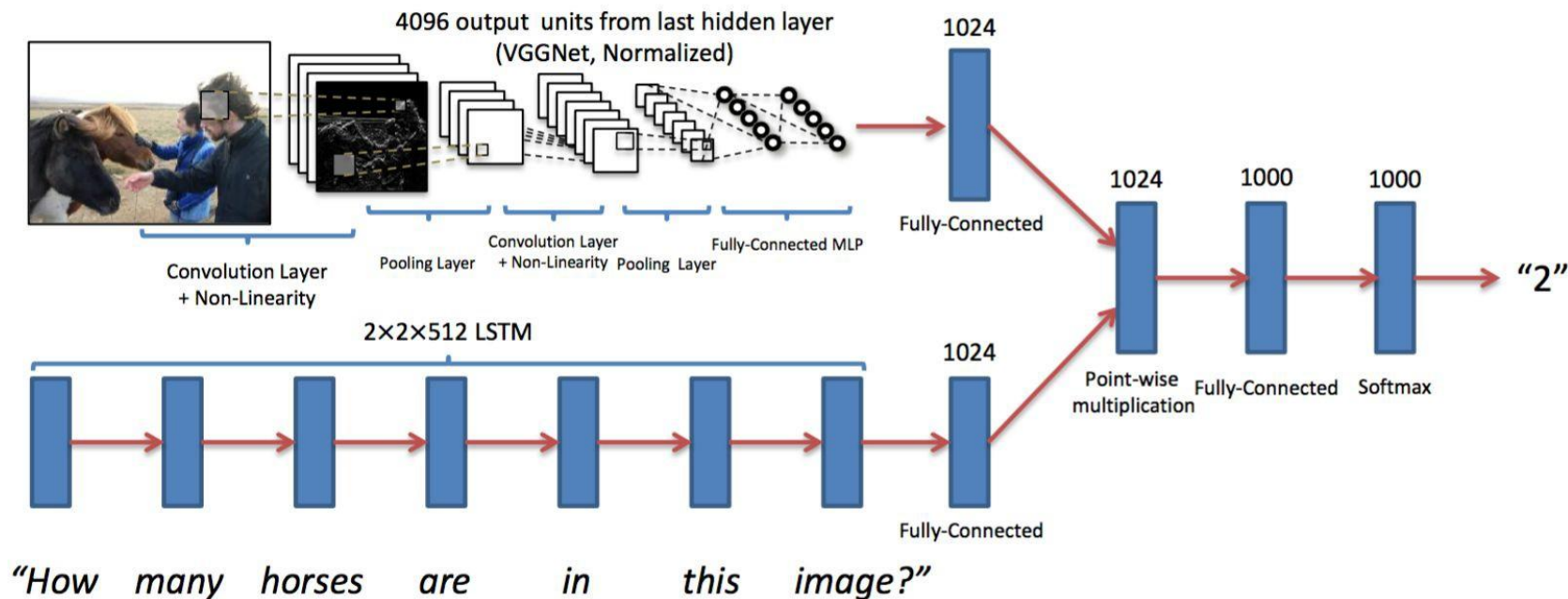
A: A husband and a wife.

Agrawal et al, "VQA: Visual Question Answering", ICCV 2015

Zhu et al, "Visual 7W: Grounded Question Answering in Images", CVPR 2016

Figure from Zhu et al, copyright IEEE 2016. Reproduced for educational purposes.

Visual Question Answering (VQA)



Agrawal et al, "Visual 7W: Grounded Question Answering in Images", CVPR 2015
Figures from Agrawal et al, copyright IEEE 2015. Reproduced for educational purposes.

视觉对话：关于图像的对话



Das et al, "Visual Dialog", CVPR 2017
Figures from Das et al, copyright IEEE 2017. Reproduced with permission.

视觉语言导航：前往客厅

代理将指令编码为语言，并使用RNN生成一系列动作，随着每次移动后视觉输入的变化而更新。

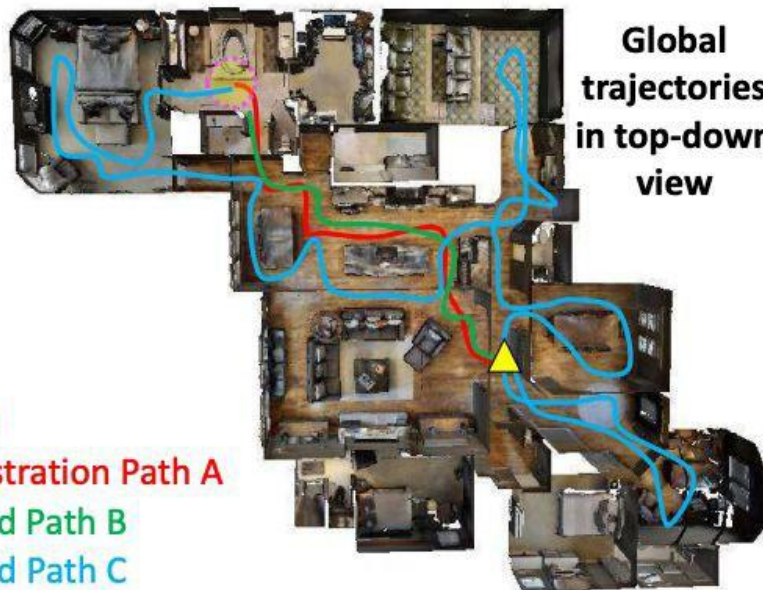
Instruction

Turn right and head towards the *kitchen*. Then turn left, pass a *table* and enter the *hallway*. Walk down the hallway and turn into the *entry way* to your right *without doors*. Stop in front of the *toilet*.

Local visual scene



Global trajectories in top-down view



▲ Initial Position

● Target Position

— Demonstration Path A

— Executed Path B

— Executed Path C

Wang et al, "Reinforced Cross-Modal Matching and Self-Supervised Imitation Learning for Vision-Language Navigation", CVPR 2018

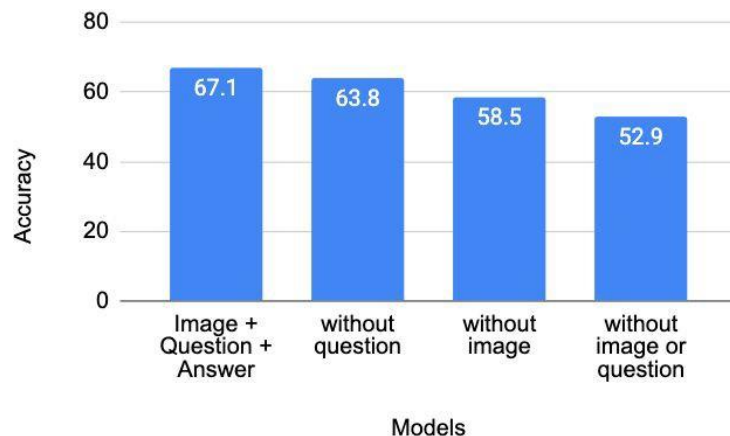
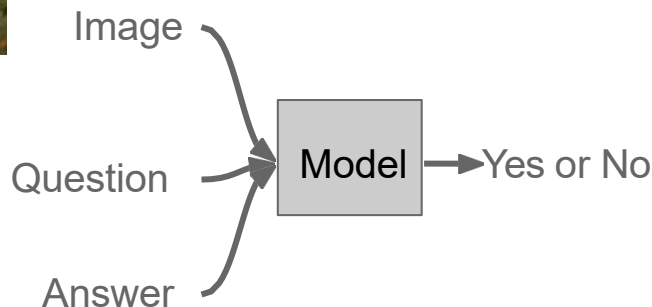
Figures from Wang et al, copyright IEEE 2017. Reproduced with permission.

视觉问答：数据集偏差

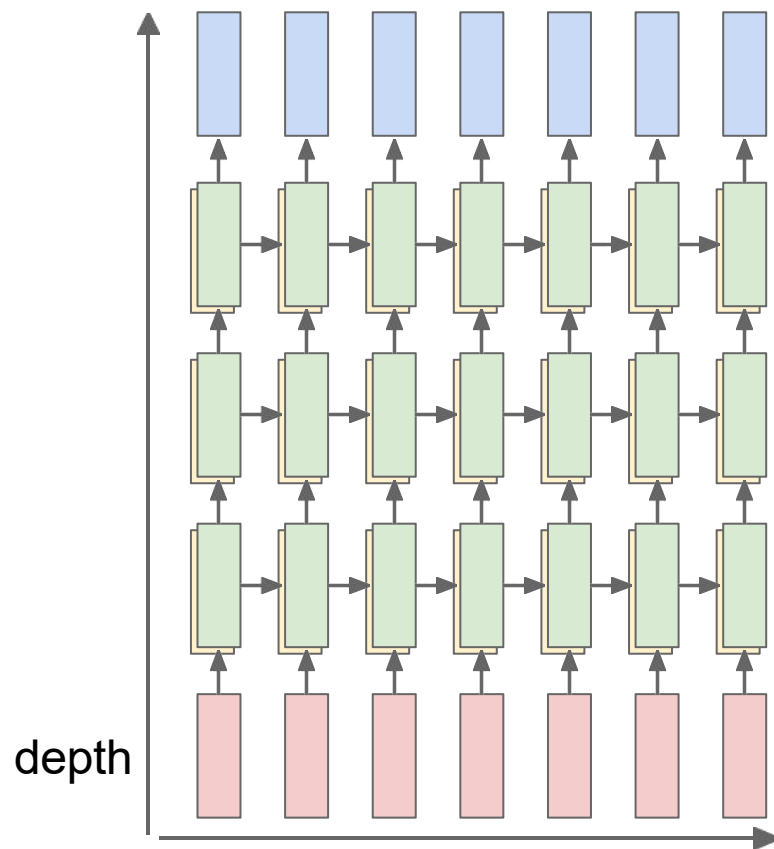
All images are [CC0](#)
Public domain: [dog](#)



What is the
dog playing
with?
Frisbee



多层RNNs



Long Short Term Memory (LSTM)

传统RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

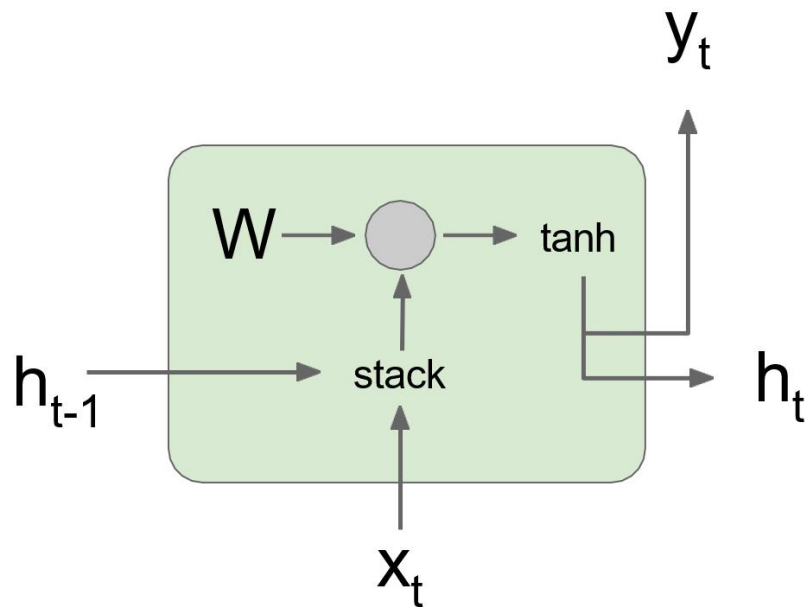
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

传统 RNN 梯度传播

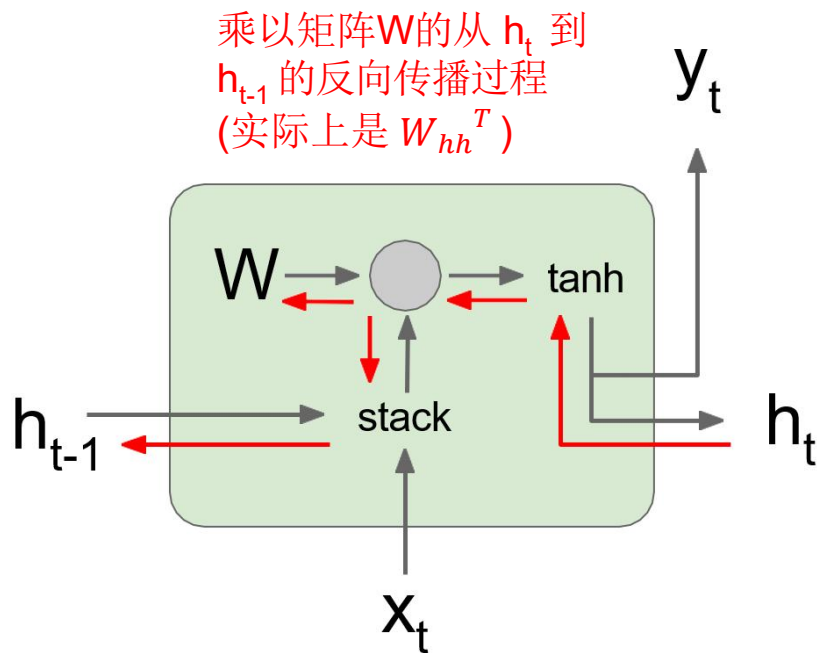
Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Vanilla RNN Gradient Flow

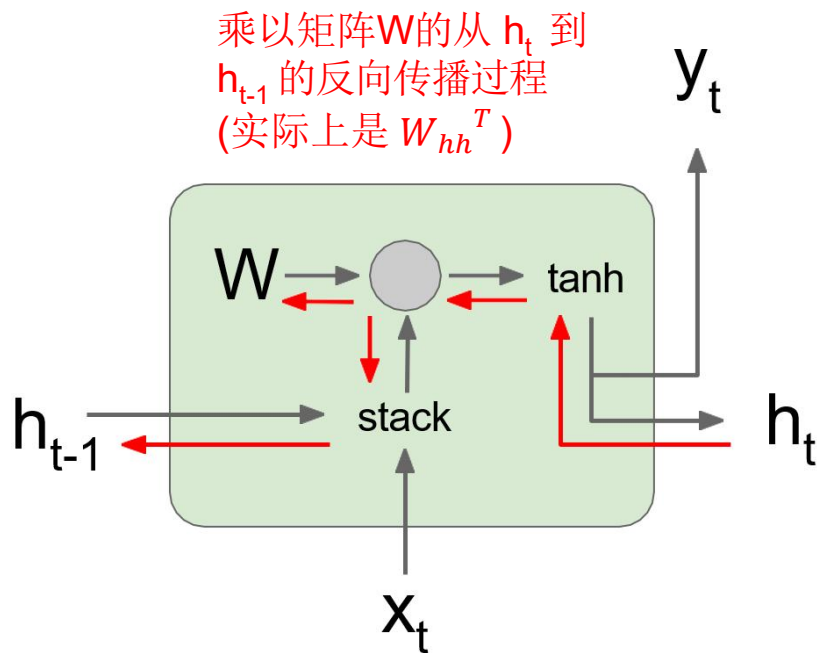
Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

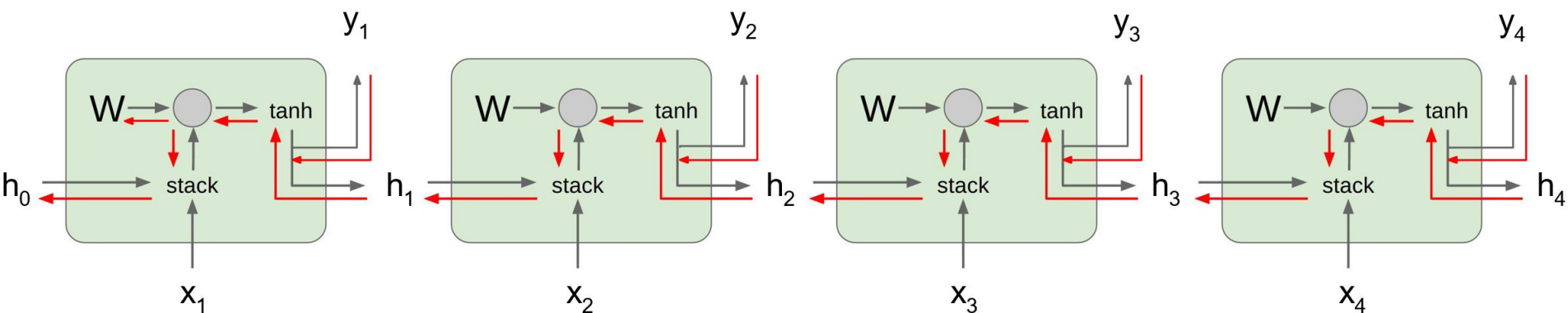


$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{hx}x_t)W_{hh}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

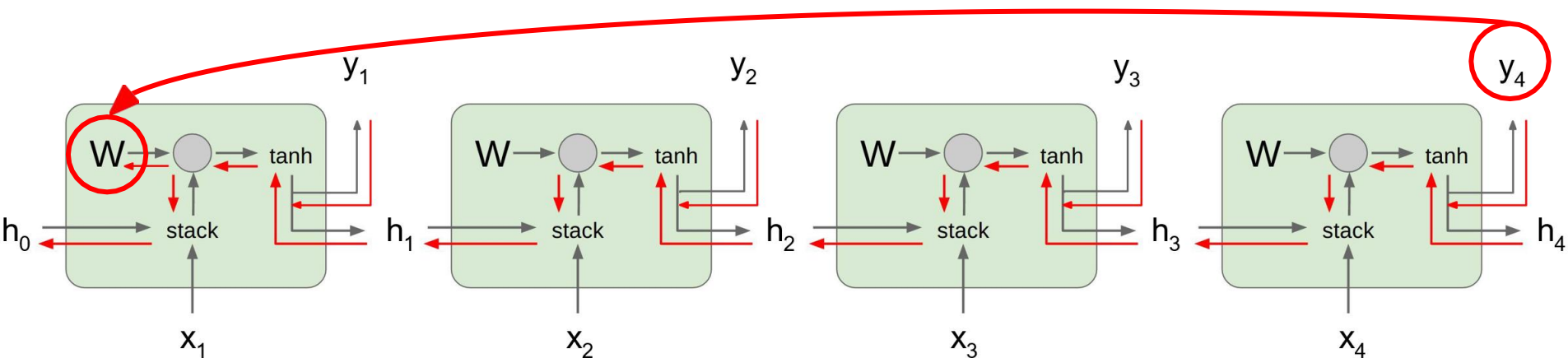


$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

传统 RNN 梯度传播

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

多个时间步下的梯度：



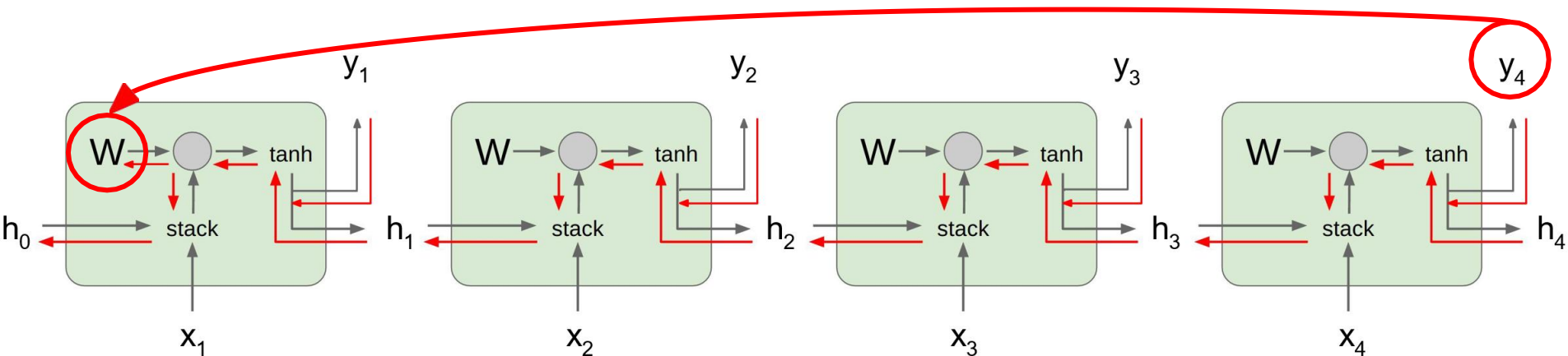
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W}$$

传统 RNN 梯度传播

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

多个时间步下的梯度：



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

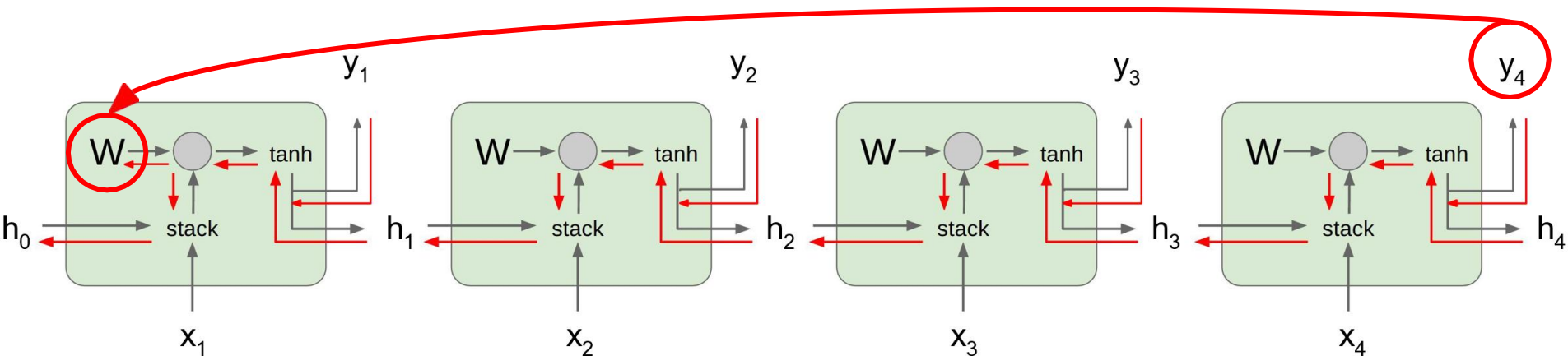
$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh} h_{t-1} + W_{xh} x_t) W_{hh}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

传统 RNN 梯度传播

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

多个时间步下的梯度：



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

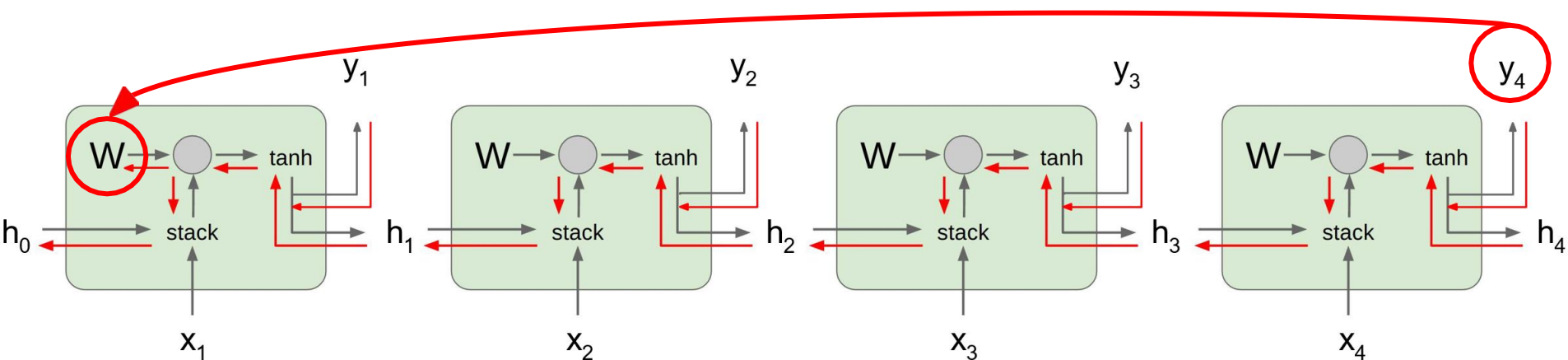
几乎总是 < 1
梯度消失

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \tanh'(W_{hh}h_{t-1} + W_{xh}x_t) \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}.$$

传统 RNN 梯度传播

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

多个时间步下的梯度：



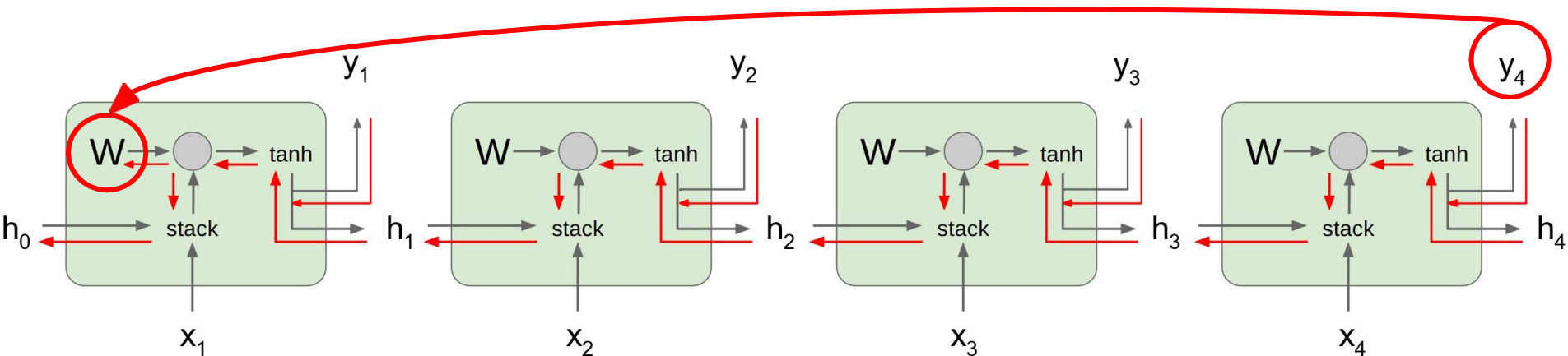
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

如果我们假设没有非线性呢？

传统 RNN 梯度传播

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

多个时间步下的梯度：



如果我们假设没有非线性呢？

最大奇异值大于1：
梯度爆炸

最大奇异值小于1：
梯度消失

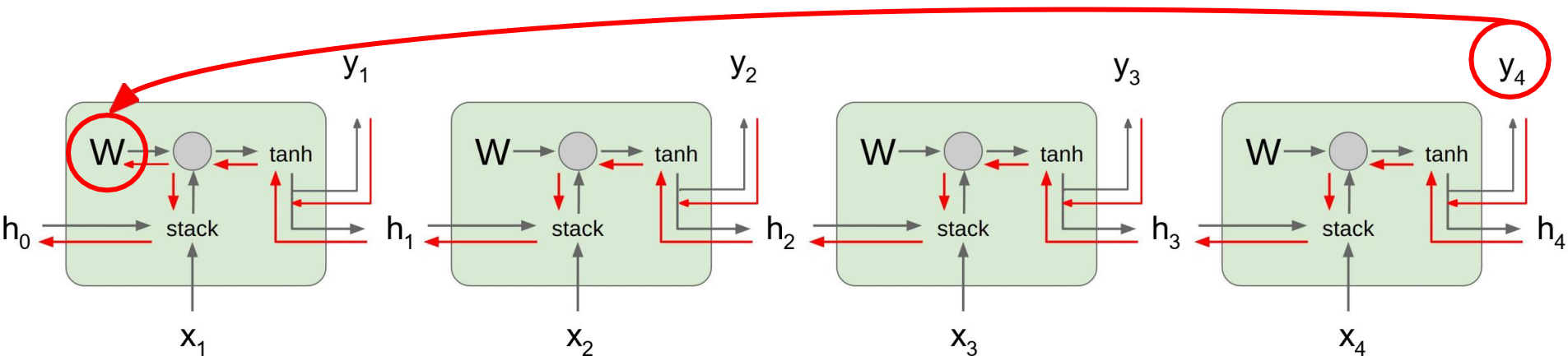
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

传统 RNN 梯度传播

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

多个时间步下的梯度:



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$
$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

如果我们假设没有非线性呢?

最大奇异值大于1:
梯度爆炸

最大奇异值小于1:
梯度消失



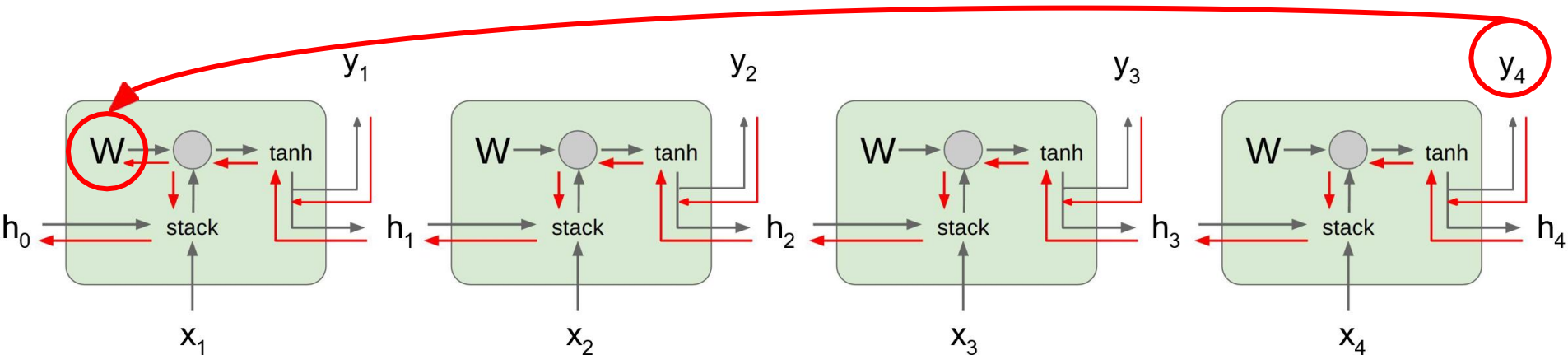
梯度裁剪:
如果梯度的范数过大, 则缩放梯度

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```

传统 RNN 梯度传播

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

多个时间步下的梯度：



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

如果我们假设没有非线性呢？

最大奇异值大于1：
梯度爆炸

最大奇异值小于1：
梯度消失



改变RNN架构

Long Short Term Memory (LSTM)

传统RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

传统RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

四个门

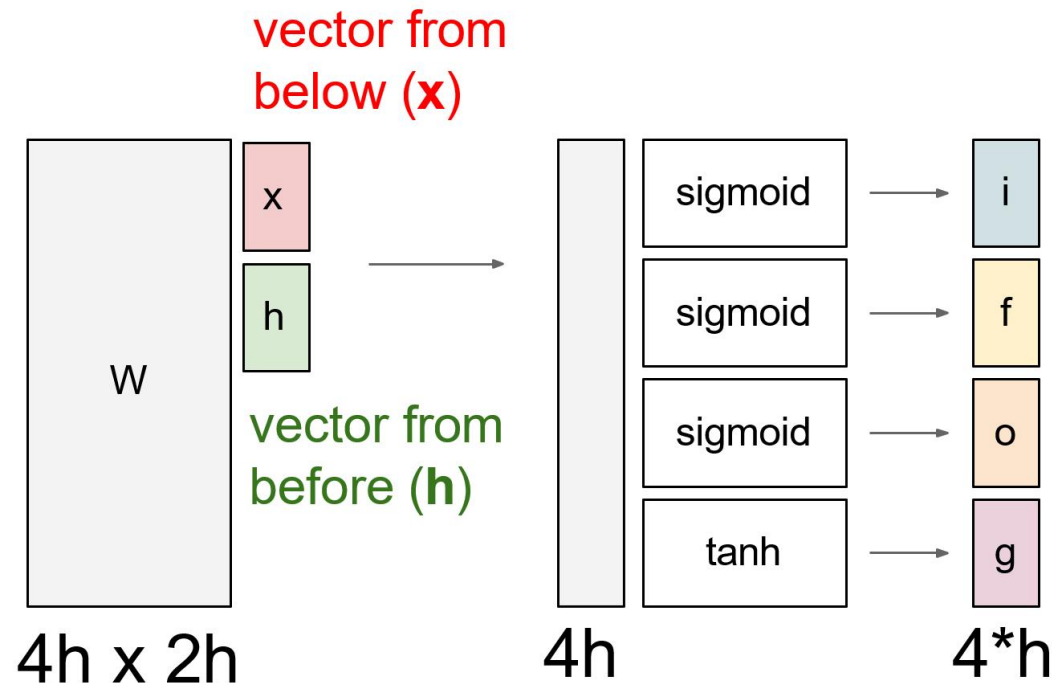
细胞状态

隐藏状态

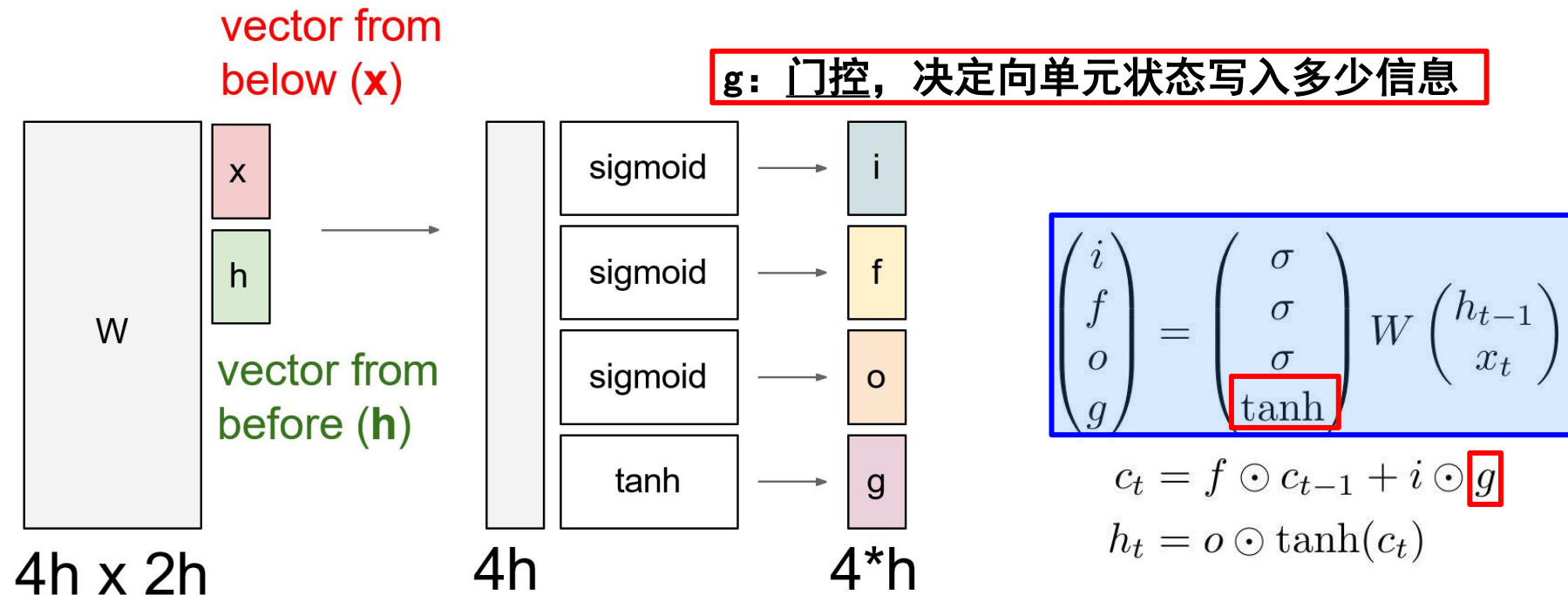
LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$
$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)



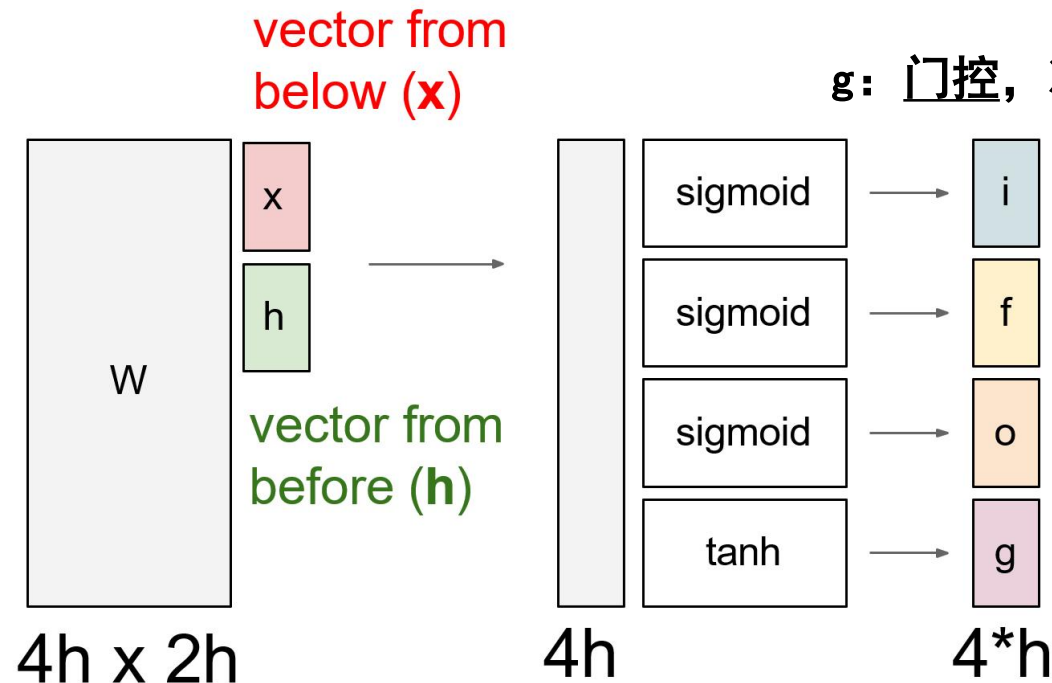
Long Short Term Memory (LSTM)



Long Short Term Memory (LSTM)

i: 输入门, 控制是否向单元写入信息

g: 门控, 决定向单元状态写入多少信息



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

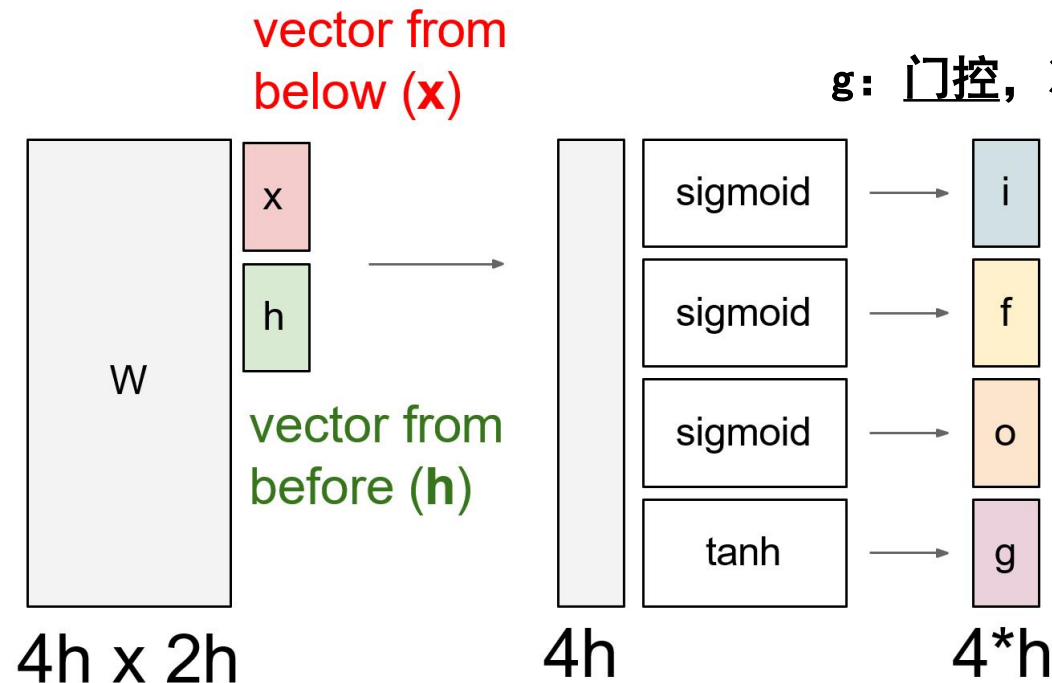
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

i: 输入门, 控制是否向单元写入信息

g: 门控, 决定向单元状态写入多少信息



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

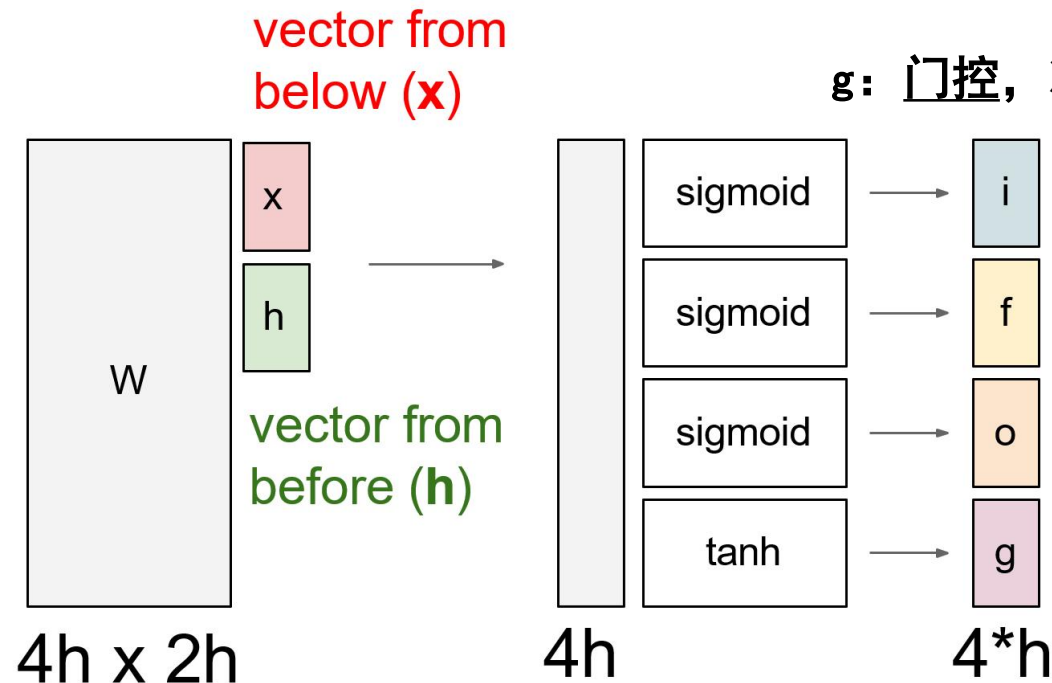
$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

i: 输入门, 控制是否向单元写入信息

f: 遗忘门, 是否清除单元状态

g: 门控, 决定向单元状态写入多少信息



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

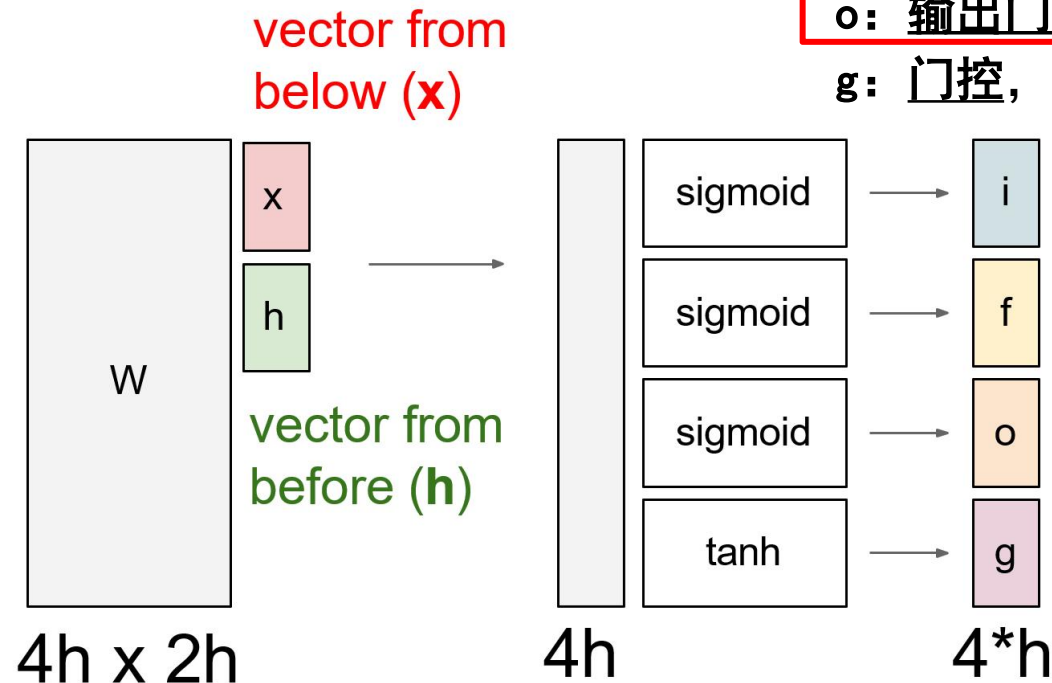
Long Short Term Memory (LSTM)

i: 输入门, 控制是否向细胞单元写入信息

f: 遗忘门, 是否清除细胞单元状态

o: 输出门, 决定当前输出输入到隐藏层的数量

g: 门控, 决定向细胞单元状态写入多少信息



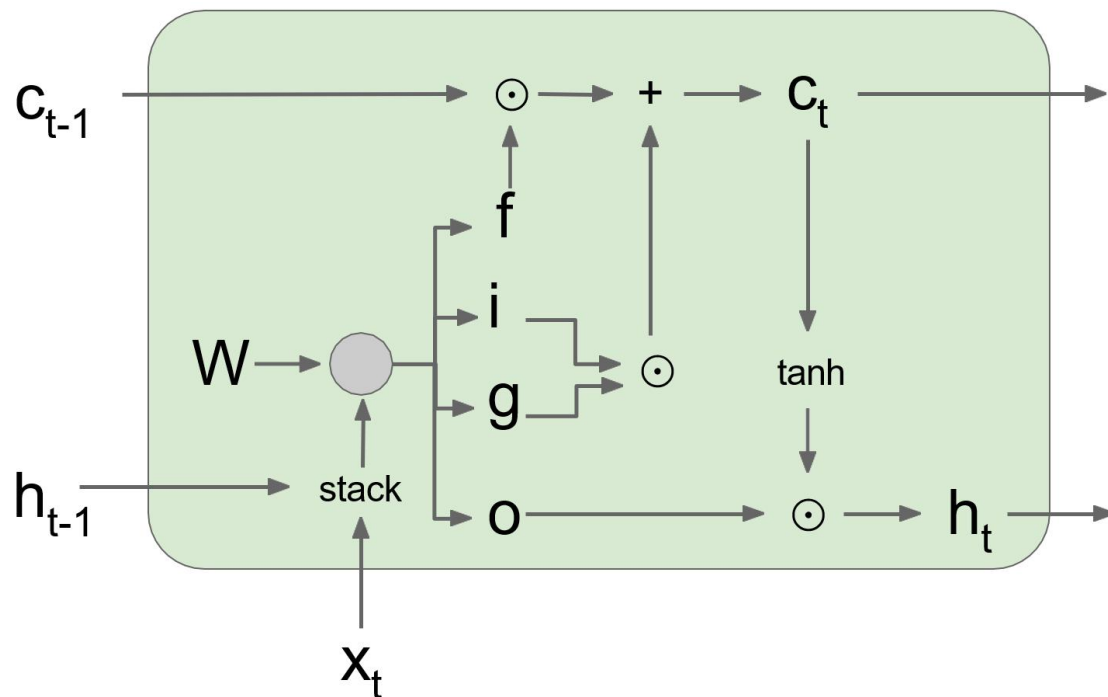
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

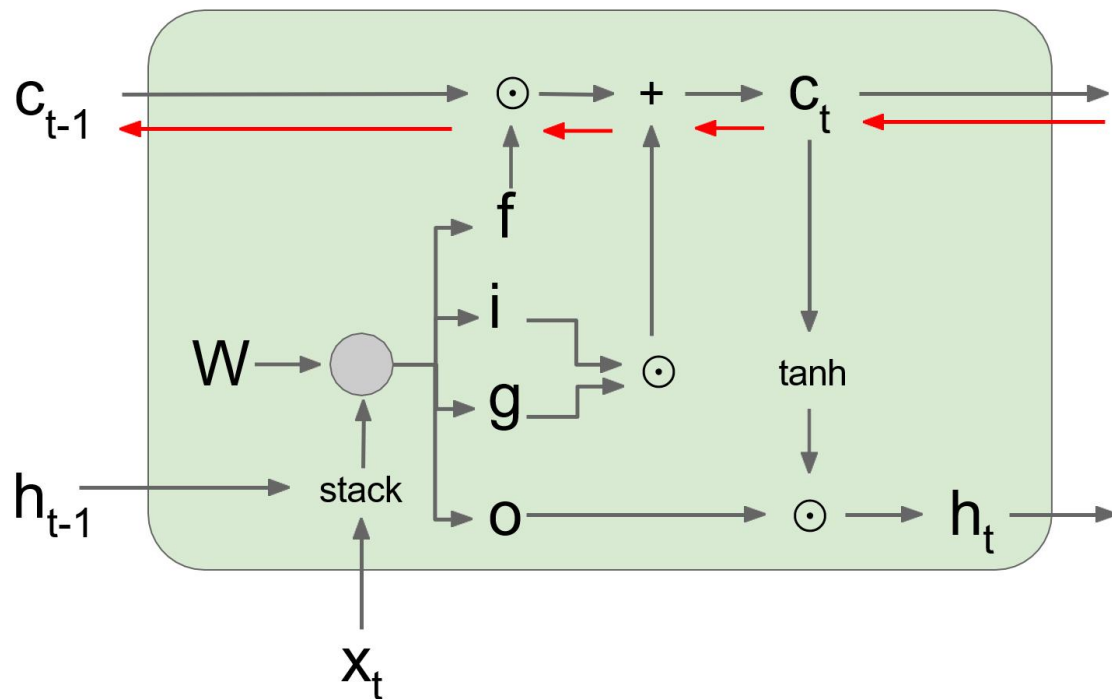


$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



从 c_t 反向传播到 c_{t-1} 仅进行与遗忘门 f 的逐元素相乘，而没有乘以权重矩阵 W 。

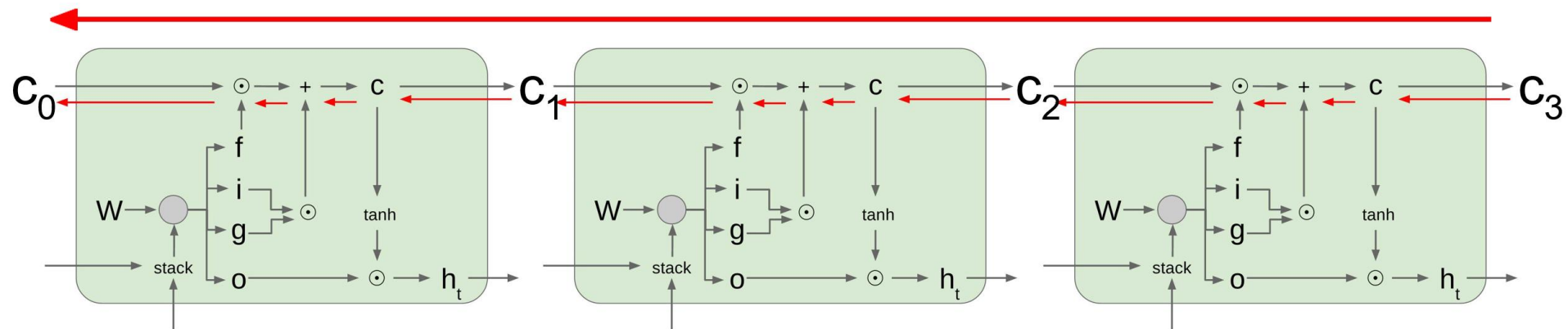
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

梯度连续流动！



注意梯度包含了遗忘门（f门）激活向量

- 这使得通过对遗忘门参数的适当更新，可以更好地控制梯度的数值。
- 同时，梯度还会通过遗忘门（f）、输入门（i）、候选门（g）和输出门（o）进行传递
- 实现对梯度数值的更好平衡

练习题

- **题目：LSTM中遗忘门的作用是？**

- A. 决定是否忽略当前输入
- B. 控制上一时刻细胞状态的保留比例
- C. 生成候选记忆
- D. 直接输出隐藏状态

- **答案：B**

练习题

- 题目：若LSTM的遗忘门全部置0，输入门为0，细胞状态会？
 - A. 完全重置为0
 - B. 保持不变
 - C. 仅依赖当前输入
 - D. 随机初始化
- 答案： B

LSTMs 解决了梯度消失的问题吗？

LSTM结构使得循环神经网络（RNN）更容易在多个时间步中保持信息。

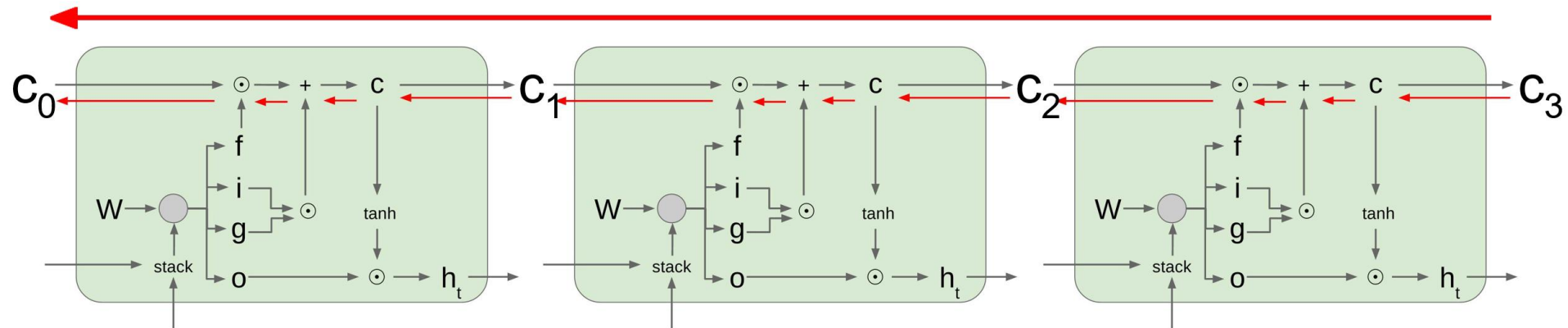
- 例如，如果遗忘门 $f=1$ 且输入门 $i=0$ ，那么该单元的信息将被无限期保留。
- 相比之下，普通RNN很难学习一个保持隐藏状态中信息的循环权重矩阵 W_h 。

LSTM不能保证完全避免梯度消失或爆炸，但它为模型学习长距离依赖关系提供了更简便的途径。

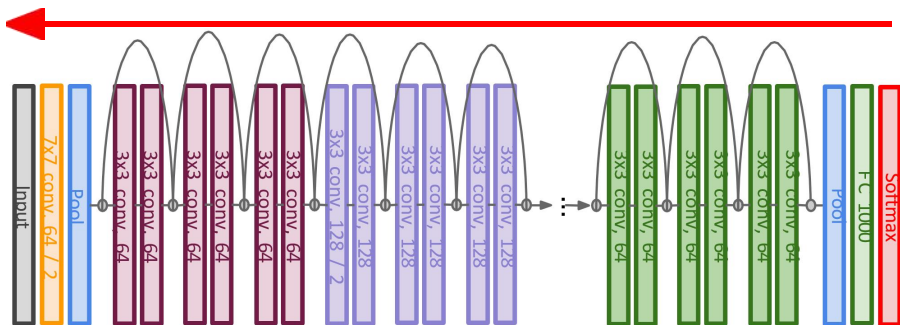
Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

梯度连续流动！



与ResNet类似！



[Hochreiter et al., 1997]

Diagram illustrating a deep convolutional neural network (CNN) architecture for handwritten digit recognition. The network consists of the following layers:

- Input:** A 28x28 grayscale image.
- Stage 1 (Green):** Two 3x3 conv layers with 64 filters, followed by a 2x2 max pool.
- Stage 2 (Purple):** Two 3x3 conv layers with 128 filters, followed by a 2x2 max pool.
- Stage 3 (Blue):** Two 3x3 conv layers with 128 filters, followed by a 2x2 max pool.
- Stage 4 (Red):** A fully connected (FC) layer with 1000 units.

The diagram shows the flow of data from the input through the convolutional and pooling layers, leading to the final classification output.

$$\begin{aligned} g &= T(x, W_T) \\ y &= g \odot H(x, W_H) + (1 - g) \odot x \end{aligned}$$

Srivastava et al, "Highway Networks", ICML DL Workshop 2015

其他 RNN 变体

GRU [*Learning phrase representations using rnn encoder-decoder for statistical machine translation*, Cho et al. 2014]

$$\begin{aligned}r_t &= \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \\z_t &= \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z) \\ \tilde{h}_t &= \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h) \\h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t\end{aligned}$$

[*LSTM: A Search Space Odyssey*, Greff et al., 2015]

[*An Empirical Exploration of Recurrent Network Architectures*, Jozefowicz et al., 2015]

MUT1:

$$\begin{aligned}z &= \text{sigm}(W_{zx}x_t + b_z) \\r &= \text{sigm}(W_{xr}x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + \tanh(x_t) + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

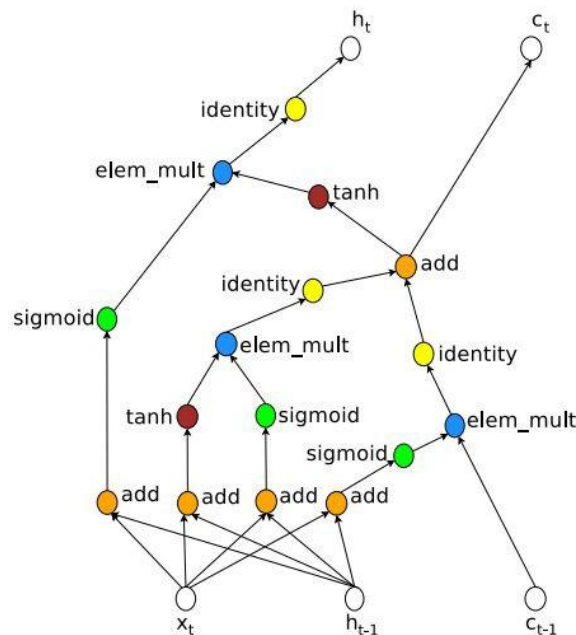
MUT2:

$$\begin{aligned}z &= \text{sigm}(W_{zx}x_t + W_{hz}h_t + b_z) \\r &= \text{sigm}(x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + W_{xh}x_t + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

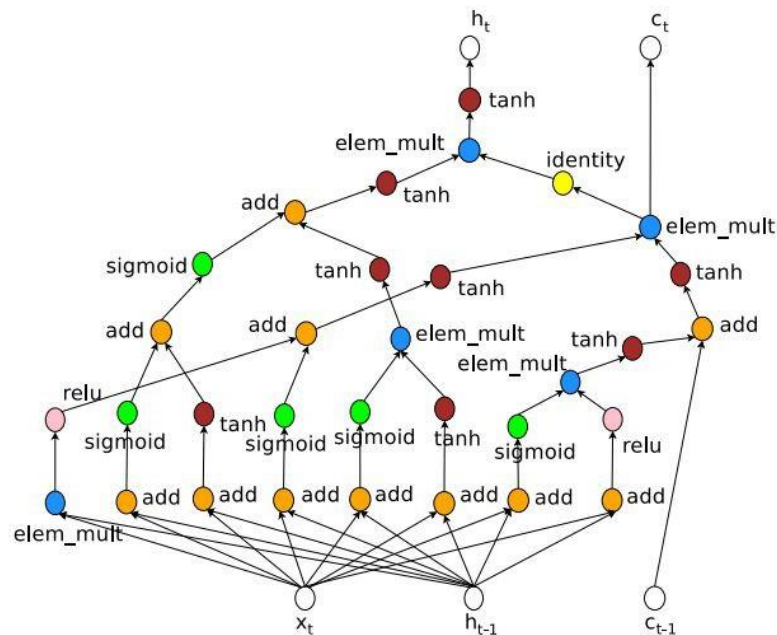
MUT3:

$$\begin{aligned}z &= \text{sigm}(W_{zx}x_t + W_{hz} \tanh(h_t) + b_z) \\r &= \text{sigm}(W_{xr}x_t + W_{hr}h_t + b_r) \\h_{t+1} &= \tanh(W_{hh}(r \odot h_t) + W_{xh}x_t + b_h) \odot z \\&+ h_t \odot (1 - z)\end{aligned}$$

RNN的神经架构搜索



LSTM cell



Cell they found

Zoph et Le, "Neural Architecture Search with Reinforcement Learning", ICLR 2017

Figures copyright Zoph et al, 2017. Reproduced with permission.

总结

- RNN 在架构设计上提供了很大的灵活性。
- 传统 RNN 结构简单，但性能不佳。
- 常用的是 LSTM 或 GRU：它们的加性交互作用有助于改善梯度流。
- RNN 中的梯度反向传播可能会出现爆炸或消失。梯度爆炸可以通过梯度裁剪来控制，梯度消失则通过加性交互（如 LSTM）来缓解。
- 更好且更简洁的架构是当前研究的热点，同时也关注序列推理的新范式。
- 需要在理论和经验上的深入理解。

提纲

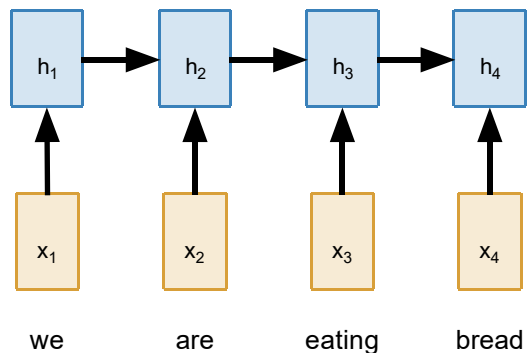
- 深度学习历史发展
- 前馈神经网络
- 卷积神经网络
- 循环神经网络
- 注意力机制与Transformer
- 深度学习应用与研讨

使用 RNNs 进行序列到序列分析

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

Encoder: $h_t = f_W(x_t, h_{t-1})$

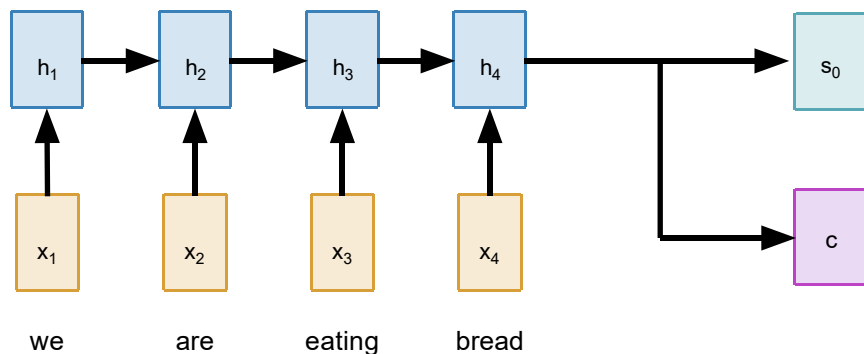


使用 RNNs 进行序列到序列分析

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

Encoder: $h_t = f_W(x_t, h_{t-1})$ 从最终隐藏状态预测:
解码器初始状态 s
Context vector c (often $c=h_T$)



使用 RNNs 进行序列到序列分析

Input: Sequence $x_1, \dots, x_T$

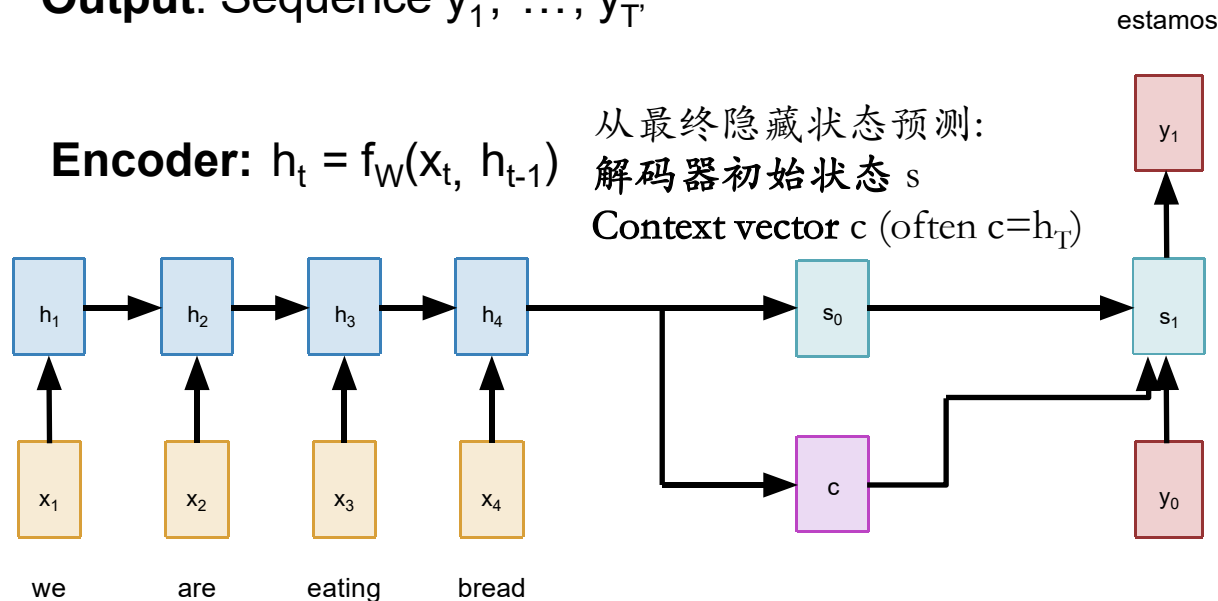
Output: Sequence $y_1, \dots, y_T$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

从最终隐藏状态预测:
解码器初始状态 s

Context vector c (often $c=h_T$)



使用 RNNs 进行序列到序列分析

Input: Sequence $x_1, \dots, x_T$

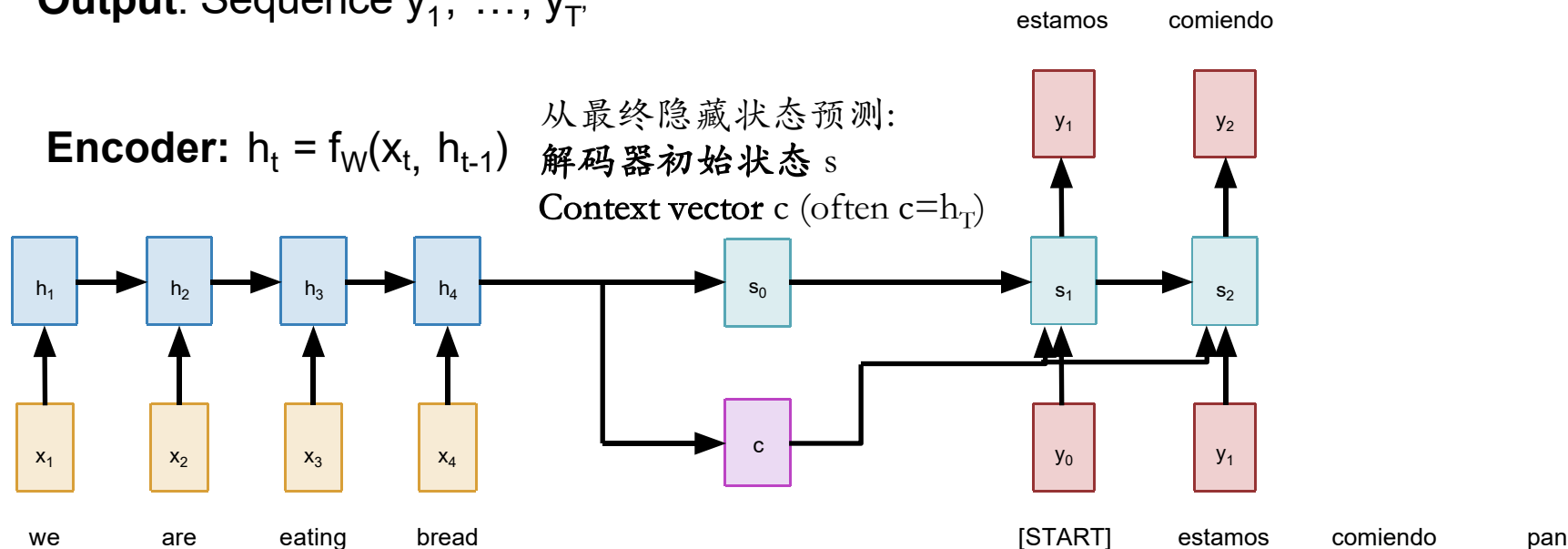
Output: Sequence $y_1, \dots, y_T$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

从最终隐藏状态预测:
解码器初始状态 s

Context vector c (often $c=h_T$)



使用 RNNs 进行序列到序列分析

Input: Sequence $x_1, \dots, x_T$

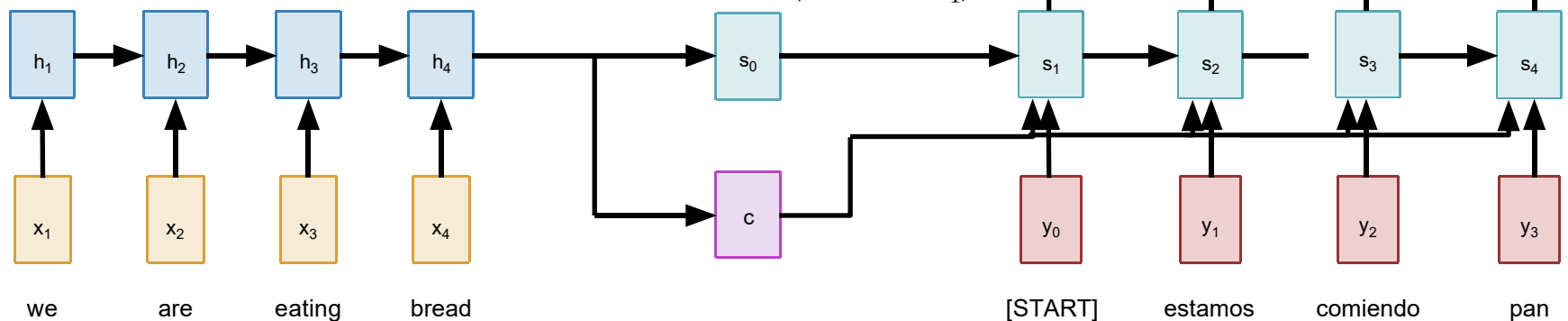
Output: Sequence $y_1, \dots, y_T$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

从最终隐藏状态预测:
解码器初始状态 s

Context vector c (often $c=h_T$)



使用 RNNs 进行序列到序列分析

Input: Sequence $x_1, \dots, x_T$

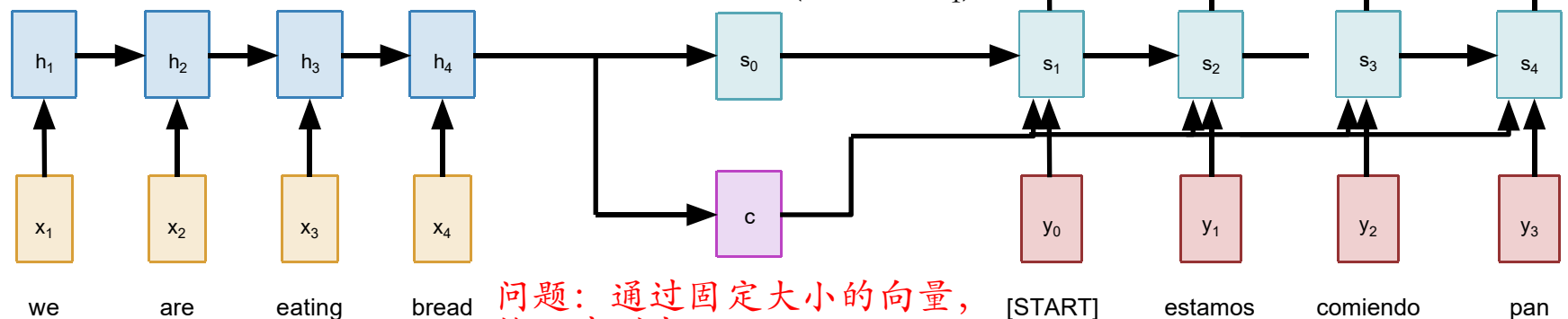
Output: Sequence $y_1, \dots, y_T$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

从最终隐藏状态预测:
解码器初始状态 s

Context vector c (often $c=h_T$)



问题: 通过固定大小的向量,
输入序列出现bottlenecked。
如果 $T = 1000$ 呢?

使用 RNNs 进行序列到序列分析

Input: Sequence $x_1, \dots, x_T$

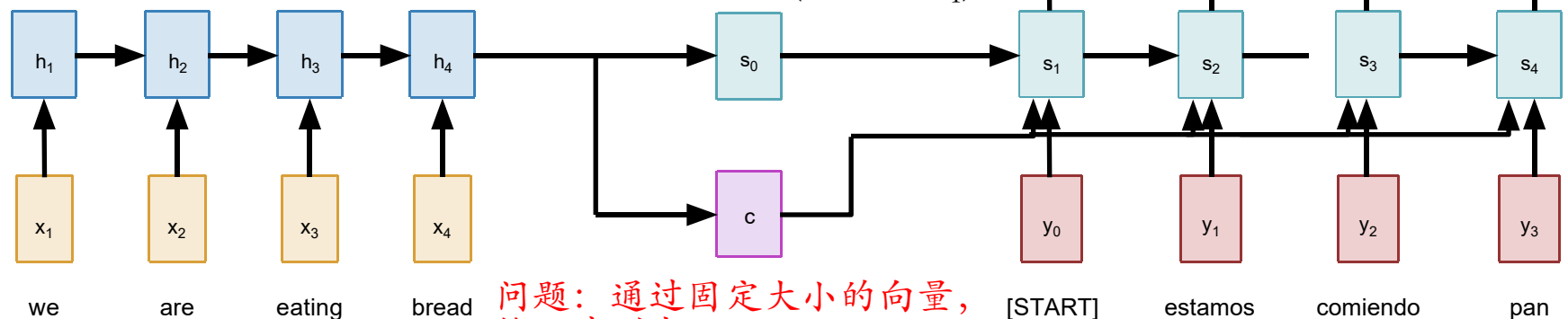
Output: Sequence $y_1, \dots, y_T$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

从最终隐藏状态预测:
解码器初始状态 s

Context vector c (often $c=h_T$)



问题: 通过固定大小的向量,
输入序列出现bottlenecked。
如果 $T = 1000$ 呢?

想法: 在解码器的每一步都使用新的上下文向量!

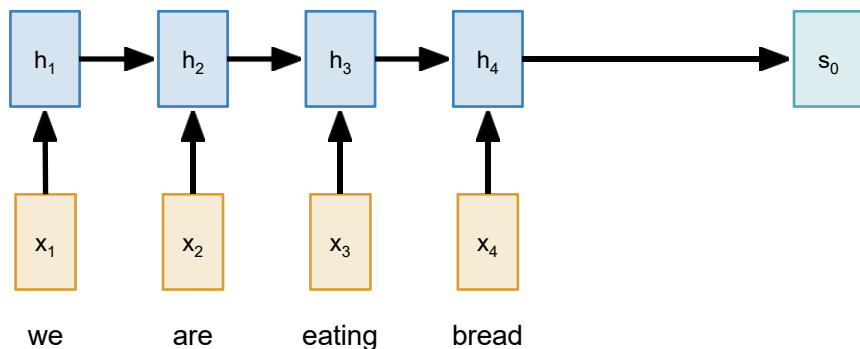
使用 RNNs和Attention进行序列到序列分析

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

Encoder: $h_t = f_w(x_t, h_{t-1})$

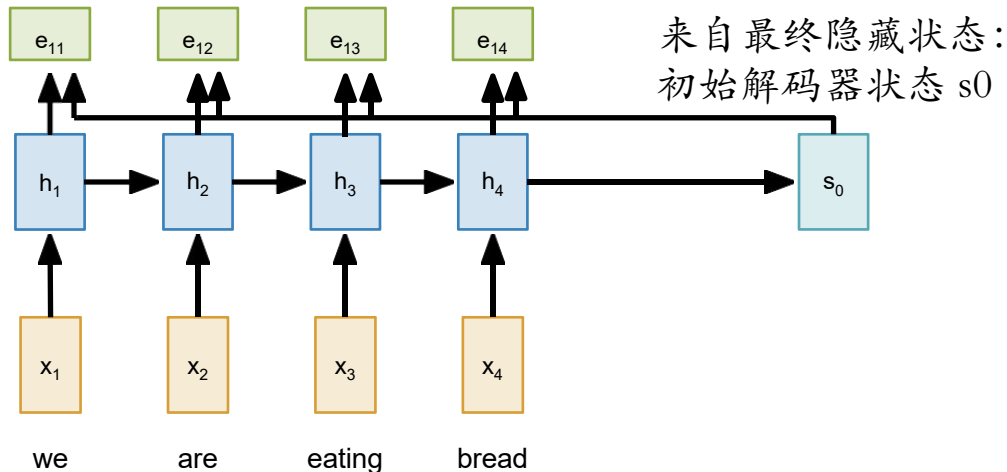
来自最终隐藏状态：
初始解码器状态 s_0



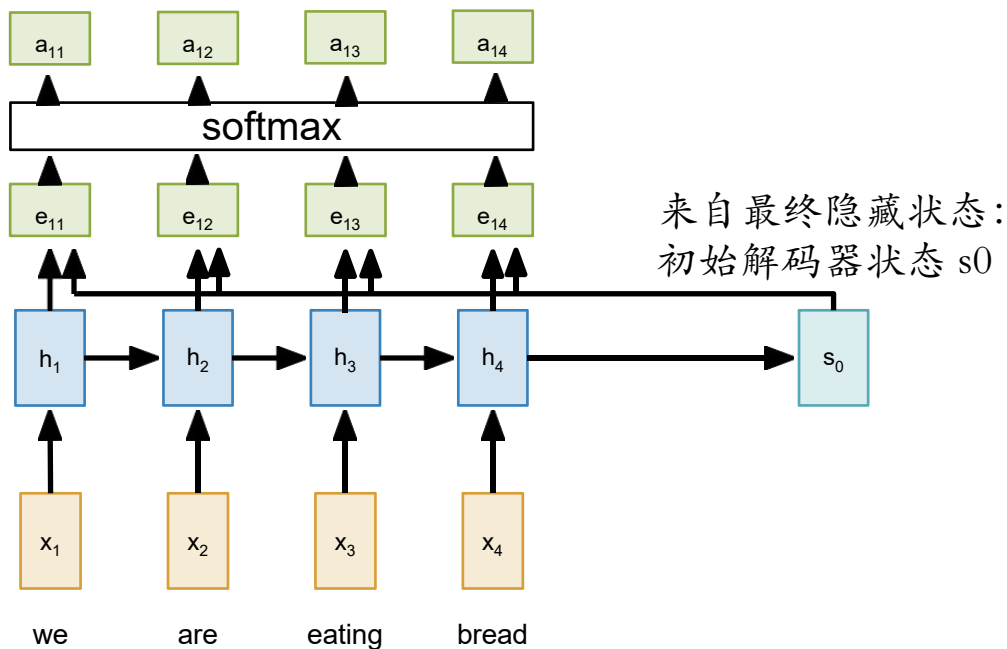
使用 RNNs和Attention进行序列到序列分析

计算 (标量) **alignment** 分数

$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad f_{\text{att}} \text{ 是一个MLP}$$



使用 RNNs和Attention进行序列到序列分析



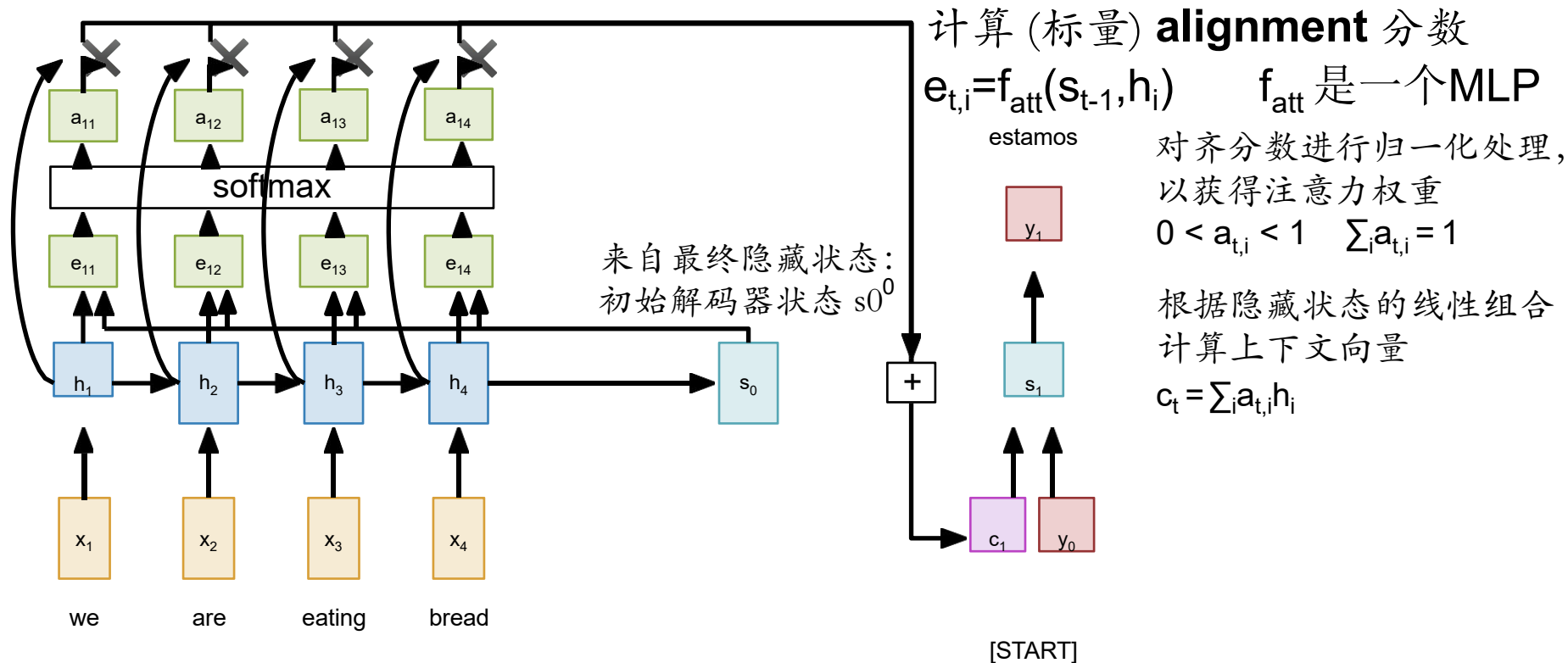
计算 (标量) **alignment** 分数

$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad f_{\text{att}} \text{ 是一个 MLP}$$

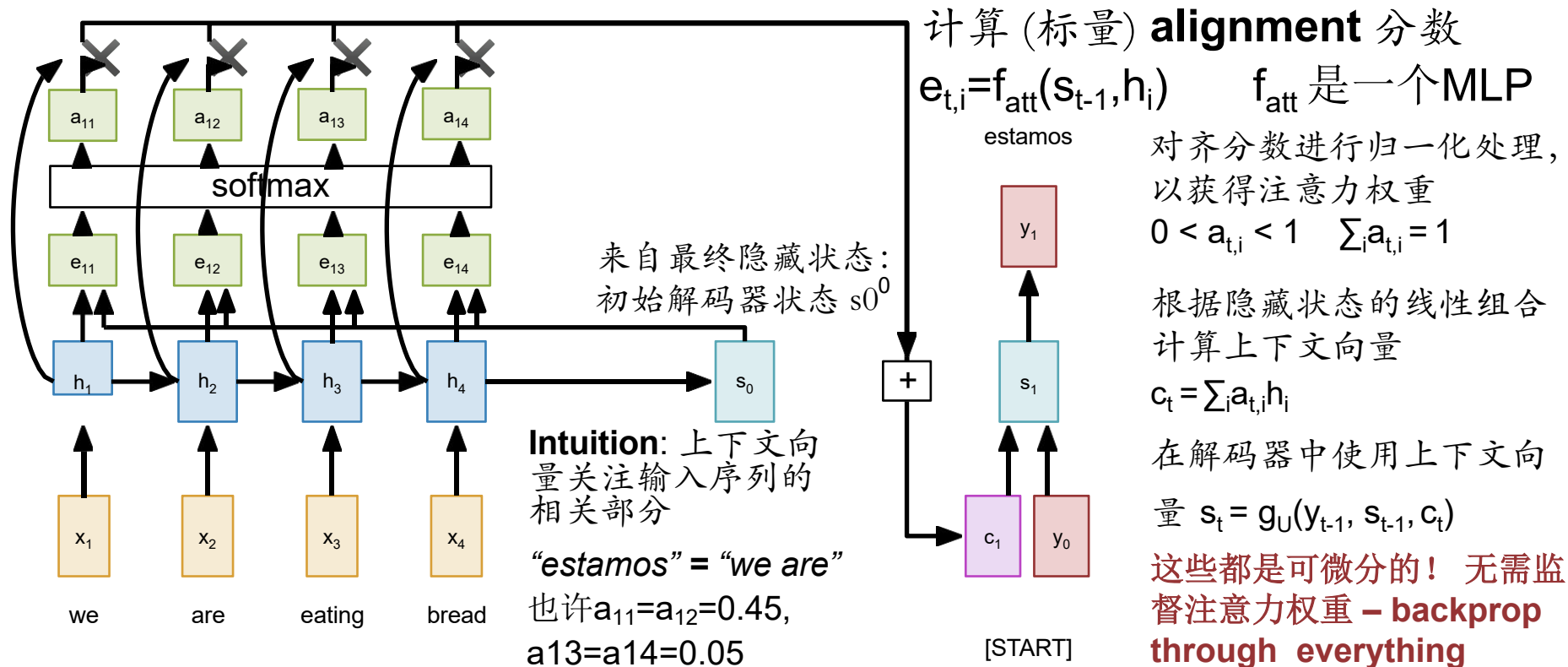
对齐分数进行归一化处理,
以获得注意力权重

$$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$$

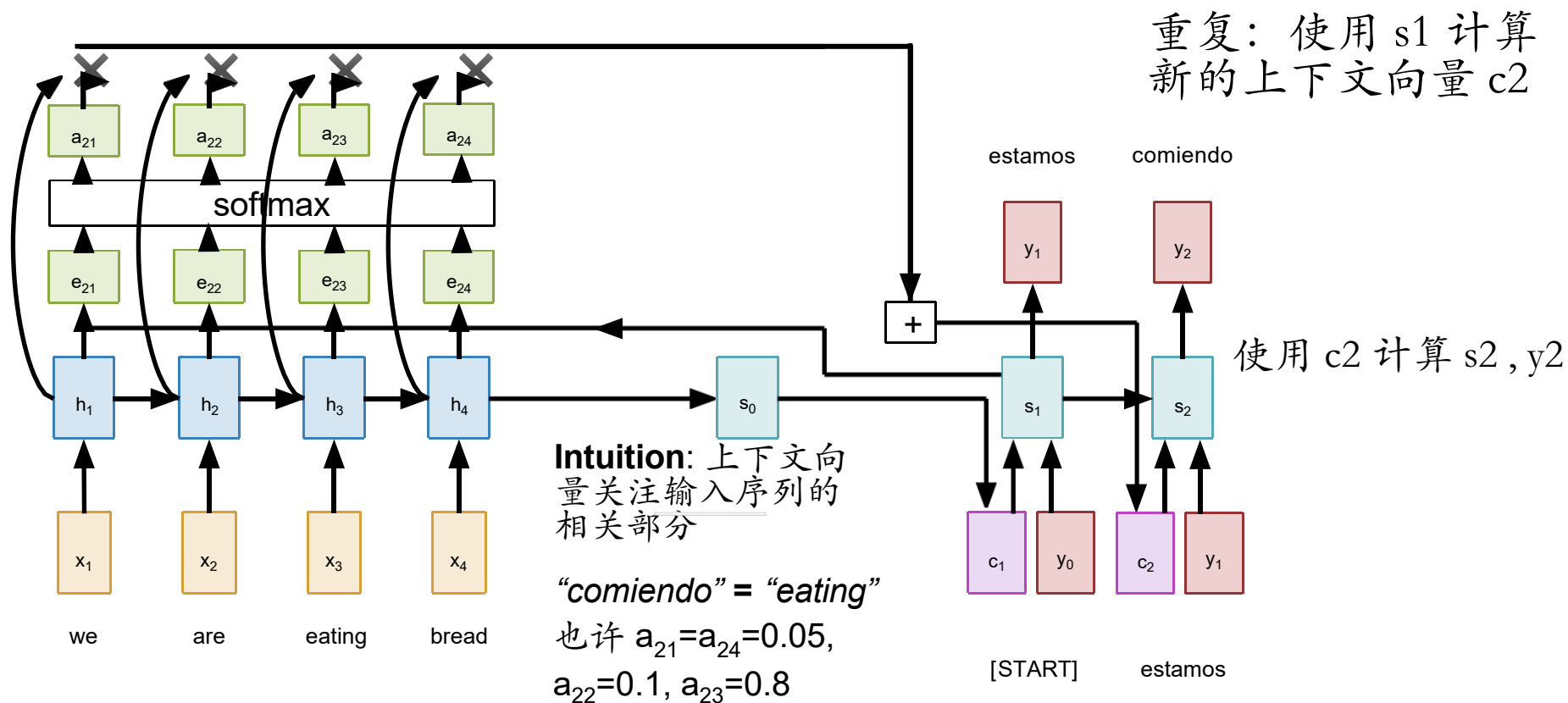
使用 RNNs和Attention进行序列到序列分析



使用 RNNs和Attention进行序列到序列分析



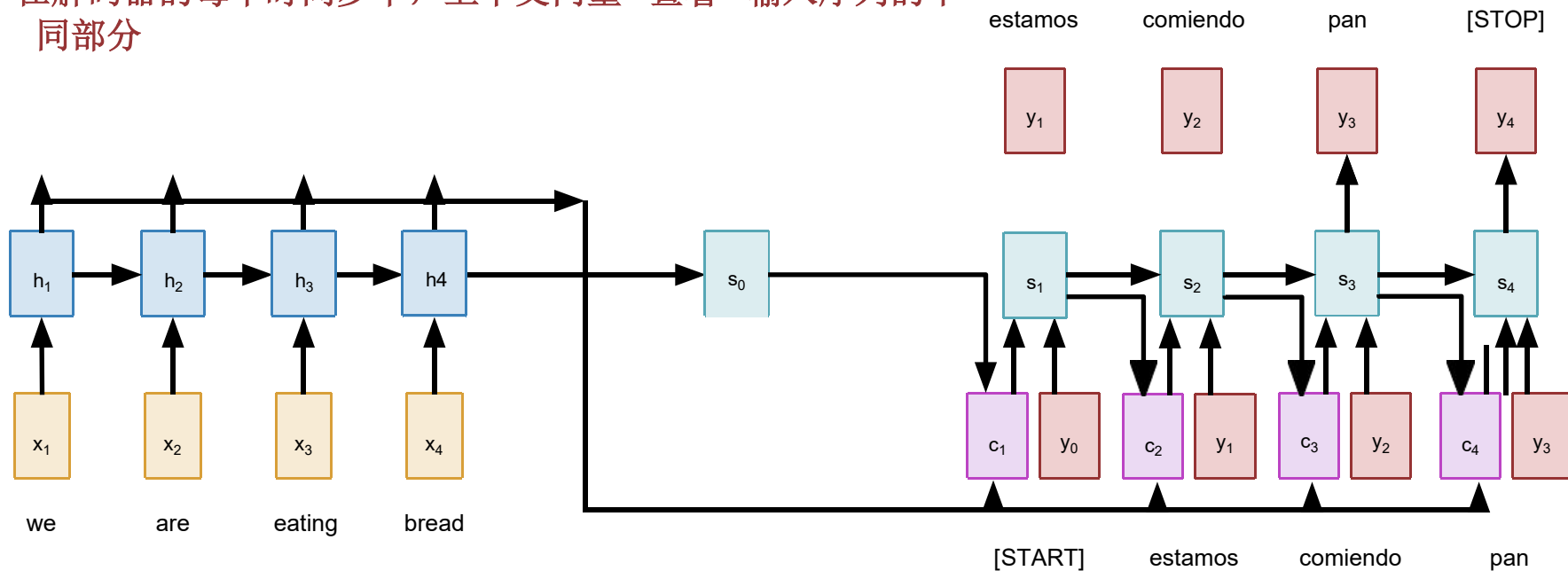
使用 RNNs 和 Attention 进行序列到序列分析



使用 RNNs和Attention进行序列到序列分析

在解码器的每个时间步中使用不同的上下文向量

- 输入序列不会因单一向量而出现瓶颈
- 在解码器的每个时间步中，上下文向量 "查看" 输入序列的不同部分



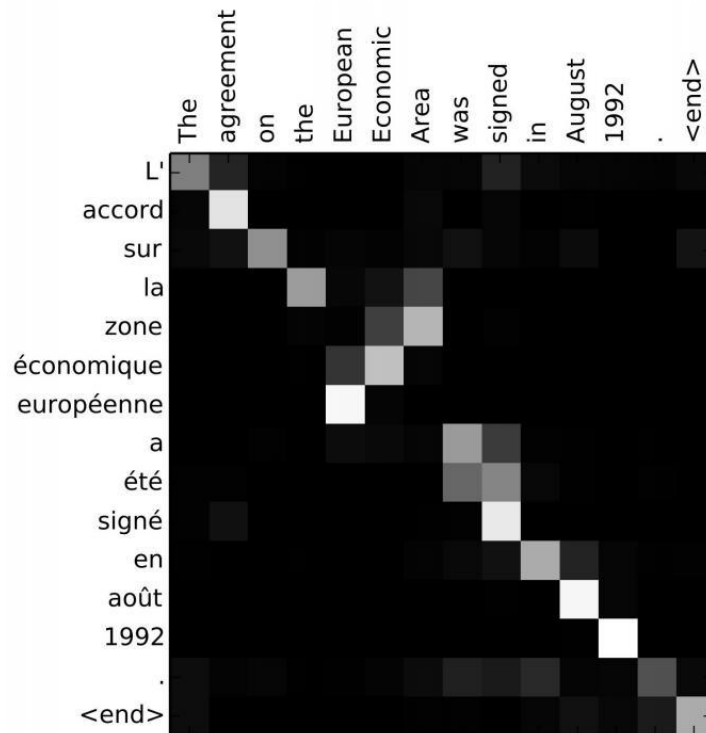
使用 RNNs和Attention进行序列到序列分析

可视化注意力权重 $a_{t,i}$

Example: 英语到法语翻译

Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L'accord sur la zone économique européenne a été signé en août 1992.”



使用 RNNs和Attention进行序列到序列分析

Example: 英语到法语翻译

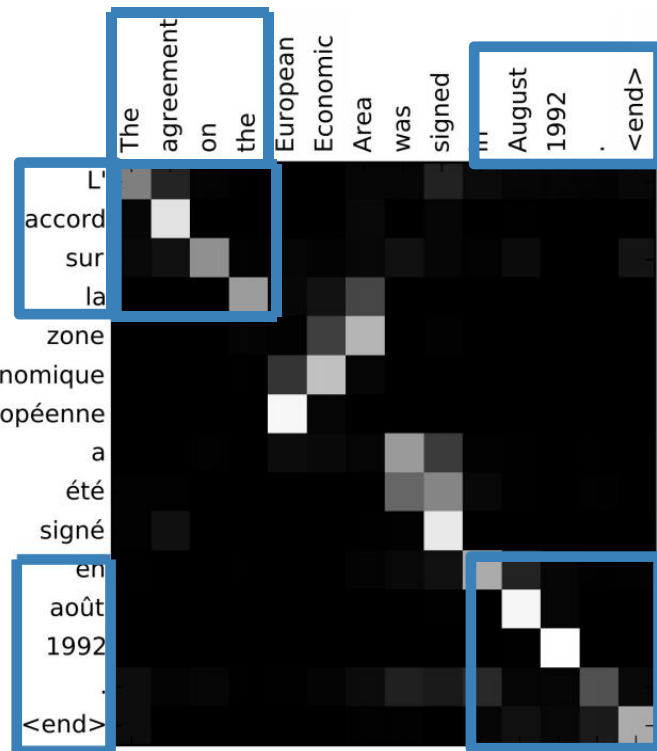
Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L'accord sur la zone économique européenne a été signé en août 1992.”

Diagonal attention means words correspond in order

Diagonal attention means words correspond in order

可视化注意力权重 $a_{t,i}$



使用 RNNs和Attention进行序列到序列分析

Example: 英语到法语翻译

Input: “The agreement on the
European Economic Area
was signed in August 1992.”

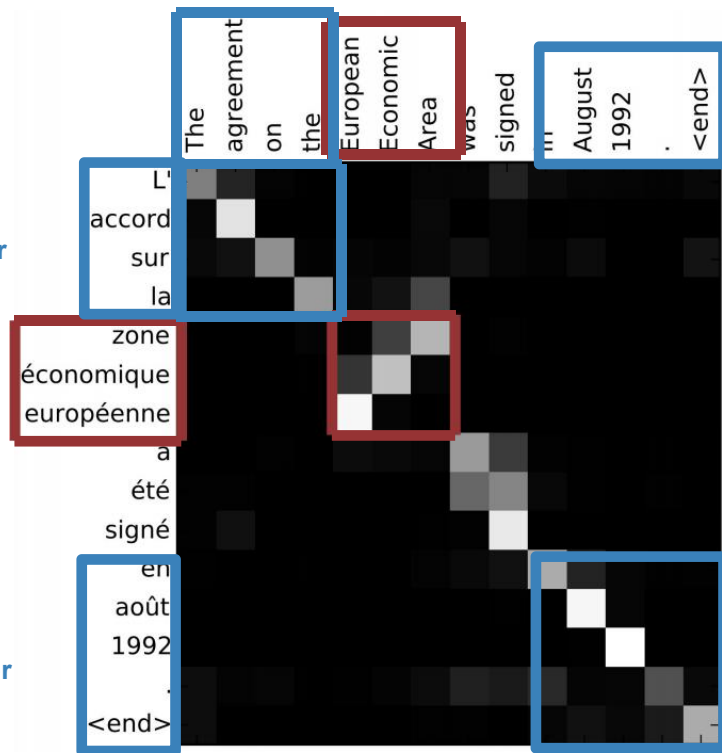
Output: “L'accord sur la
zone économique
européenne a été signé en
août 1992.”

Diagonal attention means
words correspond in order

Attention figures out
different word orders

Diagonal attention means
words correspond in order

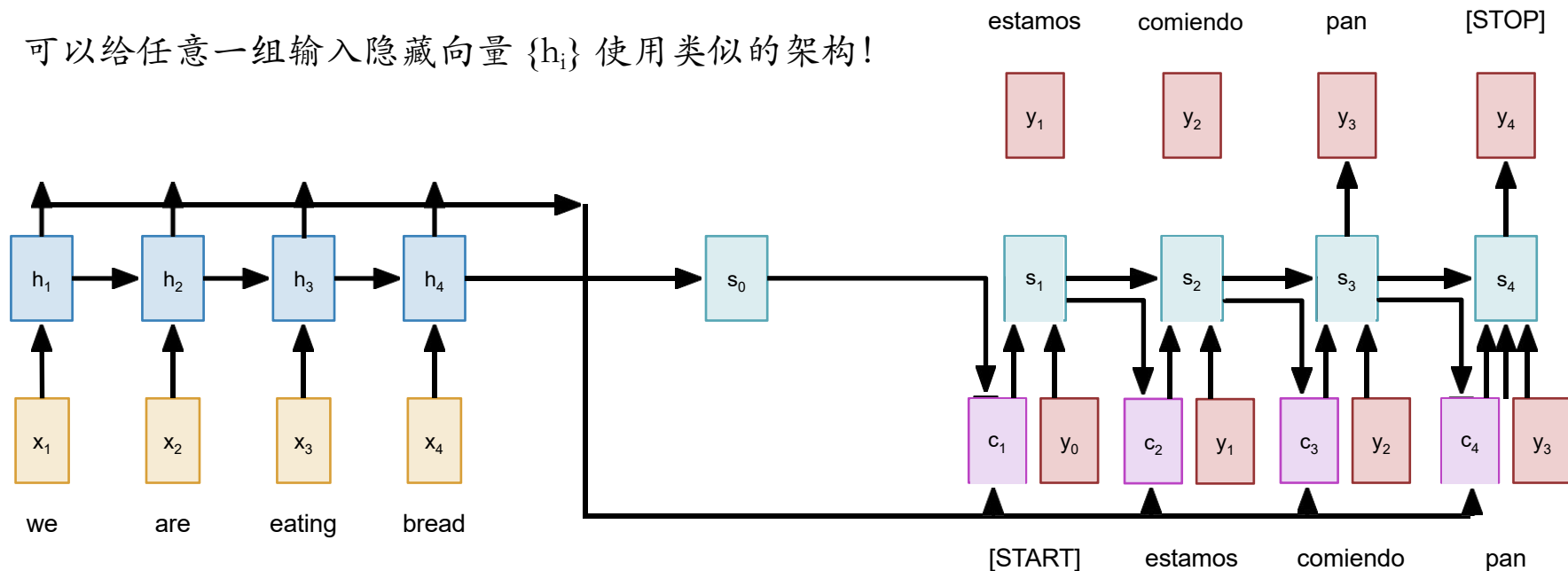
可视化注意力权重 $a_{t,i}$



使用 RNNs和Attention进行序列到序列分析

解码器并没有利用 h_i 构成有序序列这一事实，而是将它们视为无序集合 $\{h_i\}$

可以给任意一组输入隐藏向量 $\{h_i\}$ 使用类似的架构！



利用空间特征进行图像描述

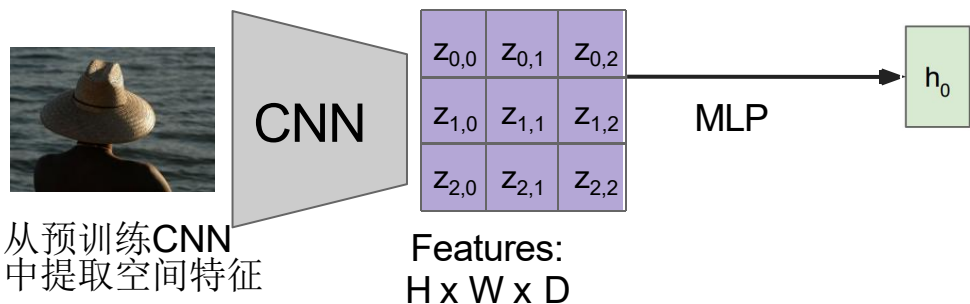
Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $h_0 = f_w(\mathbf{z})$

$\mathbf{z}$ 是空间 CNN 特征

$f_w(\cdot)$ 是一个 MLP



利用空间特征进行图像描述

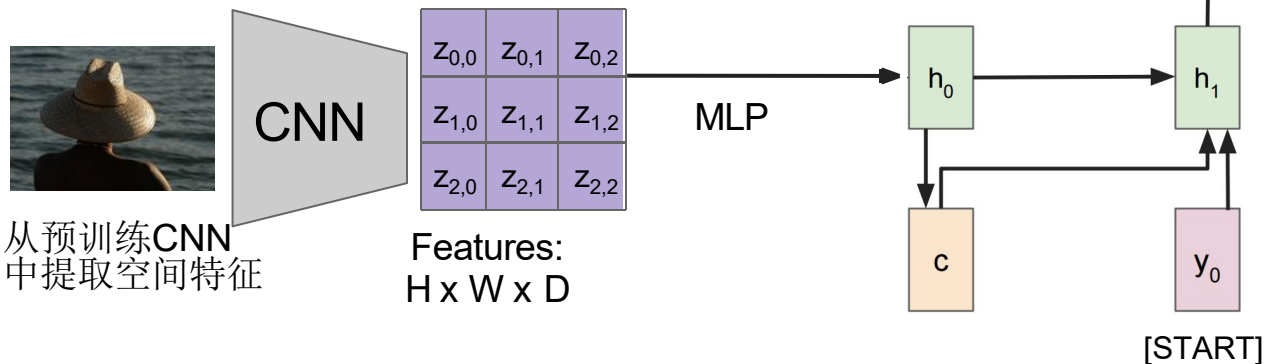
Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $h_0 = f_w(\mathbf{z})$

$\mathbf{z}$ 是空间 CNN 特征

$f_w(\cdot)$ 是一个 MLP



利用空间特征进行图像描述

Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $h_0 = f_w(\mathbf{z})$

$\mathbf{z}$ 是空间 CNN 特征

$f_w(\cdot)$ 是一个 MLP

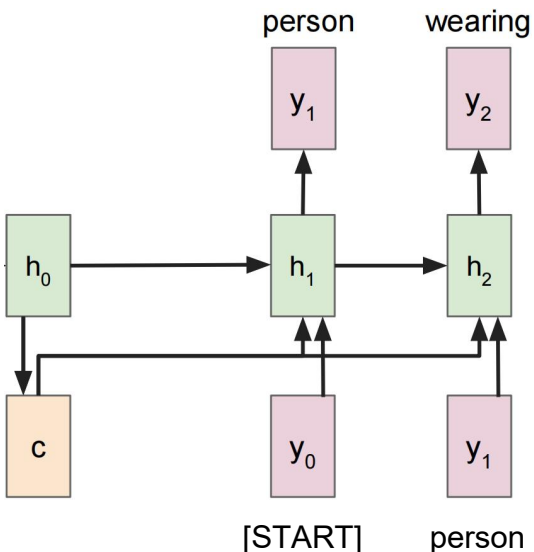


CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
 $H \times W \times D$

MLP



利用空间特征进行图像描述

Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $h_0 = f_w(\mathbf{z})$

$\mathbf{z}$ 是空间 CNN 特征

$f_w(\cdot)$ 是一个 MLP

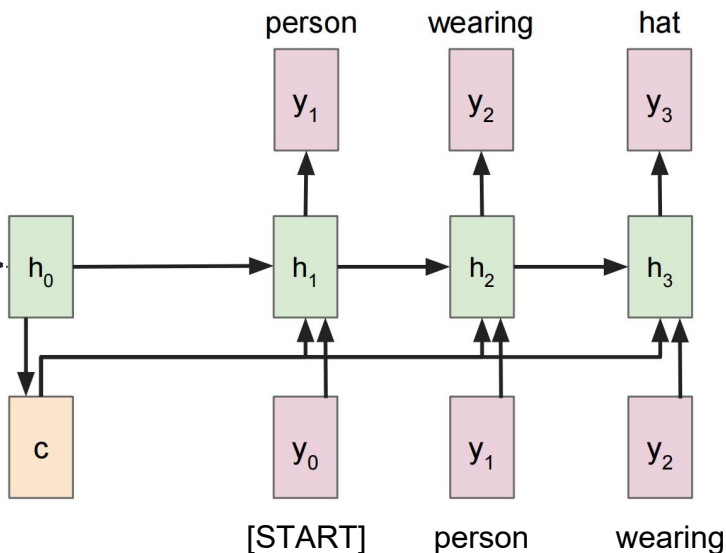


CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
 $H \times W \times D$

MLP



利用空间特征进行图像描述

Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $h_0 = f_w(\mathbf{z})$

$\mathbf{z}$ 是空间 CNN 特征

$f_w(\cdot)$ 是一个 MLP

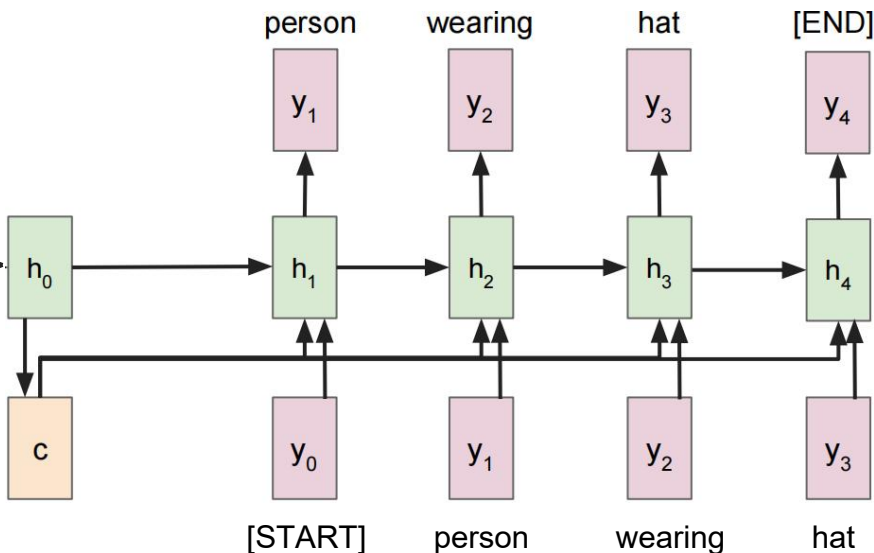


CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
 $H \times W \times D$

MLP



利用空间特征进行图像描述

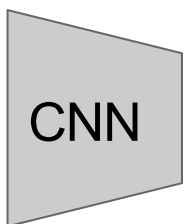
问题： 输入被“bottlenecked”在 c 中

- 模型必须将它想表达的所有信息都编码在 c 内部

如果我们想生成非常长的描述（长达 **100** 多个单词），这是否是个问题？



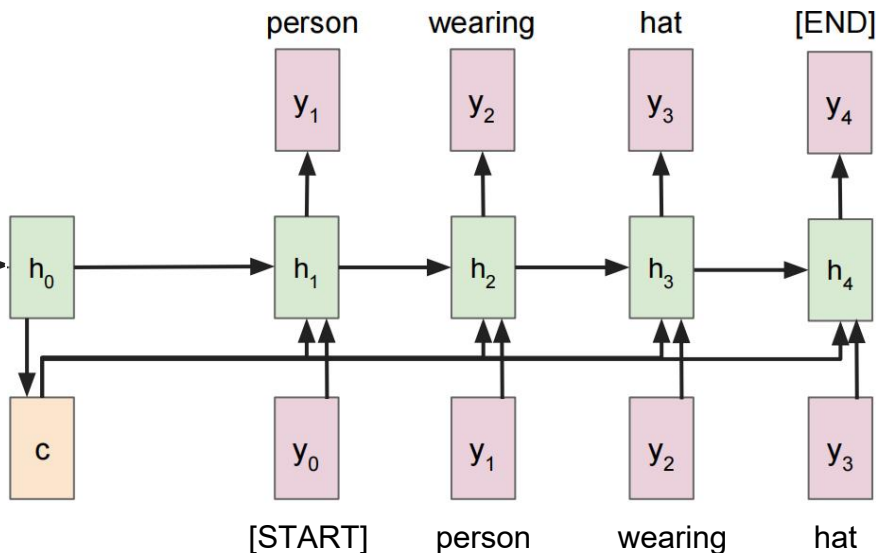
从预训练CNN
中提取空间特征



$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
 $H \times W \times D$

MLP



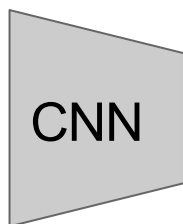
利用 RNN 和 Attention 进行图像描述

Attention理念： 每一步都有新的情境向量

每个上下文向量将关注不同的图像区域

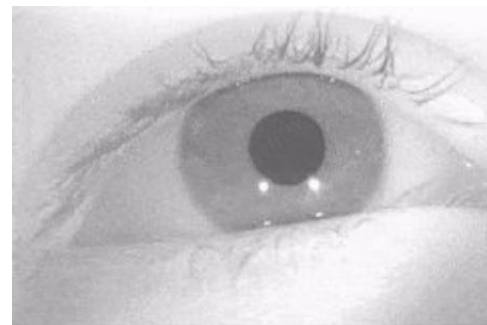


从预训练CNN
中提取空间特征



$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
 $H \times W \times D$



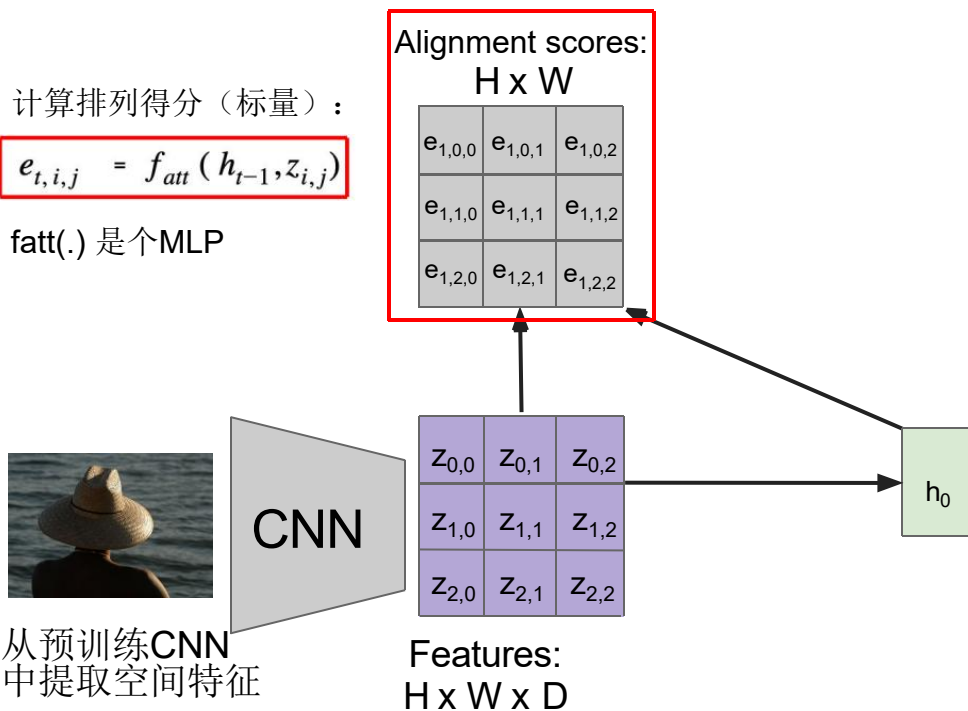
人类的注意性扫视

利用 RNN 和 Attention 进行图像描述

计算排列得分（标量）：

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

$f_{att}(\cdot)$ 是个 MLP



利用 RNN 和 Attention 进行图像描述

计算排列得分 (标量):

$$a_{t,:,:} = \text{softmax}(e_{t,:,:})$$

fatt(.) 是个 MLP

Alignment scores:
H x W

$e_{1,0,0}$	$e_{1,0,1}$	$e_{1,0,2}$
$e_{1,1,0}$	$e_{1,1,1}$	$e_{1,1,2}$
$e_{1,2,0}$	$e_{1,2,1}$	$e_{1,2,2}$

Attention:
H x W

$a_{1,0,0}$	$a_{1,0,1}$	$a_{1,0,2}$
$a_{1,1,0}$	$a_{1,1,1}$	$a_{1,1,2}$
$a_{1,2,0}$	$a_{1,2,1}$	$a_{1,2,2}$

Normalize 来获得
注意力权重:

$$a_{t,:,:} = \text{softmax}(e_{t,:,:})$$

$0 < a_{t,i,j} < 1$,
注意力值相加为1



CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D

h_0

利用 RNN 和 Attention 进行图像描述

计算排列得分 (标量):

$$a_{t,:,:} = \text{softmax}(e_{t,:,:})$$

fatt(.) 是个 MLP



CNN

从预训练CNN
中提取空间特征

Alignment scores:
H x W

$e_{1,0,0}$	$e_{1,0,1}$	$e_{1,0,2}$
$e_{1,1,0}$	$e_{1,1,1}$	$e_{1,1,2}$
$e_{1,2,0}$	$e_{1,2,1}$	$e_{1,2,2}$

Attention:
H x W

$a_{1,0,0}$	$a_{1,0,1}$	$a_{1,0,2}$
$a_{1,1,0}$	$a_{1,1,1}$	$a_{1,1,2}$
$a_{1,2,0}$	$a_{1,2,1}$	$a_{1,2,2}$

Normalize 来获得
注意力权重:

$$a_{t,:,:} = \text{softmax}(e_{t,:,:})$$

$0 < a_{t,i,j} < 1$,
注意力值相加为1

计算上下文向量:

$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D

h_0

c_1



利用 RNN 和 Attention 进行图像描述

解码器的每个时间步都使用不同的 context vector, 查看输入图像的不同部分

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

$$a_{t,:} = \text{softmax}(e_{t,:})$$

$$c_t = \sum_{i,j} a_{t,i,j} z_{i,j}$$



CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D

h_0

person

y_1

h_1

c_1

y_0

[START]

Decoder: $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$

每个时间步长中新的 context vector

利用 RNN 和 Attention 进行图像描述

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:} = \text{softmax}(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



CNN

从预训练CNN
中提取空间特征

Alignment scores:
H x W

$e_{1,0,0}$	$e_{1,0,1}$	$e_{1,0,2}$
$e_{1,1,0}$	$e_{1,1,1}$	$e_{1,1,2}$
$e_{1,2,0}$	$e_{1,2,1}$	$e_{1,2,2}$

Attention:
H x W

$a_{1,0,0}$	$a_{1,0,1}$	$a_{1,0,2}$
$a_{1,1,0}$	$a_{1,1,1}$	$a_{1,1,2}$
$a_{1,2,0}$	$a_{1,2,1}$	$a_{1,2,2}$

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D

Decoder: $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$

每个时间步长中新的 context vector

person

y_1

h_1

c_1

y_0

c_2

[START]

利用 RNN 和 Attention 进行图像描述

解码器的每个时间步都使用不同的 context vector, 查看输入图像的不同部分

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{i,j}$$



CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D

h_0

person

wearing

y_1

y_2

h_1

h_2

c_1

y_0

c_2

y_1

[START]
[START]

person

Decoder: $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$
每个时间步长中新 context vector

利用 RNN 和 Attention 进行图像描述

解码器的每个时间步都使用不同的 context vector, 查看输入图像的不同部分

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{i,j}$$

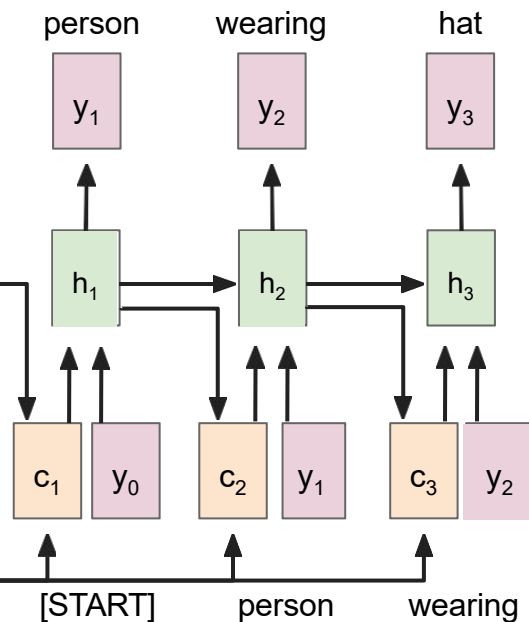


CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D

Decoder: $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$
每个时间步长中新 context vector



利用 RNN 和 Attention 进行图像描述

解码器的每个时间步都使用不同的 context vector, 查看输入图像的不同部分

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{i,j}$$

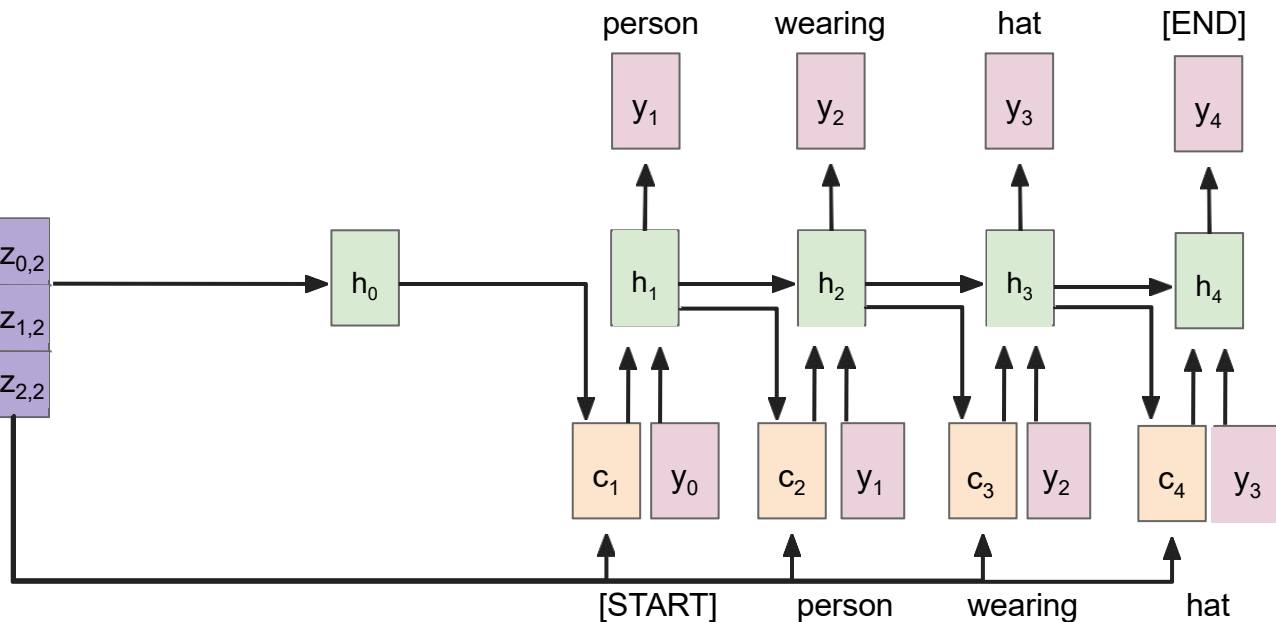


CNN

从预训练CNN
中提取空间特征

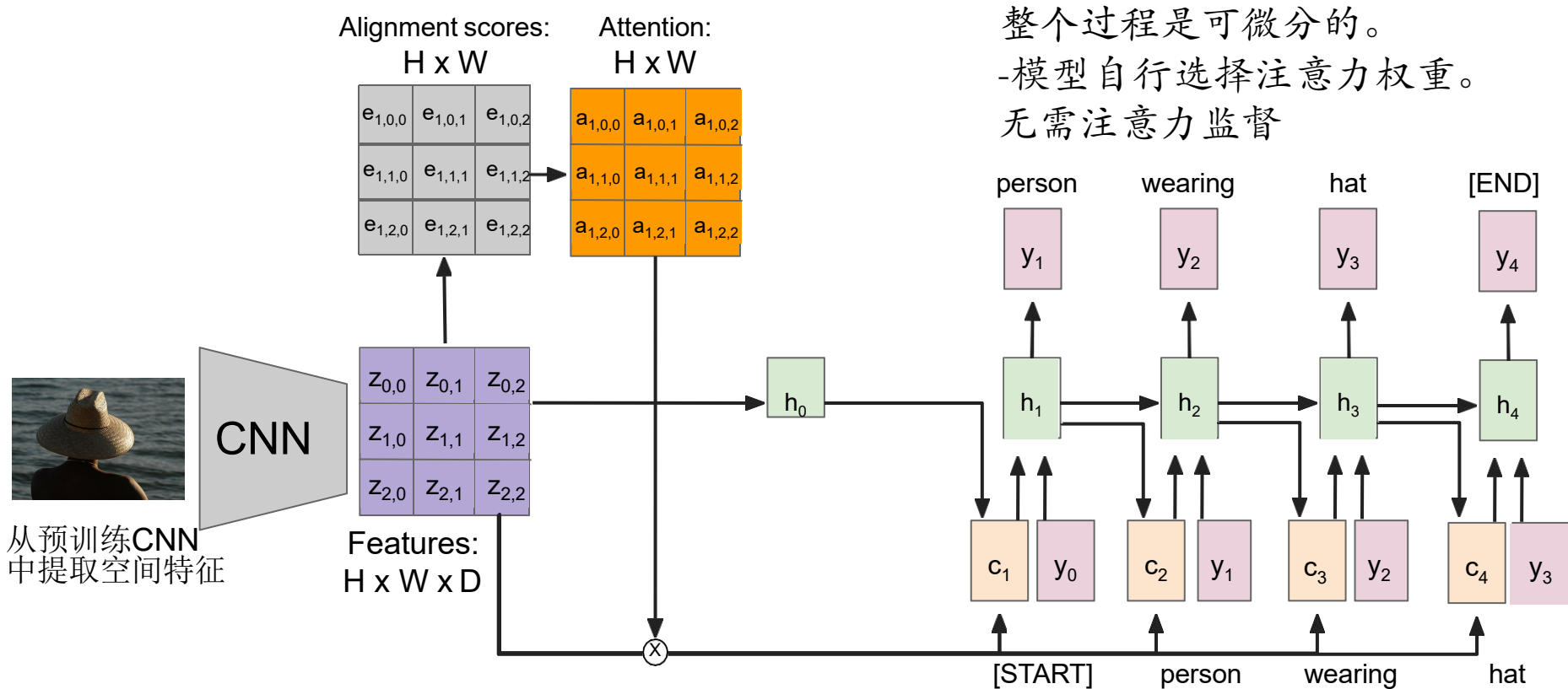
$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D



Decoder: $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$
每个时间步长中新 context vector

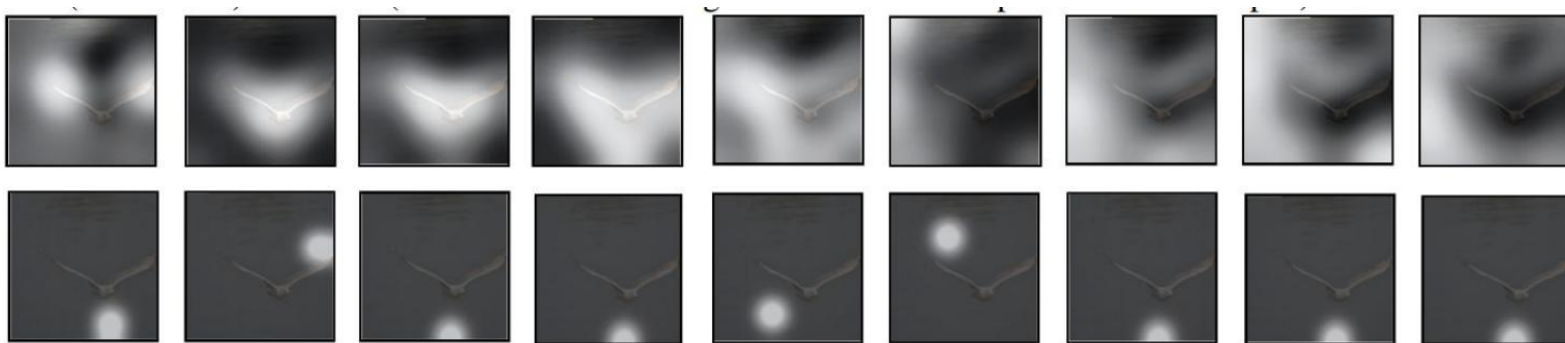
利用 RNN 和 Attention 进行图像描述



Xu et al, "Show, Attend and Tell: Neural ImageCaption Generation with Visual Attention", ICML 2015

用注意力机制进行图像描述

Soft attention



A

bird

flying

over

a

body

of

water

.

Hard attention
(requires
reinforcement
learning)

Xu et al, "Show, Attend, and Tell: Neural ImageCaption Generation with Visual Attention", ICML 2015

Figure copyright Kelvin Xu, Jimmy Lei Ba, Jamie Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard S. Zemel, and Yoshua Bengio, 2015. Reproduced with permission.

用注意力机制进行图像描述



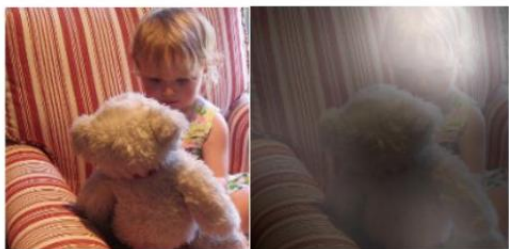
A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.

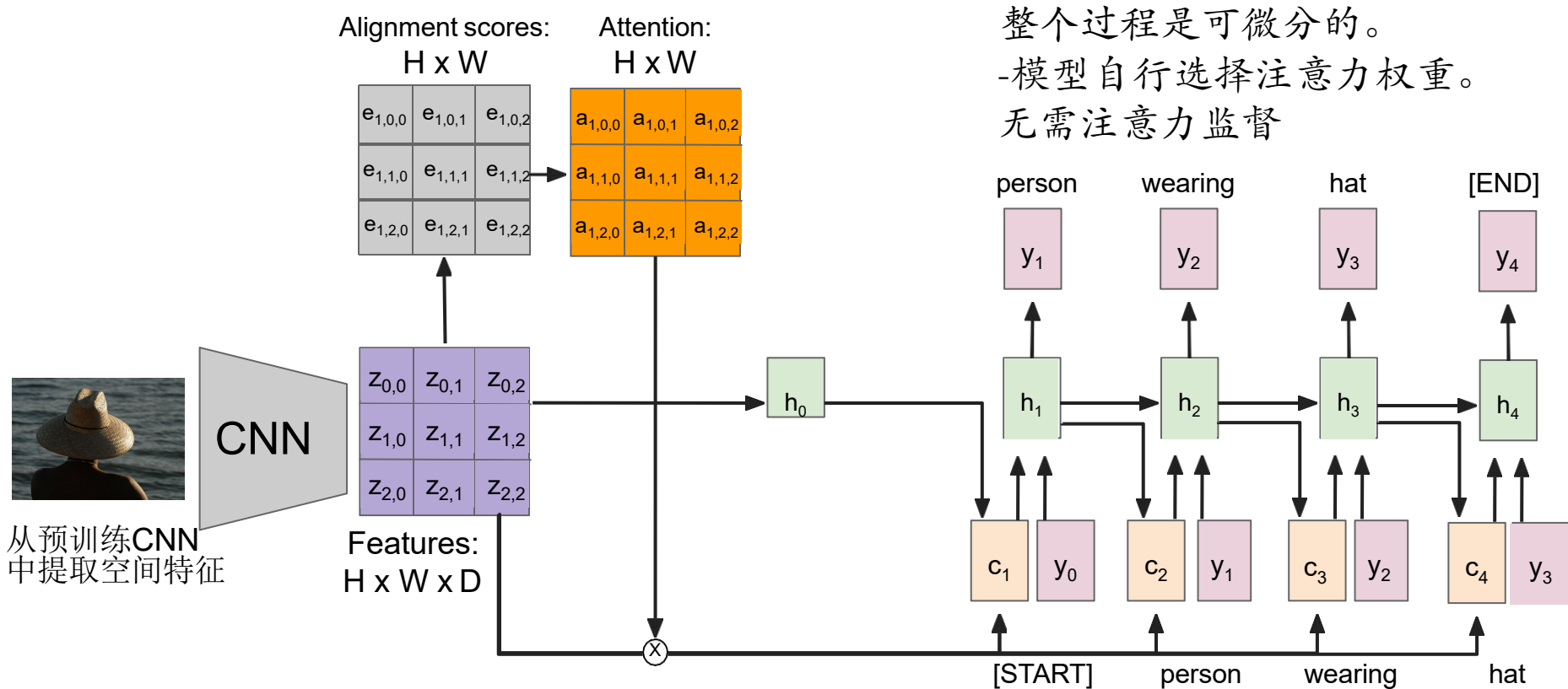


A giraffe standing in a forest with trees in the background.

Xu et al, "Show, Attend, and Tell: Neural ImageCaption Generation with Visual Attention", ICML 2015

Figure copyright Kelvin Xu, Jimmy Lei Ba, Jamie Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard S. Zemel, and Yoshua Bengio, 2015. Reproduced with permission.

利用 RNN 和 Attention 进行图像描述



Xu et al, "Show, Attend and Tell: Neural ImageCaption Generation with Visual Attention", ICML 2015

我们刚才在图像描述生成中看到的注意力机制

Features

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

h

Inputs:

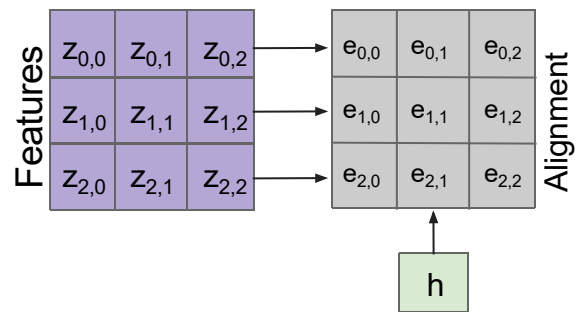
Features: $\mathbf{z}$ (shape: $H \times W \times D$)

Query: $\mathbf{h}$ (shape: D)

我们刚才在图像描述生成中看到的注意力机制

Operations:

Alignment: $e_{i,j} = f_{\text{att}}(h, z_{i,j})$

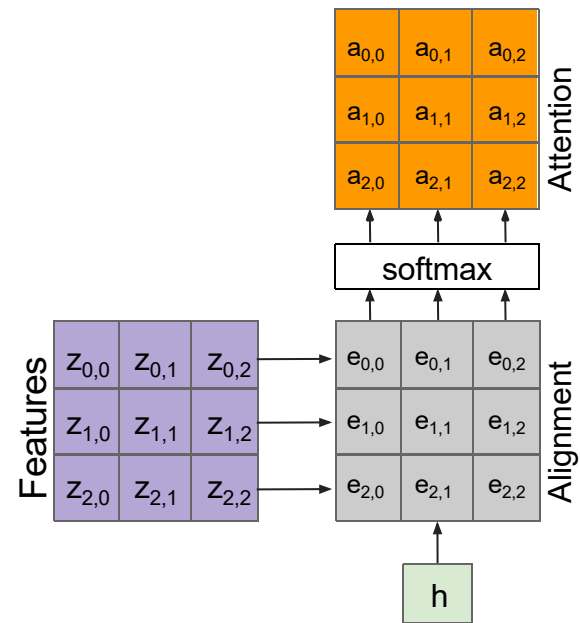


Inputs:

Features: $\mathbf{z}$ (shape: $H \times W \times D$)

Query: $\mathbf{h}$ (shape: D)

我们刚才在图像描述生成中看到的注意力机制



Operations:

Alignment: $e_{i,j} = f_{\text{att}}(h, z_{i,j})$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

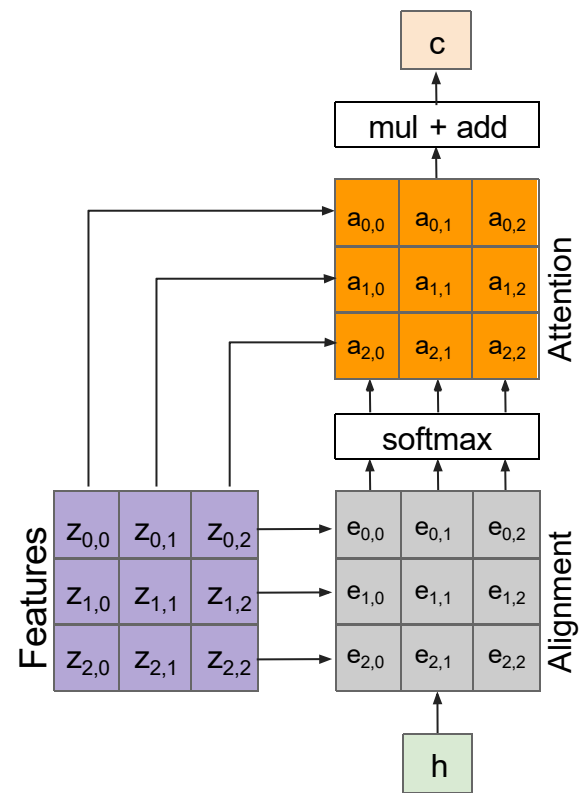
Output: $\mathbf{c} = \sum_{i,j} a_{i,j} z_{i,j}$

Inputs:

Features: $\mathbf{z}$ (shape: $H \times W \times D$)

Query: $\mathbf{h}$ (shape: D)

我们刚才在图像描述生成中看到的注意力机制



Outputs:

context vector: **c** (shape: D)

Operations:

Alignment: $e_{i,j} = f_{\text{att}}(h, z_{i,j})$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

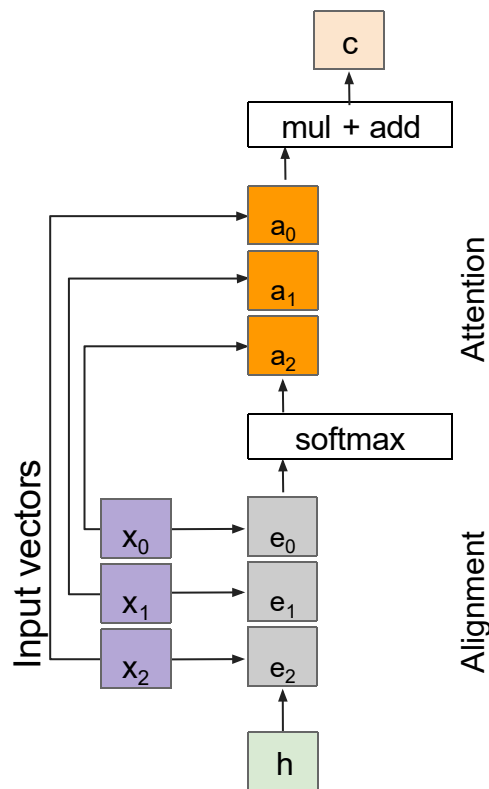
Output: $\mathbf{c} = \sum_{i,j} a_{i,j} z_{i,j}$

Inputs:

Features: **z** (shape: H x W x D)

Query: **h** (shape: D)

General attention layer



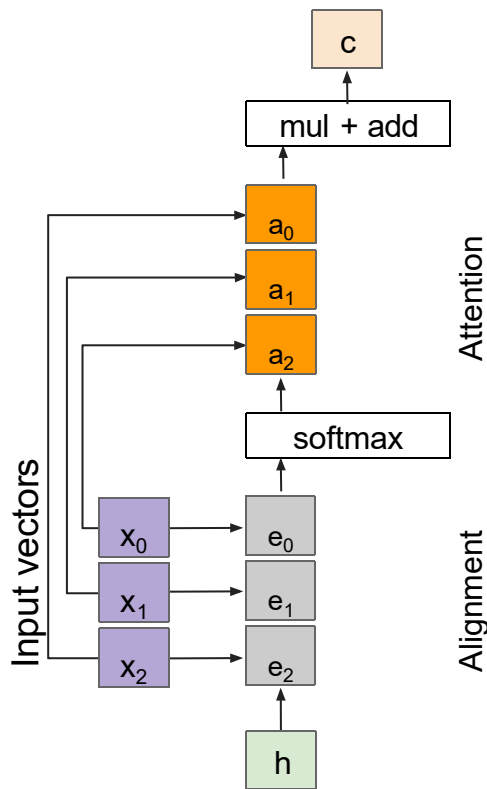
Outputs:
context vector: $\mathbf{c}$ (shape: D)

Operations:
Alignment: $e_i = f_{\text{att}}(h, x_i)$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $\mathbf{c} = \sum_i a_i x_i$

Inputs:
Input vectors: $\mathbf{x}$ (shape: $N \times D$)
Query: $\mathbf{h}$ (shape: D)

- 注意力操作具有置换不变性.
- 它不关心特征的排列顺序
 - 将 $H \times W = N$ 展开为 N 个向量

General attention layer



Outputs:

context vector: $\mathbf{c}$ (shape: D)

Operations:

Alignment: $\mathbf{e}_i = \mathbf{h} \cdot \mathbf{x}_i$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $\mathbf{c} = \sum_i \mathbf{a}_i \mathbf{x}_i$

Inputs:

Input vectors: $\mathbf{x}$ (shape: N x D)

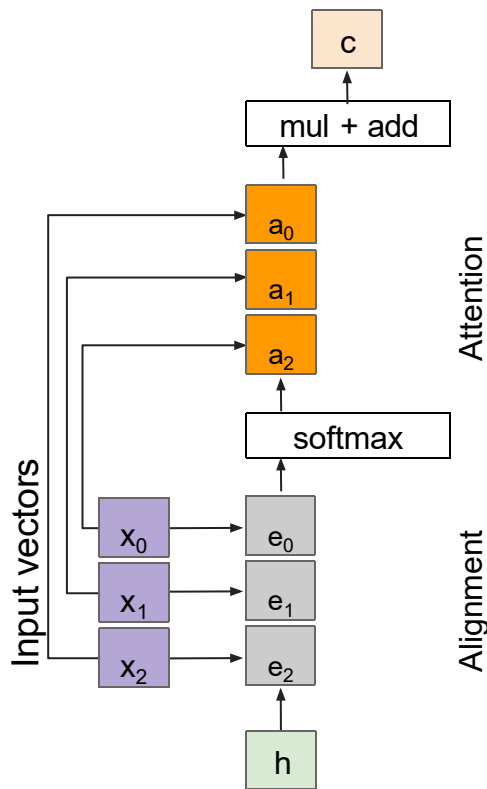
Query: $\mathbf{h}$ (shape: D)

将 $f_{\text{att}}(\cdot)$ 更改为简单的点积形式

仅在键 (Key) 和值 (Value) 上效果较好

转换技巧将在后面的几张幻灯片中提到。

General attention layer



Outputs:

context vector: $\mathbf{c}$ (shape: D)

Operations:

Alignment: $\mathbf{e}_i = \mathbf{h} \cdot \mathbf{x}_i / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $\mathbf{c} = \sum_i \mathbf{a}_i \mathbf{x}_i$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Query: $\mathbf{h}$ (shape: D)

将 $f_{\text{att}}(\cdot)$ 更改为带缩放的简单点积形式：
更高的维度意味着点积求和中包含更多项。

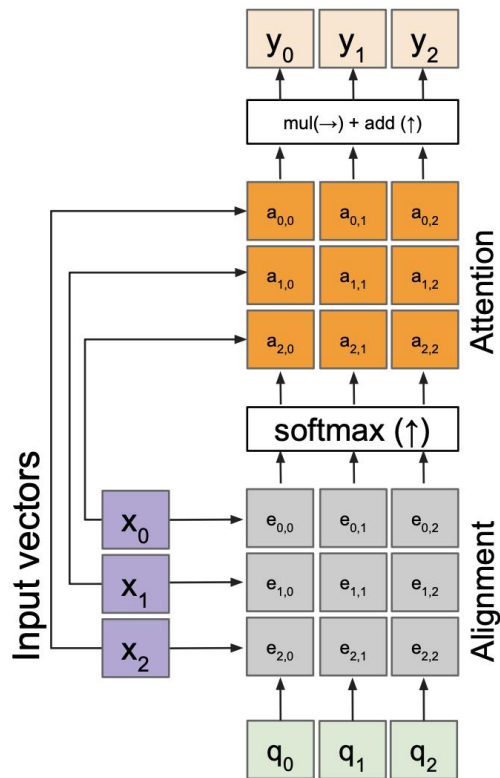
-因此，logits 的方差更大，向量幅值较大时会产生更高的 logits。

-假设 logits 独立同分布，softmax 之后的分布会具有较低的熵（更集中）。

-最终，这些高幅值向量会导致 softmax 输出高度偏向某一项，几乎忽略其他项的权重。

-为减弱大幅值向量的影响，通常将点积结果除以 $\sqrt{D}$

General attention layer



Outputs:

context vectors: $\mathbf{y}$ (shape: D)

多个查询向量

- 每个查询创建一个新的输出上下文向量

Operations:

Alignment: $e_{i,j} = q_j \cdot x_i / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $y_j = \sum_i a_{i,j} x_i$

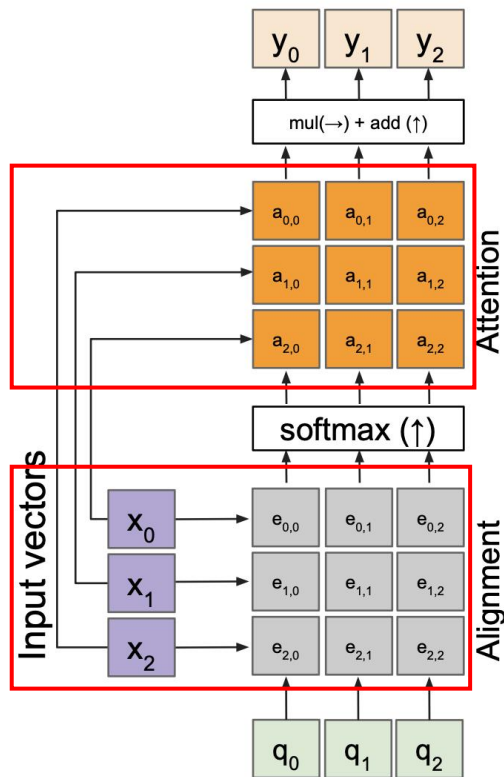
Inputs:

Input vectors: $\mathbf{x}$ (shape: N x D)

Queries: $\mathbf{q}$ (shape: M x D)

多个查询向量

General attention layer



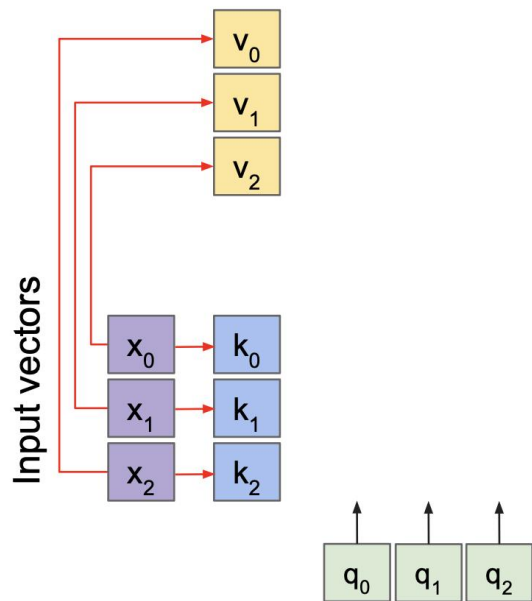
Outputs:
context vectors: $\mathbf{y}$ (shape: D)

Operations:
Alignment: $e_{i,j} = \mathbf{q}_j \cdot \mathbf{x}_i / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} x_i$

请注意，输入向量同时用于对齐和注意力计算。

- 我们可以通过在每个步骤之前添加不同的全连接层（FC层）来增加层的表达能力。

General attention layer



Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$
Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

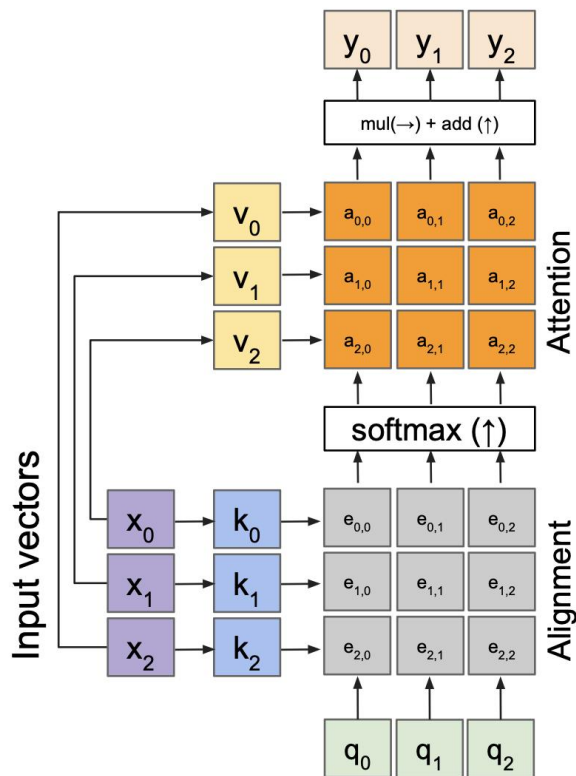
请注意，输入向量同时用于对齐计算和注意力计算。

- 我们可以通过在每个步骤之前添加不同的全连接（FC）层，来增加该层的表达能力。

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)
Queries: $\mathbf{q}$ (shape: $M \times D_k$)

General attention layer



Outputs:

context vectors: $\mathbf{y}$ (shape: D_y)

现在，输入和输出维度可以根据key和value的设置而变化。

Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $y_j = \sum_i a_{i,j} v_i$

注意到，输入向量同时用于对齐和注意力计算。

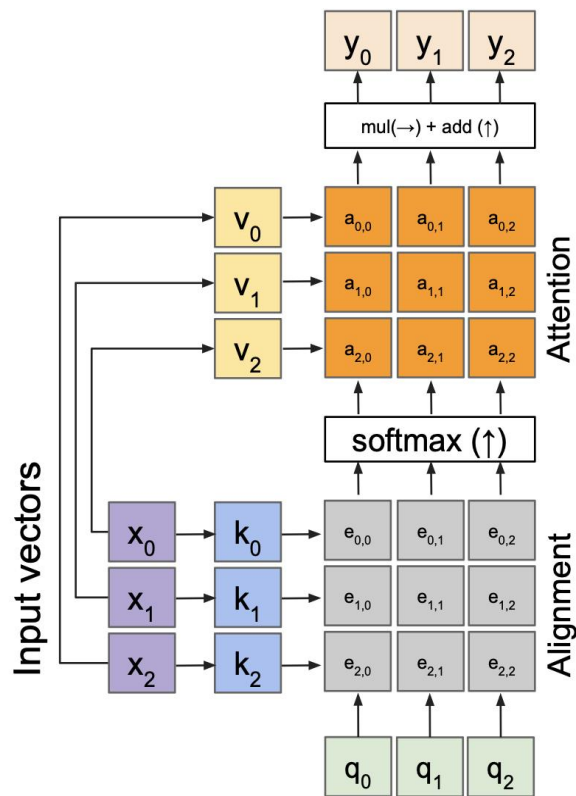
- 我们可以通过在两个步骤之前添加不同的全连接层来增强该层的表达能力。

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Queries: $\mathbf{q}$ (shape: $M \times D_q$)

General attention layer



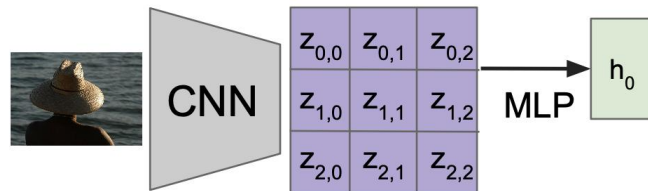
Outputs:
context vectors: y (shape: D_v)

Operations:
Key vectors: $k = xW_k$
Value vectors: $v = xW_v$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $a = \text{softmax}(e)$
Output: $y_j = \sum_i a_{i,j} v_i$

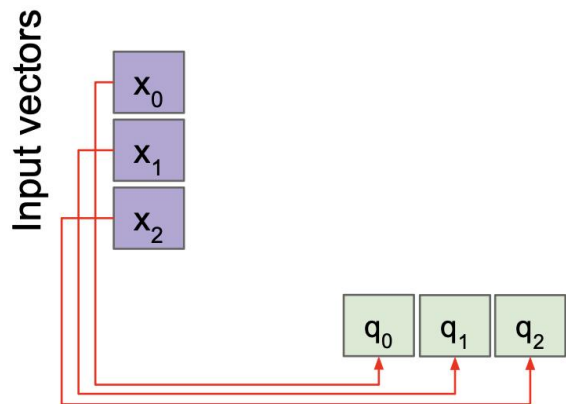
Inputs:
Input vectors: x (shape: $N \times D$)
Queries: q (shape: $M \times D_k$)

请记住，查询向量是输入向量的一个函数。

Encoder: $h_0 = f_w(z)$
where z is spatial CNN features
 $f_w(\cdot)$ is an MLP



Self attention layer



Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$

Alignment: $e_{i,j} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

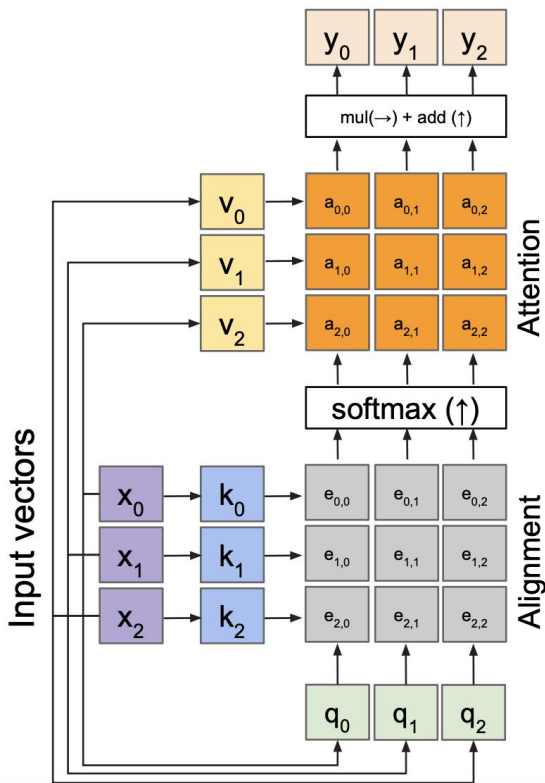
Queries: $\mathbf{q}$ (shape: $M \times D_k$)

我们可以从输入向量计算查询向量，因此定义了一个“自注意力”层。

查询向量是通过全连接（FC）层计算得出的。

不再输入查询向量了

Self attention layer



Outputs:
context vectors: $\mathbf{y}$ (shape: D_v)

Operations:

Key vectors: $\mathbf{k} = \mathbf{x}W_k$

Value vectors: $\mathbf{v} = \mathbf{x}W_v$

Query vectors: $\mathbf{q} = \mathbf{x}W_q$

Alignment: $e_{i,j} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D}$

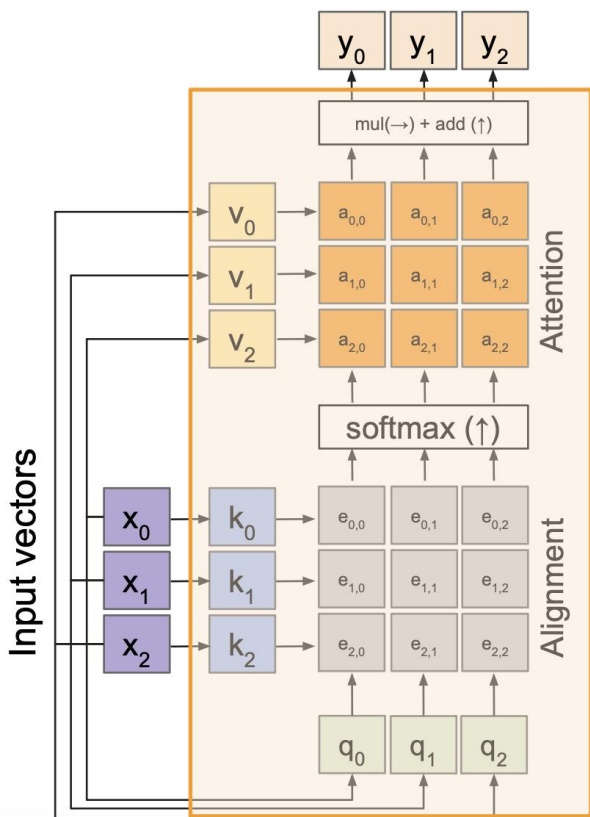
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Self attention layer-在一组输入进行注意力操作



Outputs:

context vectors: $\mathbf{y}$ (shape: D_v)

Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$

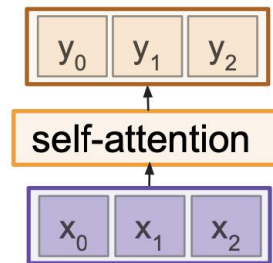
Alignment: $e_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

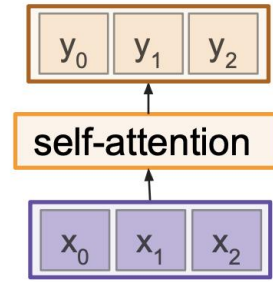
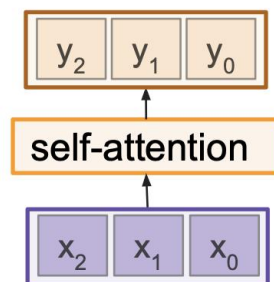
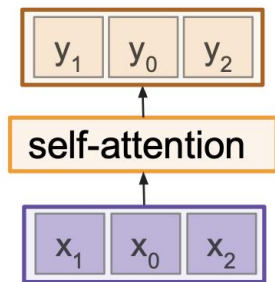
Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)



Self attention layer-在一组输入进行注意力操作

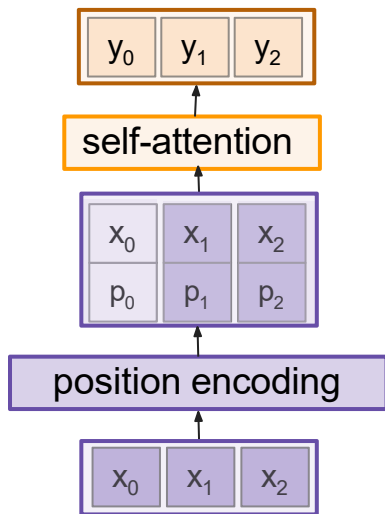


置换不变 (*Permutation equivariant*)

自注意力层不关心输入的顺序

Problem: 如何对像语言或在空间上有序的特征这样的有序序列进行编码?

位置编码



Concatenate/add special positional encoding p_j to each input vector x_j

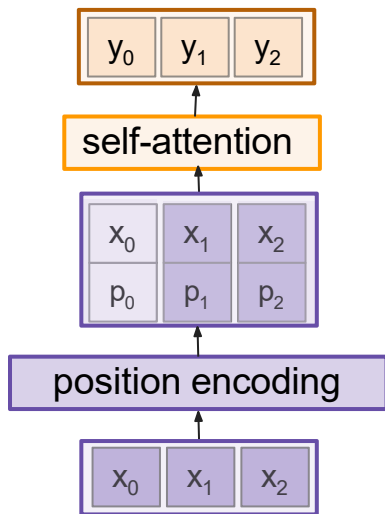
我们使用一个函数 $pos: N \rightarrow R^d$ 将向量的第 j 个位置处理为一个 d 维向量。

所以, $p_j = pos(j)$

$pos(\cdot)$ 的期望特性:

1. 它应该为每个时间步（句子中单词的位置）输出一个**唯一**的编码；
2. 任意两个时间步之间的**距离**应在不同长度的句子中保持一致；
3. 我们的模型应该能够在无需额外努力的情况下泛化到**更长**的句子。它的值应该是**有界的**；
4. 它必须是**确定性的**。

位置编码



Concatenate/add special positional encoding p_j to each input vector x_j

我们使用一个函数 $pos: N \rightarrow R^d$ 将向量的第 j 个位置处理为一个 d 维向量。

所以, $p_j = pos(j)$

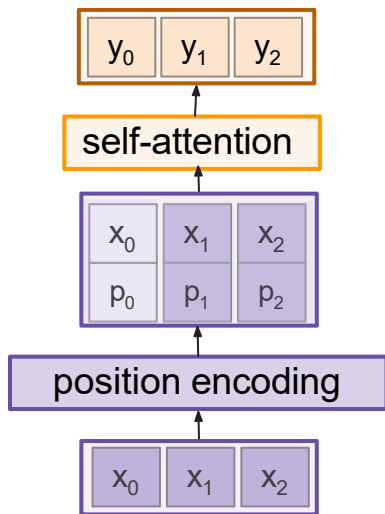
$pos(\cdot)$ 的选择

1. 学习一个查询表:
 - 学习参数用于 $pos(t)$ for $t \in [0, T)$
 - 查询表包含 $T \times d$ 个参数.

$pos(\cdot)$ 的期望特性:

1. 它应该为每个时间步 (句子中单词的位置) 输出一个**唯一**的编码;
2. 任意两个时间步之间的**距离**应在不同长度的句子中保持一致;
3. 我们的模型应该能够在无需额外努力的情况下泛化到**更长**的句子。它的值应该是**有界的**;
4. 它必须是**确定性的**。

位置编码



Concatenate/add special positional encoding p_j to each input vector x_j

我们使用一个函数 $pos: N \rightarrow R^d$ 将向量的第 j 个位置处理为一个 d 维向量。

所以, $p_j = pos(j)$

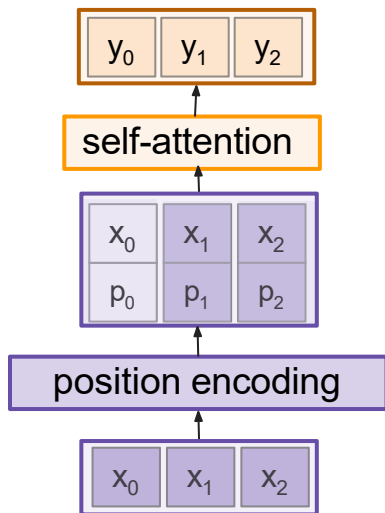
$pos(\cdot)$ 的选择

1. 学习一个查询表:
 - 学习参数用于 $pos(t)$ for $t \in [0, T)$
 - 查询表包含 $T \times d$ 个参数.
2. 由 $pos(t)$ 的期望特性设计一个固定的函数

$$p(t) = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \\ \sin(\omega_2 \cdot t) \\ \cos(\omega_2 \cdot t) \\ \\ \vdots \\ \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}_d$$

$$\text{where } \omega_k = \frac{1}{10000^{2k/d}}$$

位置编码



Concatenate/add special positional encoding p_j to each input vector x_j

我们使用一个函数 $pos: N \rightarrow R^d$ 将向量的第 j 个位置处理为一个 d 维向量。

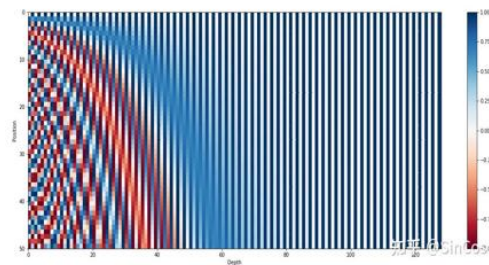
所以, $p_j = pos(j)$

$pos(\cdot)$ 的选择

1. 学习一个查询表:
 - 学习参数用于 $pos(t)$ for $t \in [0, T)$
 - 查询表包含 $T \times d$ 个参数.
2. 由 $pos(t)$ 的期望特性设计一个固定的函数

$$p(t) = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \sin(\omega_2 \cdot t) \\ \cos(\omega_2 \cdot t) \\ \vdots \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}_d$$

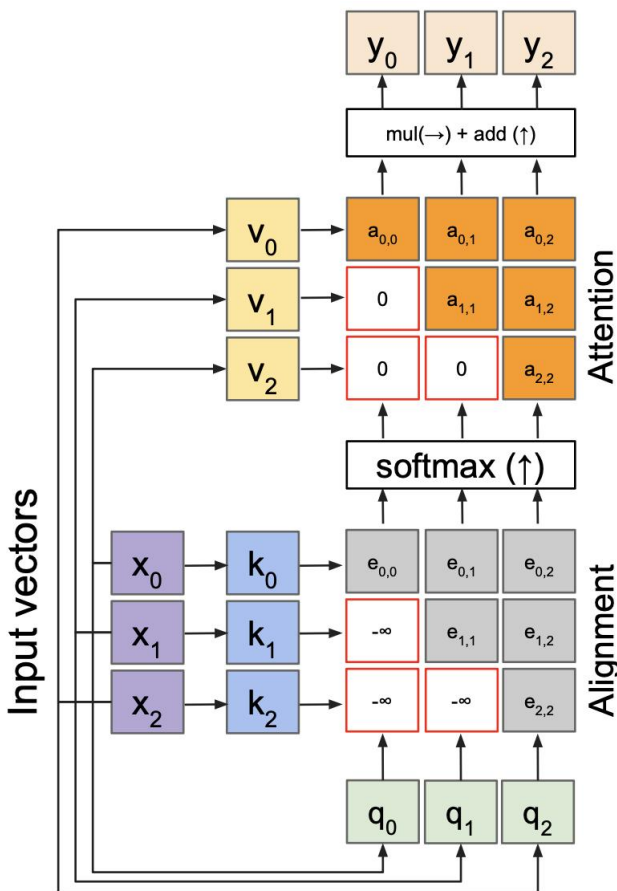
Intuition:



$$\text{where } \omega_k = \frac{1}{10000^{2k/d}}$$

[image source](#)

Masked self-attention layer



Outputs:
context vectors: y (shape: D_y)

Operations:
Key vectors: $k = xW_k$
Value vectors: $v = xW_v$
Query vectors: $q = xW_q$
Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$
Attention: $a = \text{softmax}(e)$
Output: $y_j = \sum_i a_{i,j} v_i$

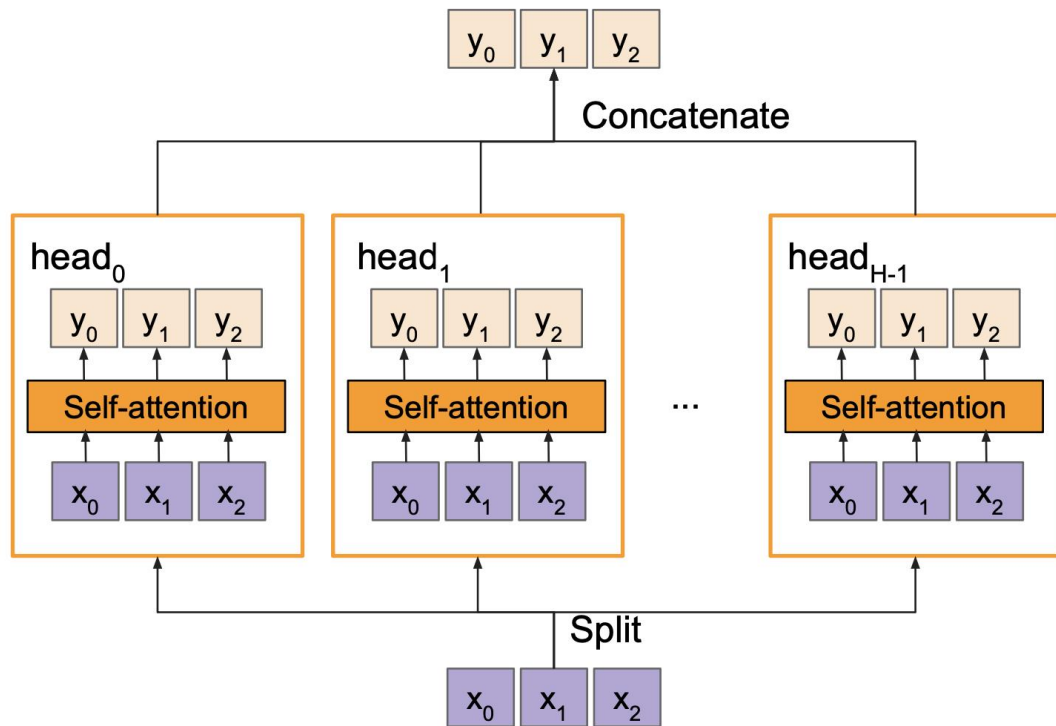
Inputs:
Input vectors: x (shape: $N \times D$)

防止向量查看未来的向量

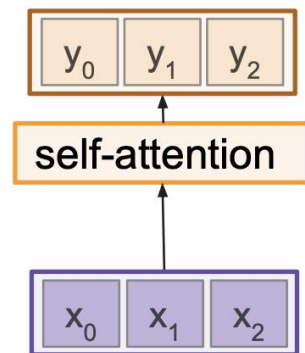
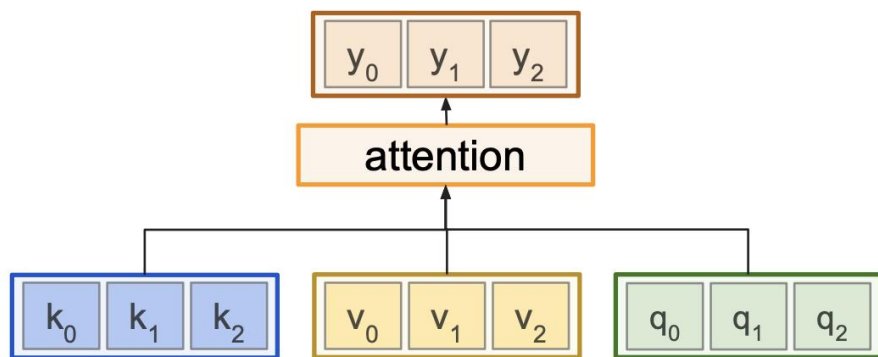
- 手动设置alignment scores
为负无穷

Multi-head self attention layer

- 多个自注意力头并行



General attention *vs.* self-attention



练习题

- **题目：Self-Attention中，Query和Key的点积作用是？**
 - A. 计算特征维度归一化
 - B. 衡量两个位置之间的相关性
 - C. 直接生成输出向量
 - D. 减少参数量
- **答案：B**

练习题

- **题目：Transformer的位置编码（Positional Encoding）用于？**
 - A. 替换输入嵌入
 - B. 引入序列的顺序信息
 - C. 减少注意力头数量
 - D. 加速梯度下降
- **答案：B**

练习题

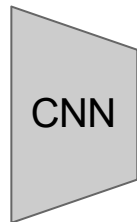
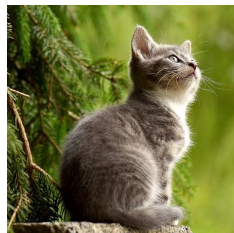
• **题目：相比RNN，Transformer的主要优势是？**

- A. 更高的参数效率
- B. 天然支持并行计算
- C. 无需激活函数
- D. 梯度永不消失

• **答案： B**

Example: 带自注意力机制的CNN

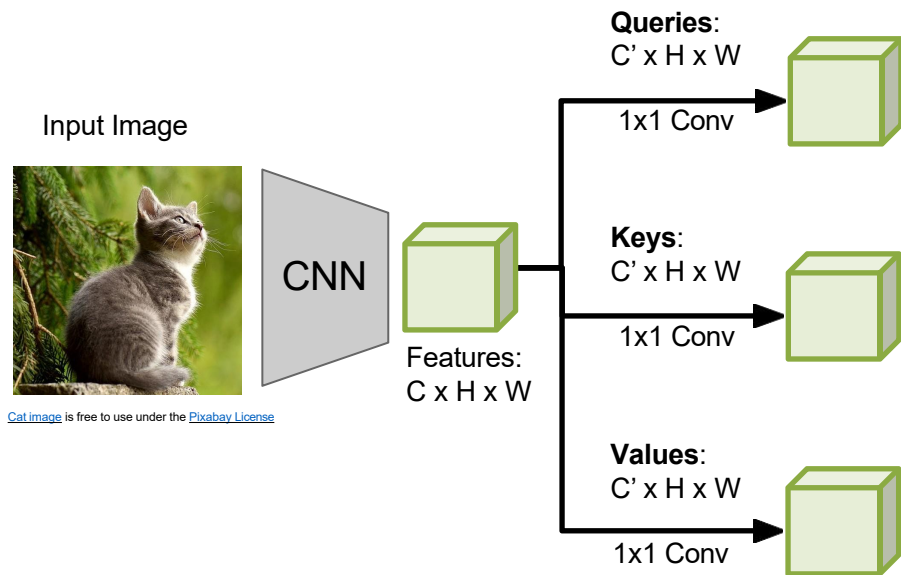
Input Image



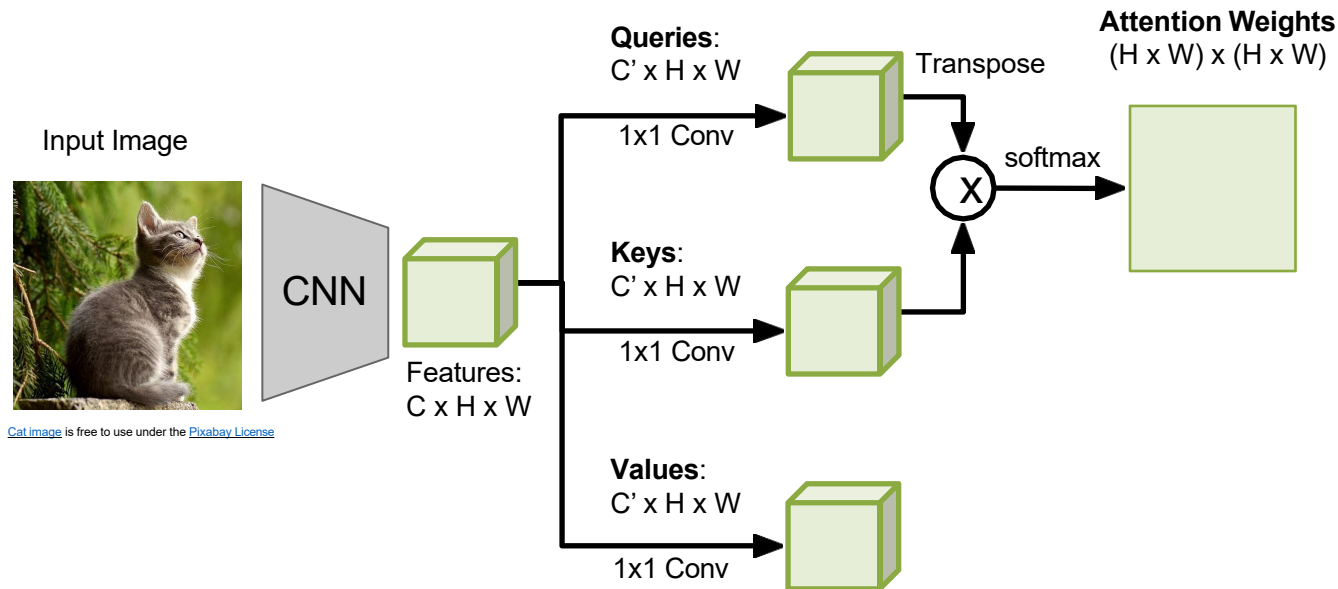
Features:
 $C \times H \times W$

[Cat image](#) is free to use under the [Pixabay License](#)

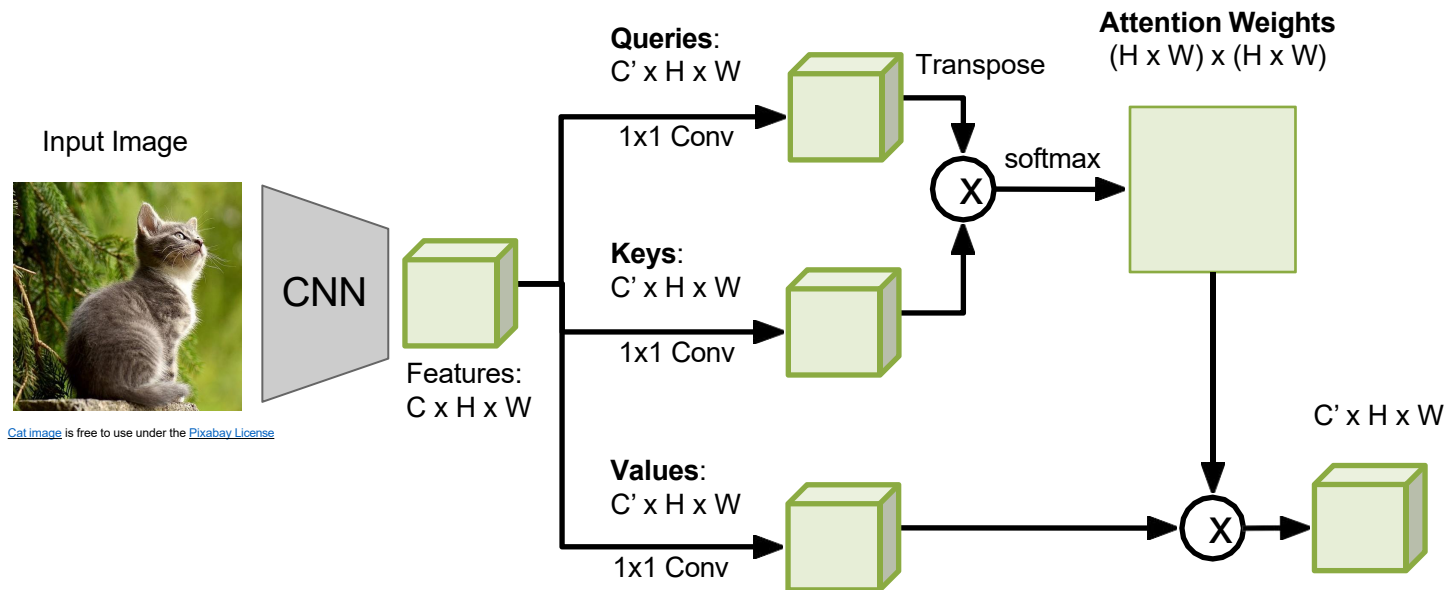
Example: 带自注意力机制的CNN



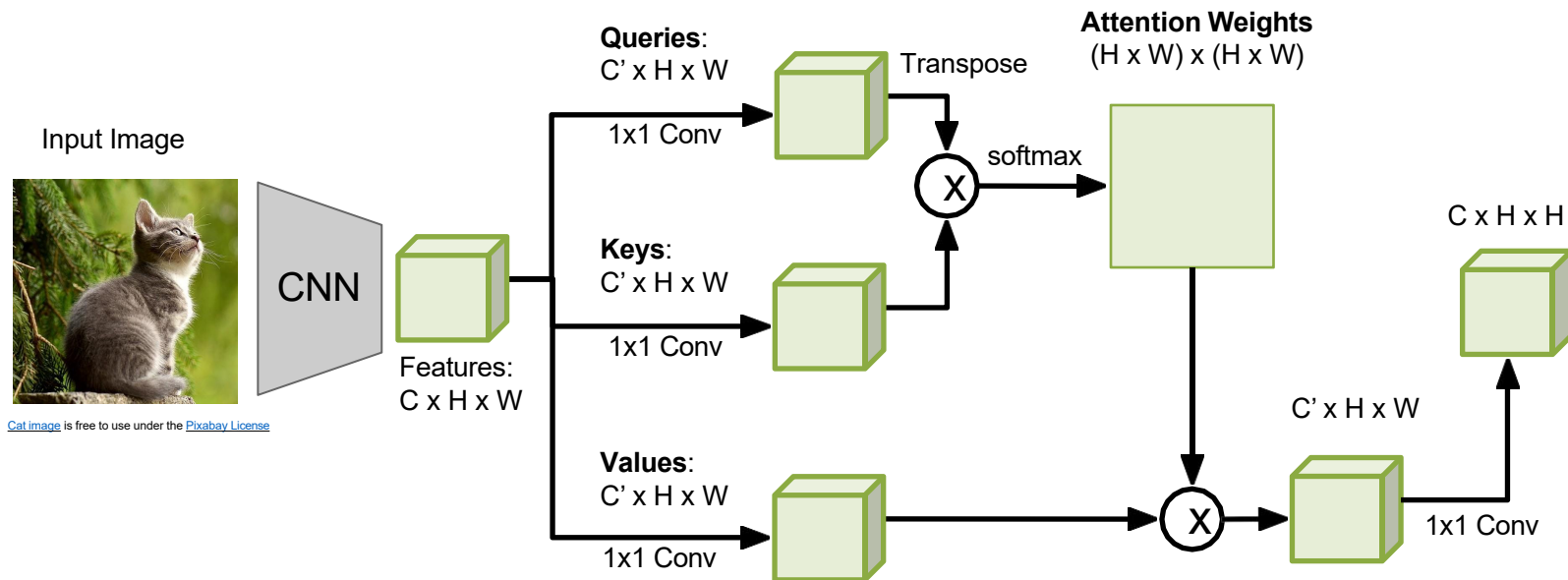
Example: 带自注意力机制的CNN



Example: 带自注意力机制的CNN

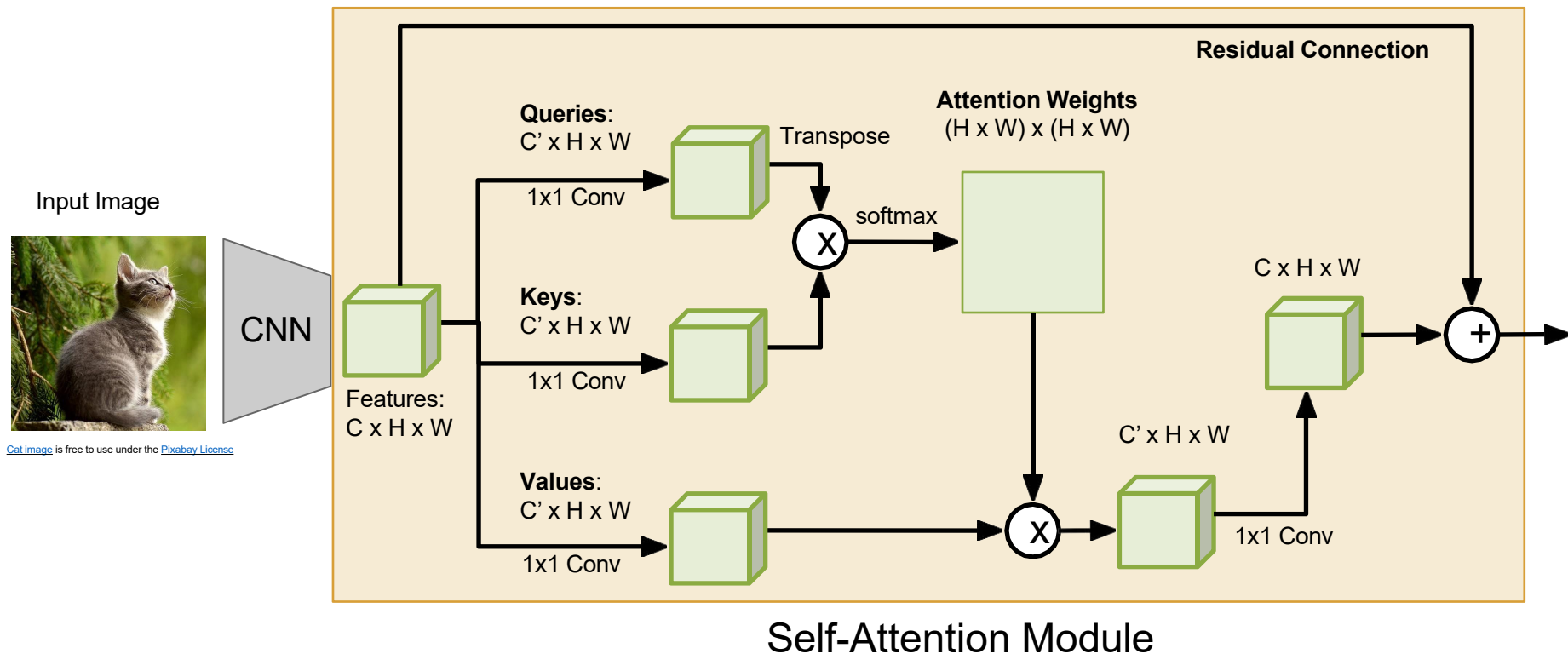


Example: 带自注意力机制的CNN



[Cat image](#) is free to use under the [Pixabay License](#)

Example: 带自注意力机制的CNN



RNNs 与 Transformer 的比较

RNNs

(+) **LSTMs** 在处理长序列时表现不错。

(-) 输入需是有序序列

(-) 顺序计算：后续的隐藏状态只能在前一个计算完成后才能进行。

Transformer:

(+) 擅长处理长序列。每一次注意力计算都会关注所有输入。

(+) 可以结合位置编码处理无序集合或有序序列。

(+) 并行计算：所有输入的对齐和注意力得分都可以同时完成。

(-) 需要大量内存：对于一个自注意力头，需要计算并存储 $N \times M$ 个对齐和注意力分数。

(但目前 **GPUs** 显存正在变得更大，性能变得更强)

Attention Is All You Need

Ashish Vaswani*

Google Brain

avaswani@google.com

Noam Shazeer*

Google Brain

noam@google.com

Niki Parmar*

Google Research

nikip@google.com

Jakob Uszkoreit*

Google Research

usz@google.com

Llion Jones*

Google Research

llion@google.com

Aidan N. Gomez*[†]

University of Toronto

aidan@cs.toronto.edu

Łukasz Kaiser*

Google Brain

lukaszkaiser@google.com

Illia Polosukhin*[‡]

illia.polosukhin@gmail.com

“ImageNet Moment for Natural Language Processing”

Pretraining:

Download a lot of text from the internet

Train a giant Transformer model for language modeling

Finetuning:

Fine-tune the Transformer on your own NLP task

On the Opportunities and Risks of Foundation Models

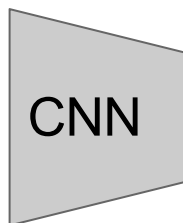
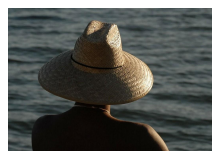
Rishi Bommasani* Drew A. Hudson Ehsan Adeli Russ Altman Simran Arora
Sydney von Arx Michael S. Bernstein Jeannette Bohg Antoine Bosselut Emma Brunskill
Erik Brynjolfsson Shyamal Buch Dallas Card Rodrigo Castellon Niladri Chatterji
Annie Chen Kathleen Creel Jared Quincy Davis Dorottya Demszky Chris Donahue
Moussa Doumbouya Esin Durmus Stefano Ermon John Etchemendy Kawin Ethayarajh
Li Fei-Fei Chelsea Finn Trevor Gale Lauren Gillespie Karan Goel Noah Goodman
Shelby Grossman Neel Guha Tatsunori Hashimoto Peter Henderson John Hewitt
Daniel E. Ho Jenny Hong Kyle Hsu Jing Huang Thomas Icard Saahil Jain
Dan Jurafsky Pratyusha Kalluri Siddharth Karamcheti Geoff Keeling Fereshte Khani
Omar Khattab Pang Wei Koh Mark Krass Ranjay Krishna Rohith Kuditipudi
Ananya Kumar Faisal Ladhak Mina Lee Tony Lee Jure Leskovec Isabelle Levent
Xiang Lisa Li Xuechen Li Tengyu Ma Ali Malik Christopher D. Manning
Suvir Mirchandani Eric Mitchell Zanele Munyikwa Suraj Nair Avanika Narayan
Deepak Narayanan Ben Newman Allen Nie Juan Carlos Niebles Hamed Nilforoshan
Julian Nyarko Giray Ogut Laurel Orr Isabel Papadimitriou Joon Sung Park Chris Piech
Eva Portelance Christopher Potts Aditi Raghunathan Rob Reich Hongyu Ren
Frieda Rong Yusuf Roohani Camilo Ruiz Jack Ryan Christopher Ré Dorsa Sadigh
Shiori Sagawa Keshav Santhanam Andy Shih Krishnan Srinivasan Alex Tamkin
Rohan Taori Armin W. Thomas Florian Tramèr Rose E. Wang William Wang Bohan Wu
Jiajun Wu Yuhuai Wu Sang Michael Xie Michihiro Yasunaga Jiaxuan You Matei Zaharia
Michael Zhang Tianyi Zhang Xikun Zhang Yuhui Zhang Lucia Zheng Kaitlyn Zhou
Percy Liang*¹

Center for Research on Foundation Models (CRFM)
Stanford Institute for Human-Centered Artificial Intelligence (HAI)
Stanford University

Image Captioning using Transformers

Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$



$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

从一个预训练的
CNN提取空
间特征

Features:
 $H \times W \times D$

Image Captioning using Transformers

Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $\mathbf{c} = T_w(\mathbf{z})$

其中 $\mathbf{z}$ 是 CNN 提取的空间特征

$T_w(\cdot)$ 是 transformer 的 encoder

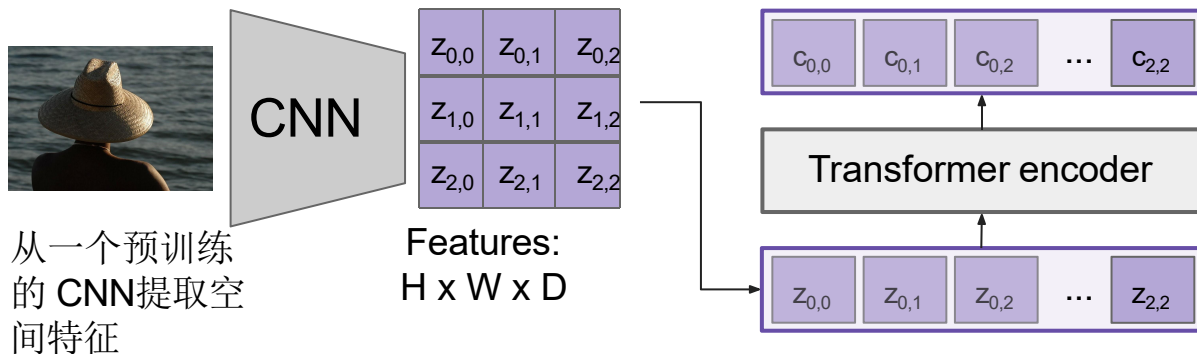


Image Captioning using Transformers

Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $\mathbf{c} = T_w(\mathbf{z})$

其中 $\mathbf{z}$ 是 CNN 提取的空间特征

$T_w(\cdot)$ 是 transformer 的 encoder

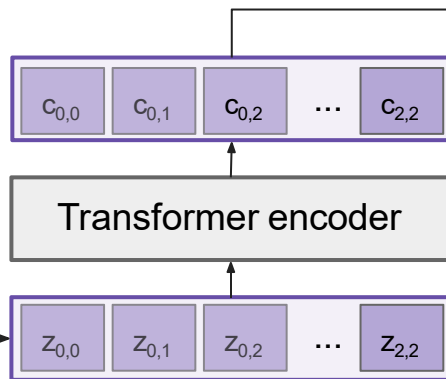


CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

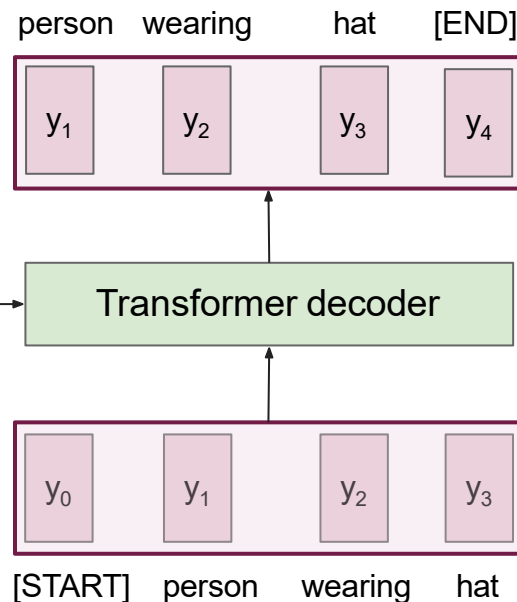
Features:
 $H \times W \times D$

从一个预训练的
CNN 提取空
间特征

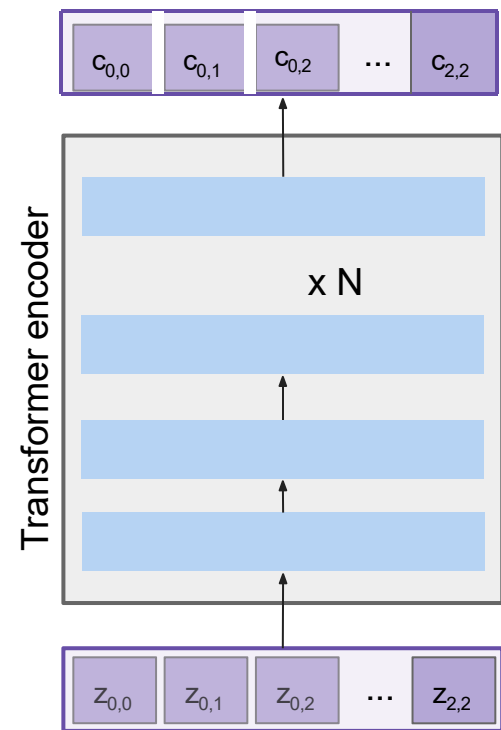


Decoder: $y_t = T_D(y_{0:t-1}, \mathbf{c})$

其中, $T_D(\cdot)$ 是 transformer 的 decoder



The Transformer *encoder* block

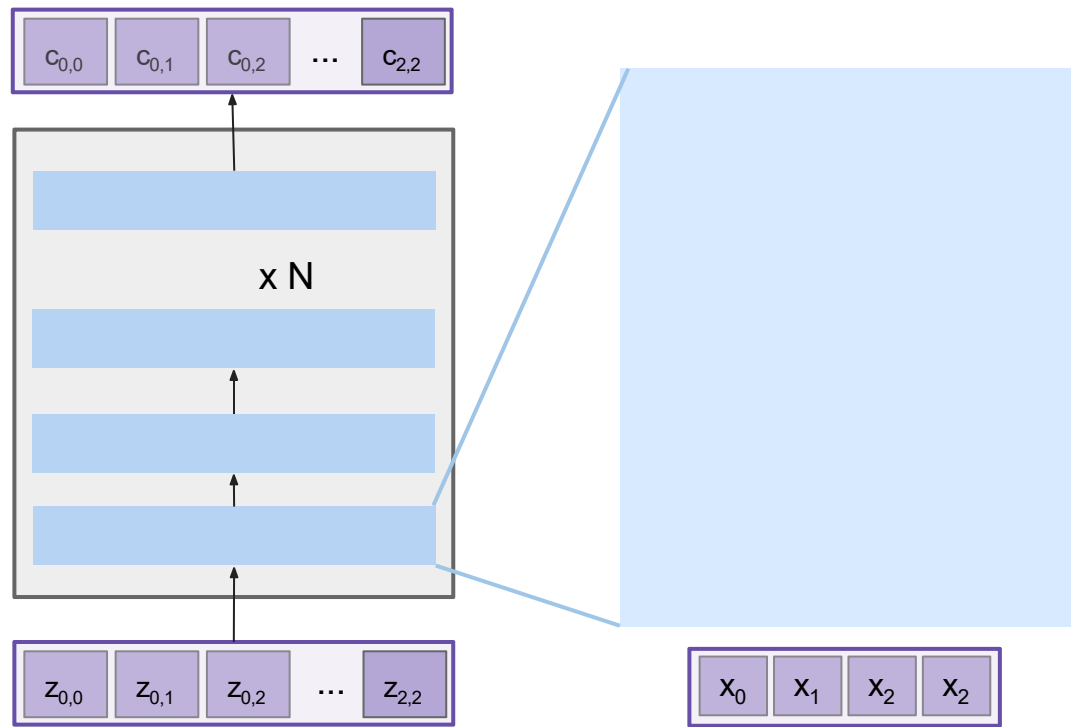


由N个encoder块组成

原论文中 $N = 6$, $D_q = 512$

Vaswani et al, "Attention is all you need", NeurIPS 2017

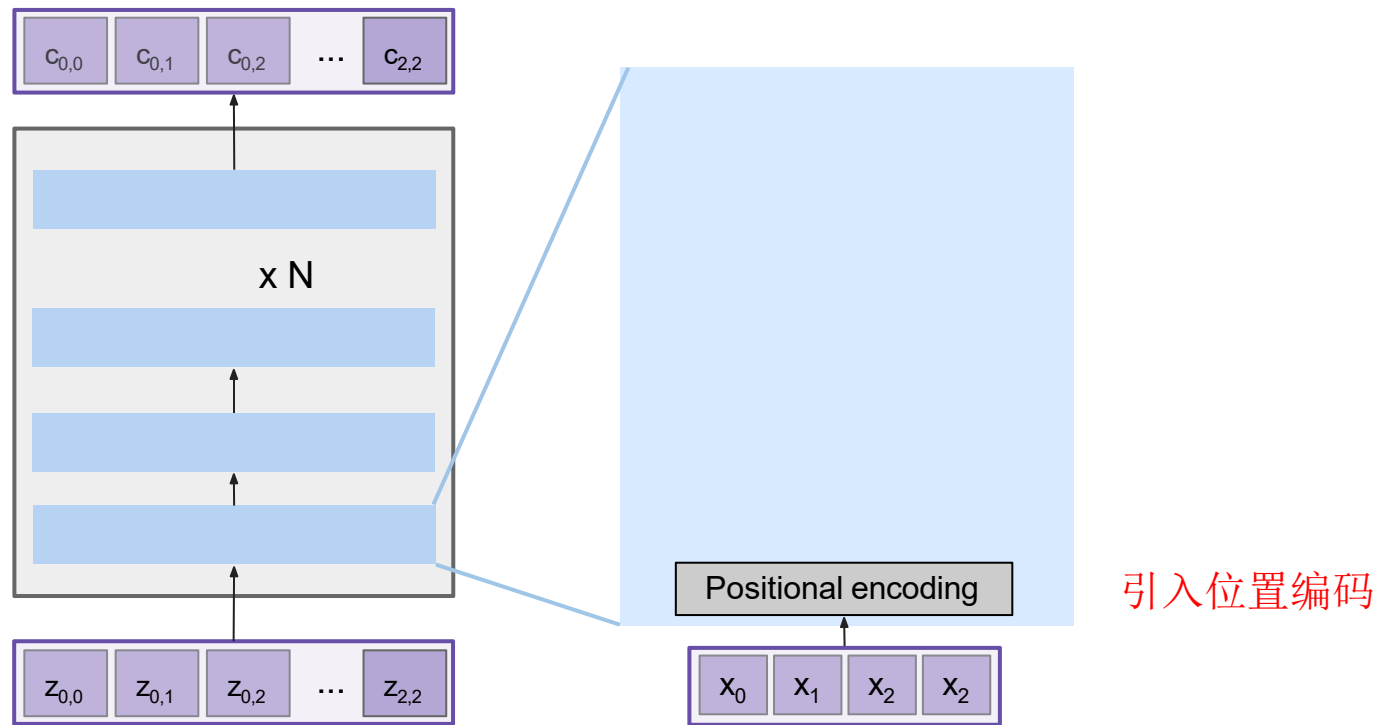
The Transformer *encoder* block



让我们来深入了解一个encoder块

Vaswani et al, "Attention is all you need", NeurIPS 2017

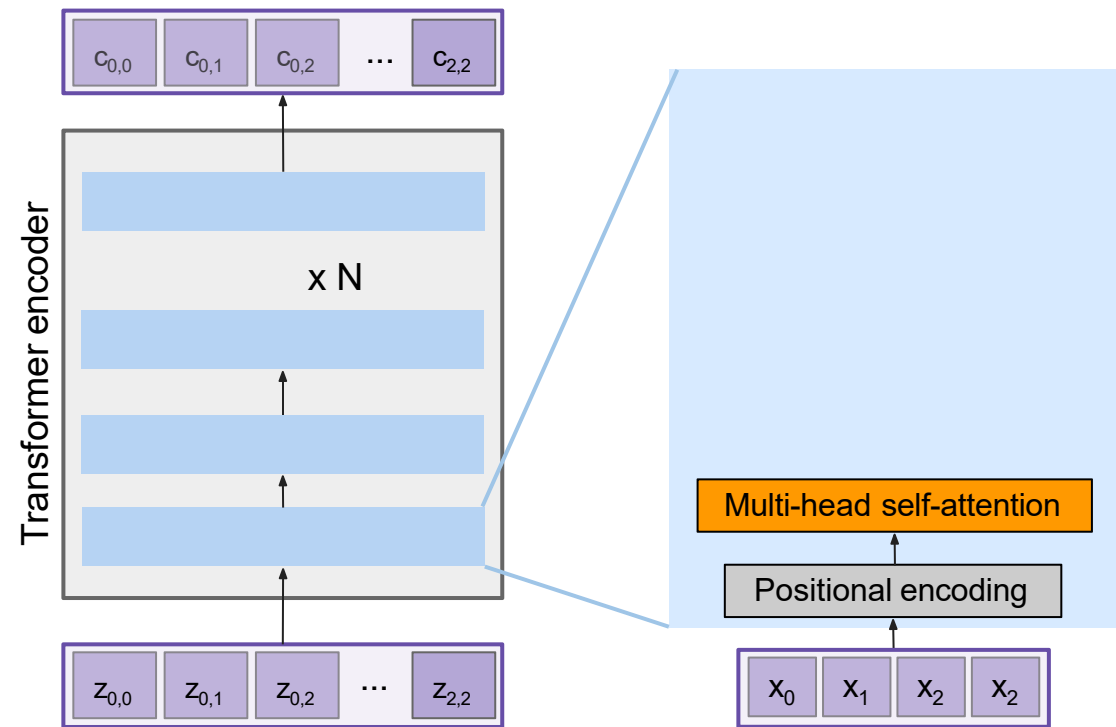
The Transformer *encoder* block



引入位置编码

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer *encoder* block

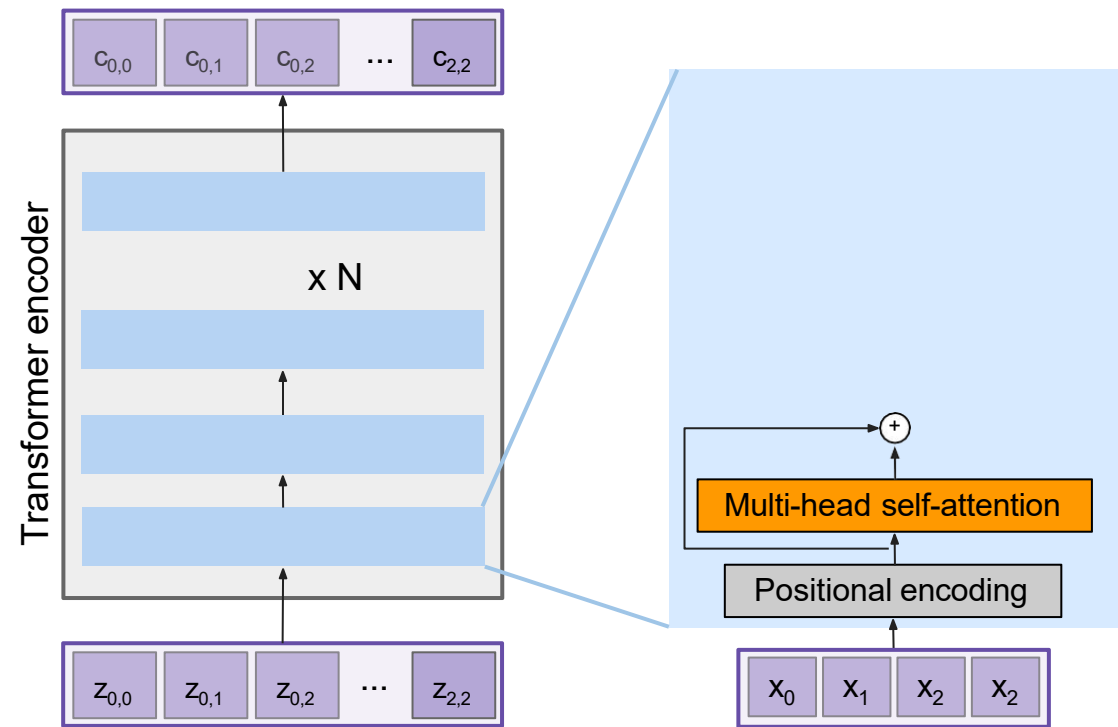


在一组输入上进行注意力操作

引入位置编码

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer *encoder* block



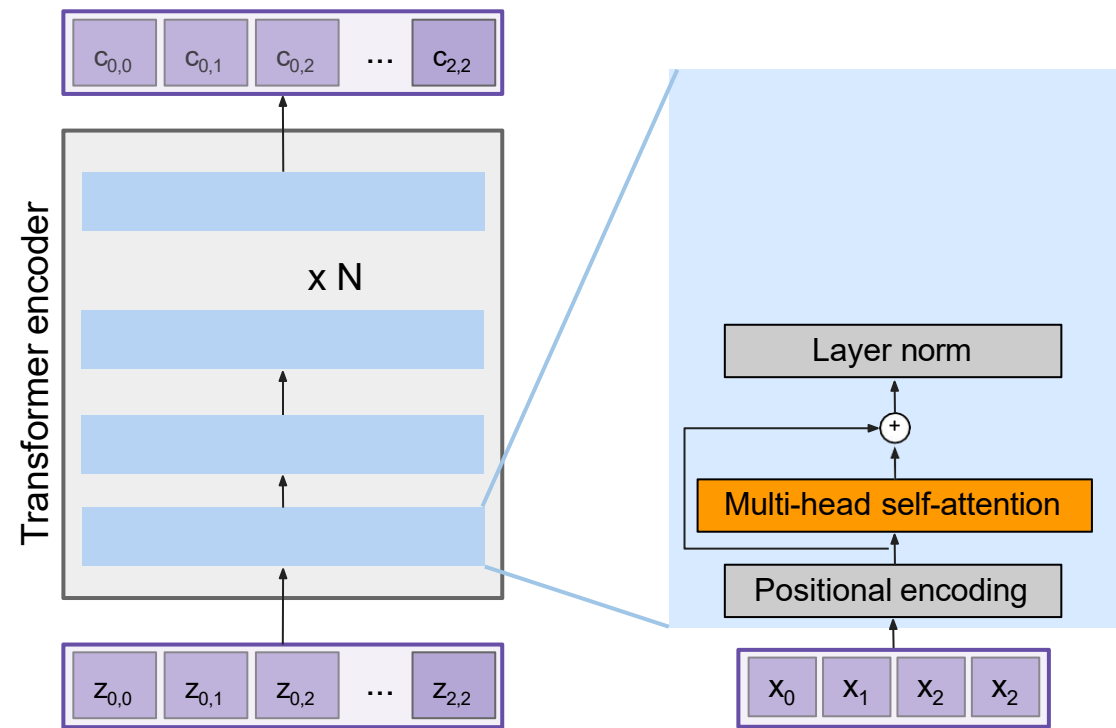
残差连接

在一组输入上进行注意力操作

引入位置编码

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer *encoder* block



对每个向量单独进行层归一化

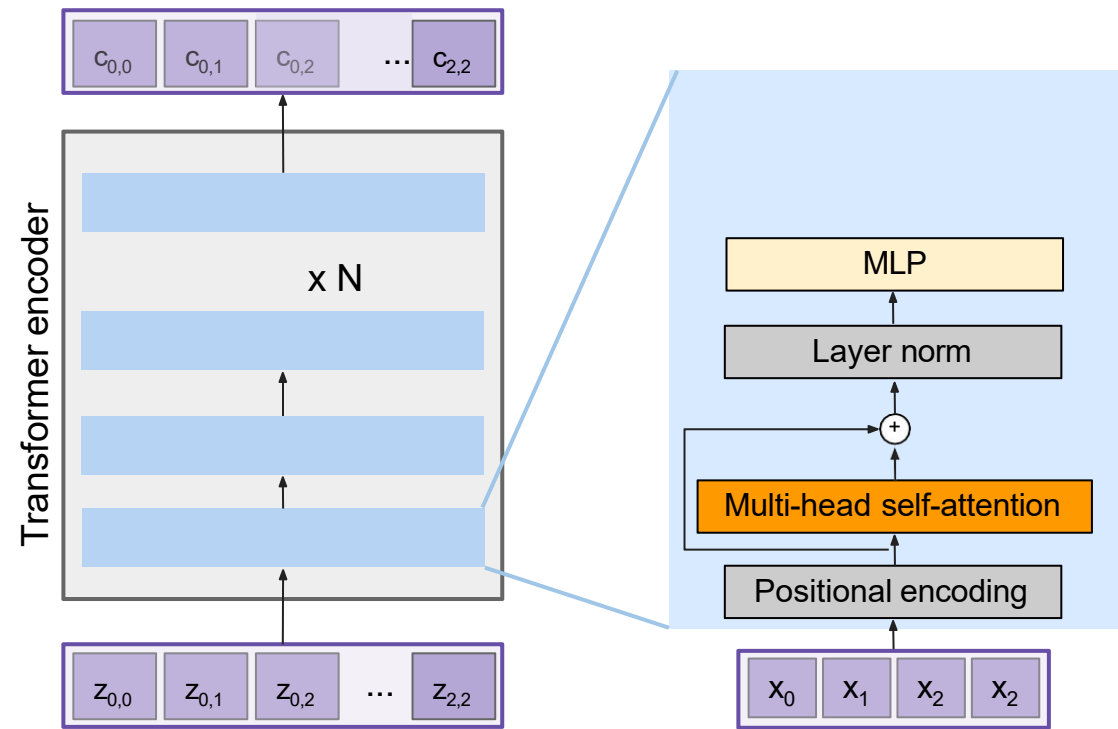
残差连接

在一组输入上进行注意力操作

引入位置编码

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer *encoder* block



对每个向量单独应用MLP

对每个向量单独进行层归一化

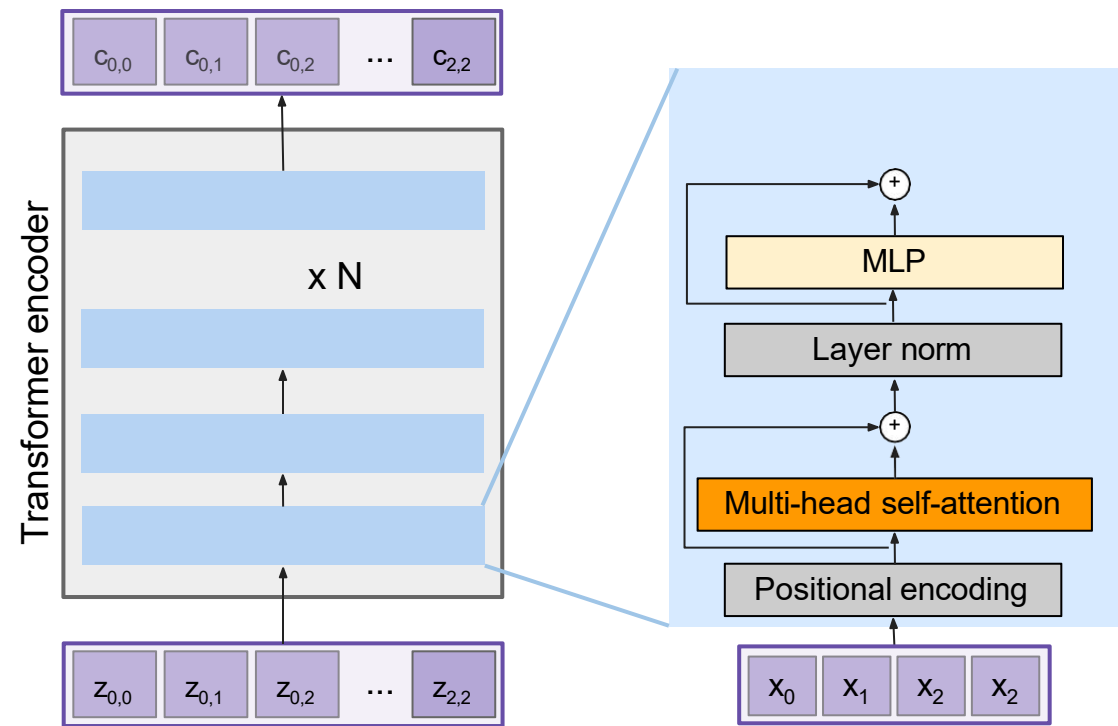
残差连接

在一组输入上进行注意力操作

引入位置编码

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer *encoder* block



残差连接

对每个向量单独应用MLP

对每个向量单独进行层归一化

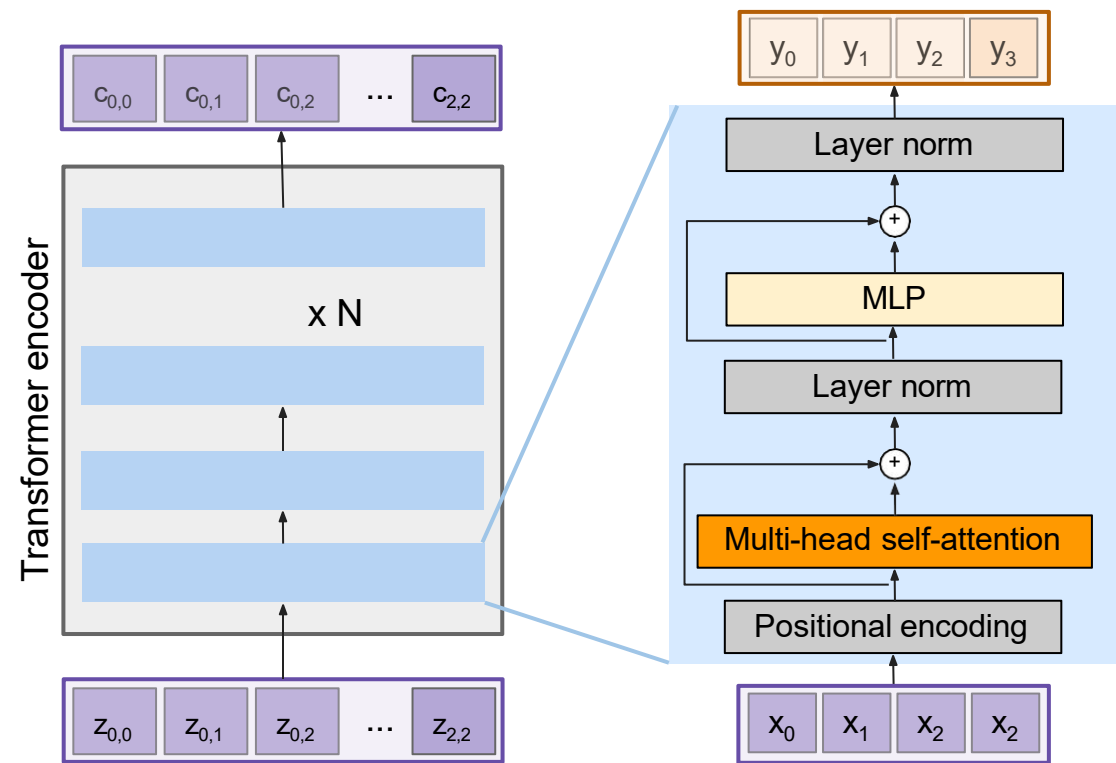
残差连接

在一组输入上进行注意力操作

引入位置编码

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer *encoder* block



Transformer Encoder Block:

Inputs: Set of vectors x

Outputs: Set of vectors y

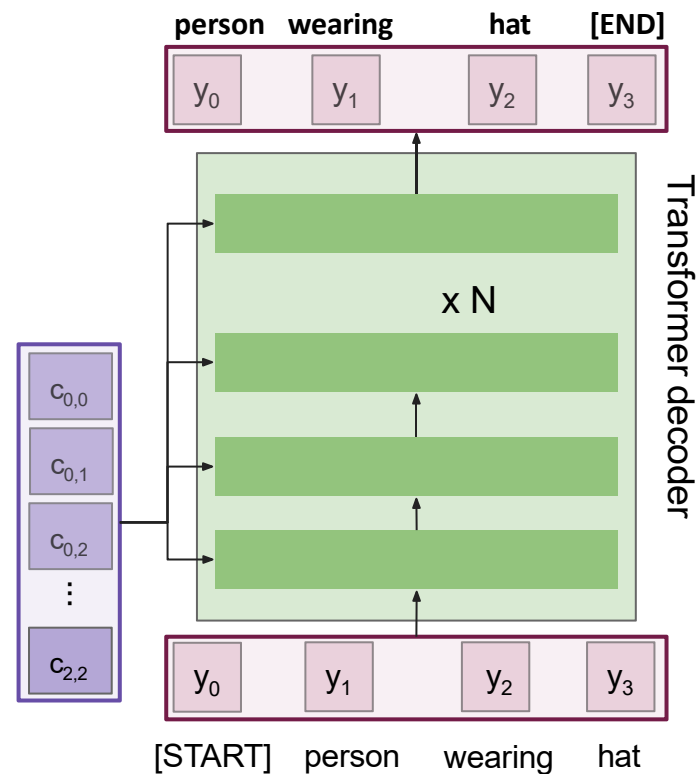
自注意力机制是向量之间唯一的交互方式。

Layer norm 和 MLP 对每个向量单独进行

高度可扩展、高度可并行，但内存占用高。

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer *decoder* block

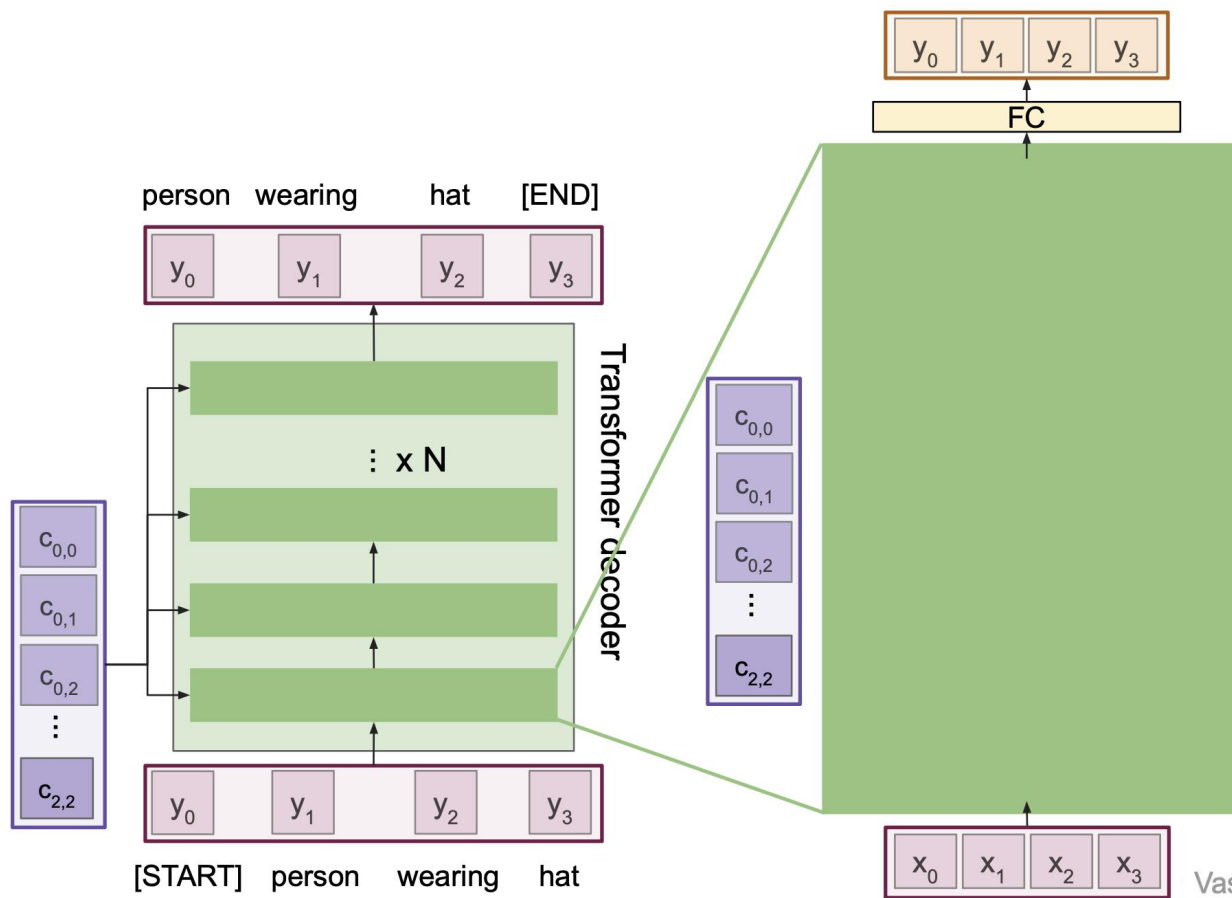


由 N 个 decoder 块组成.

在原论文中 $N = 6, D_q = 512$

Vaswani et al, "Attention is all you need", NeurIPS 2017

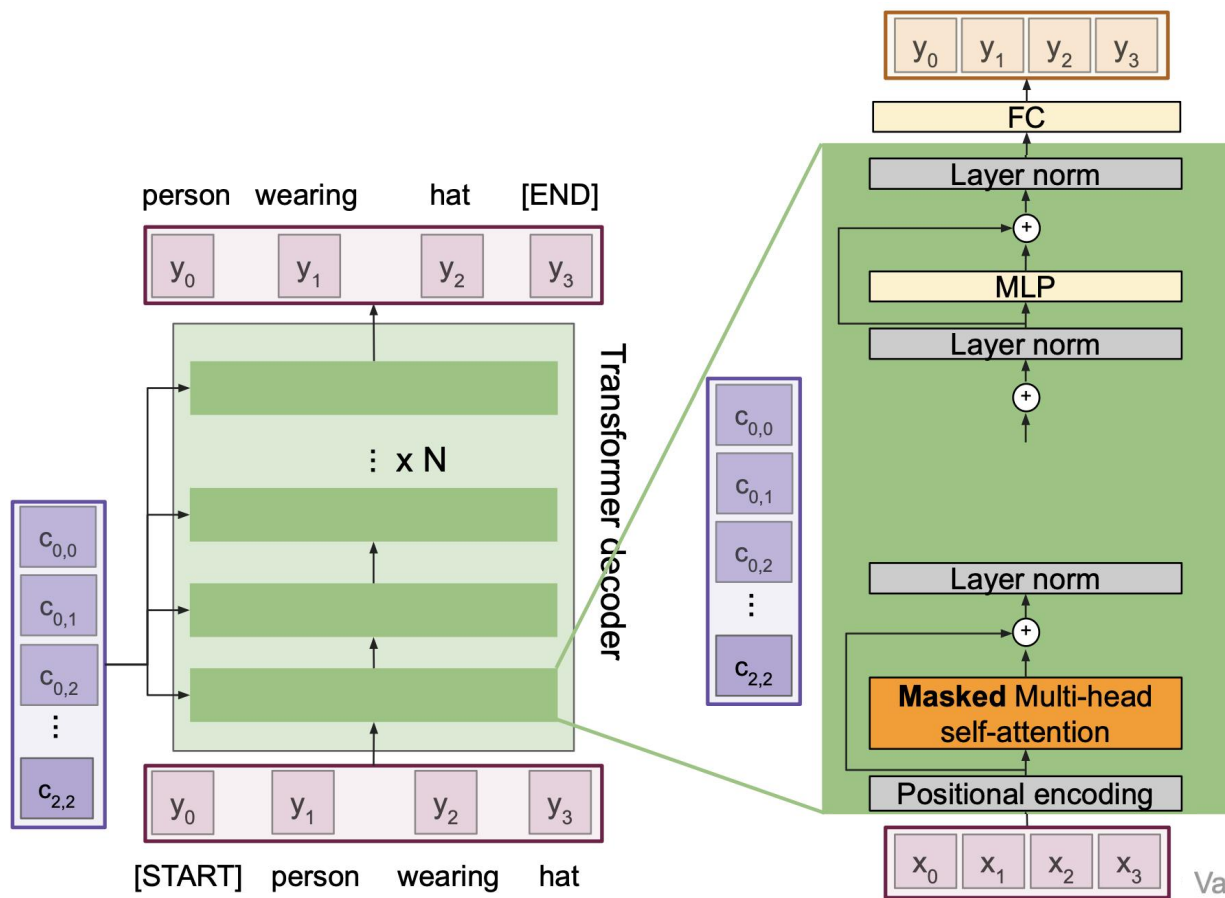
The Transformer *decoder* block



让我们来深入了解一个decoder块

Vaswani et al, "Attention is all you need", NeurIPS 2017

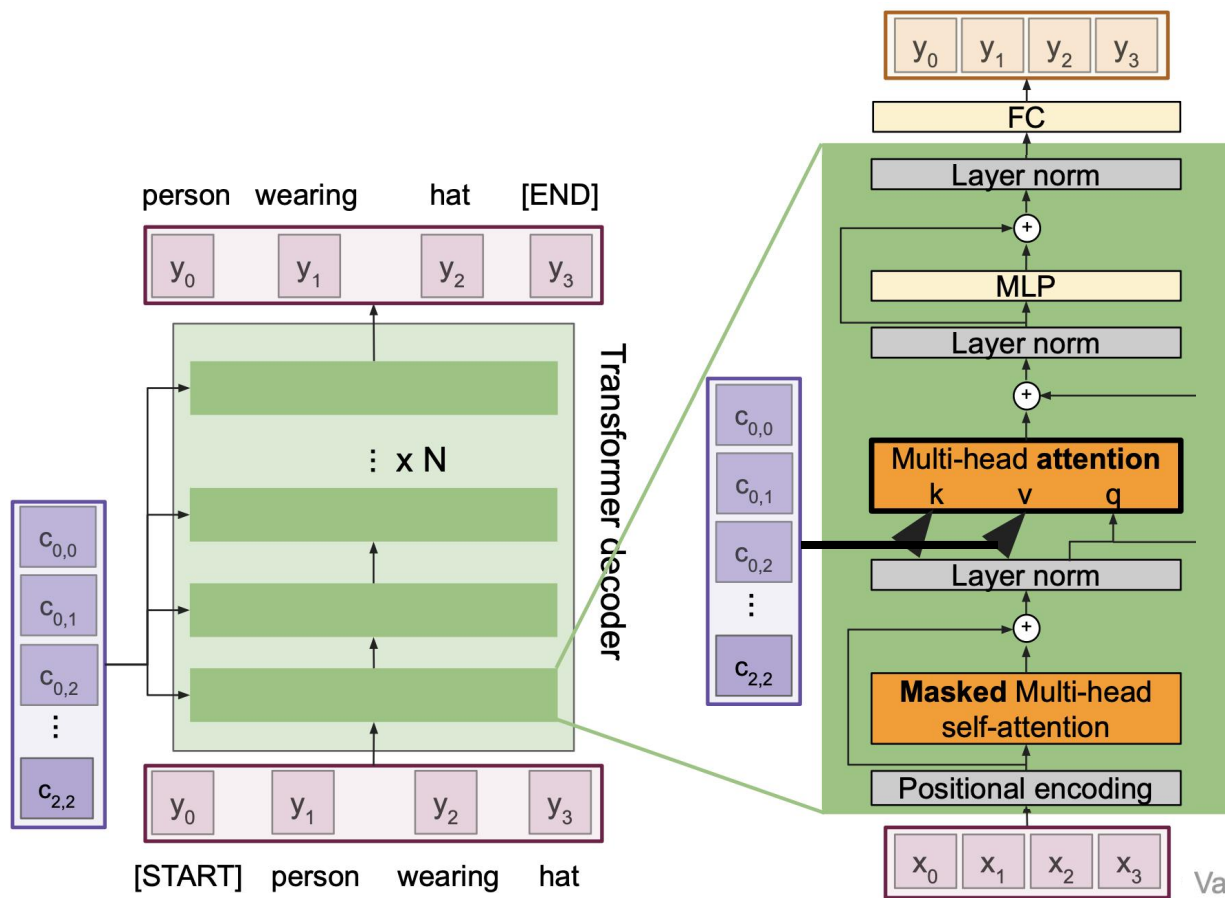
The Transformer *decoder* block



网络大部分类似
transformer encoder

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer *decoder* block

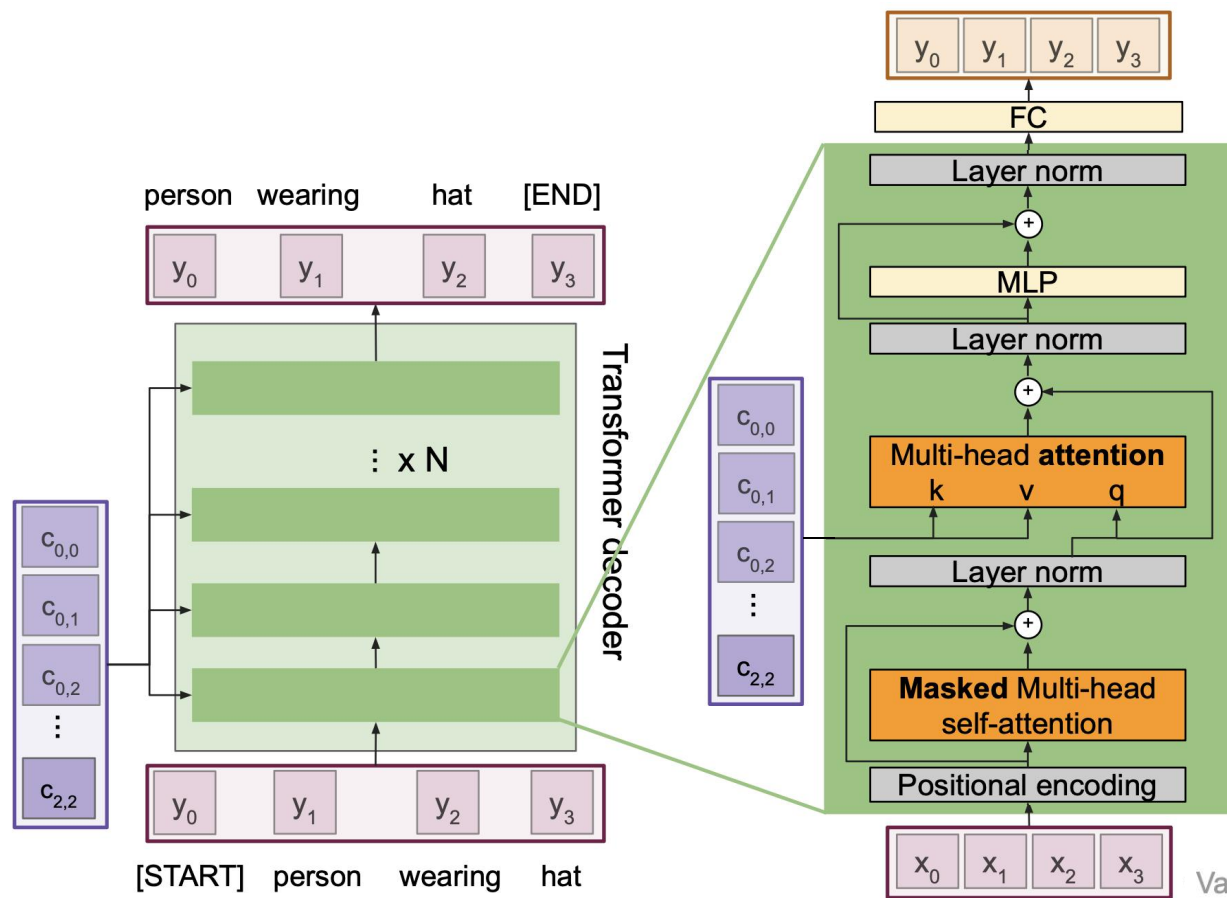


多头注意力机制块在
transformer
encoder的输出上进行
注意力操作

对于图像描述任务，这是我们将图像特征注入decoder的方法。

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer *decoder* block



Transformer Decoder Block:

Inputs: Set of vectors x and Set of context vectors c .

Outputs: Set of vectors y .

Masked Self-attention 只与之前的输入交互。

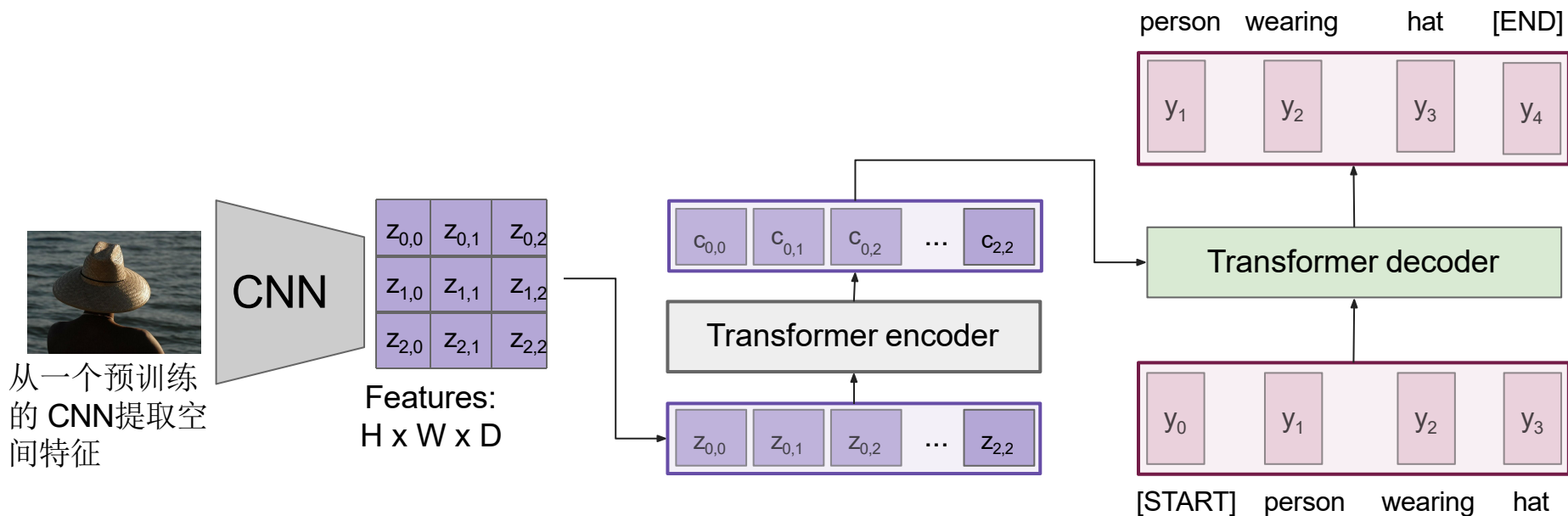
Multi-head attention块不是 self-attention，它在transformer encoder的输出上进行

高度可扩展、高度可并行，但内存占用高。

Vaswani et al, "Attention is all you need", NeurIPS 2017

Image Captioning using transformers

- 完全没有循环结构



从一个预训练的
CNN提取空
间特征

Image Captioning using transformers

- 或许我们不再需要卷积?

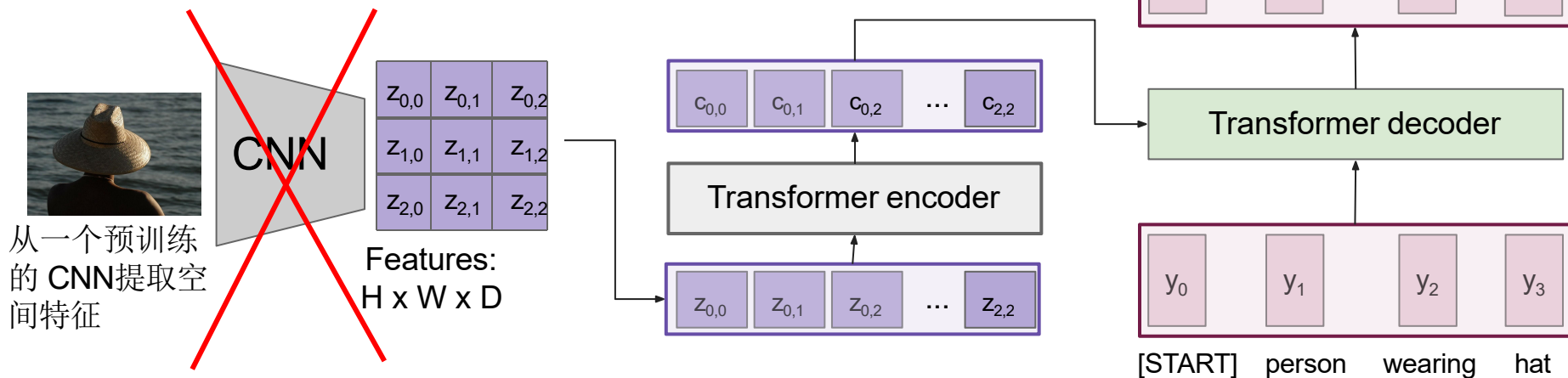
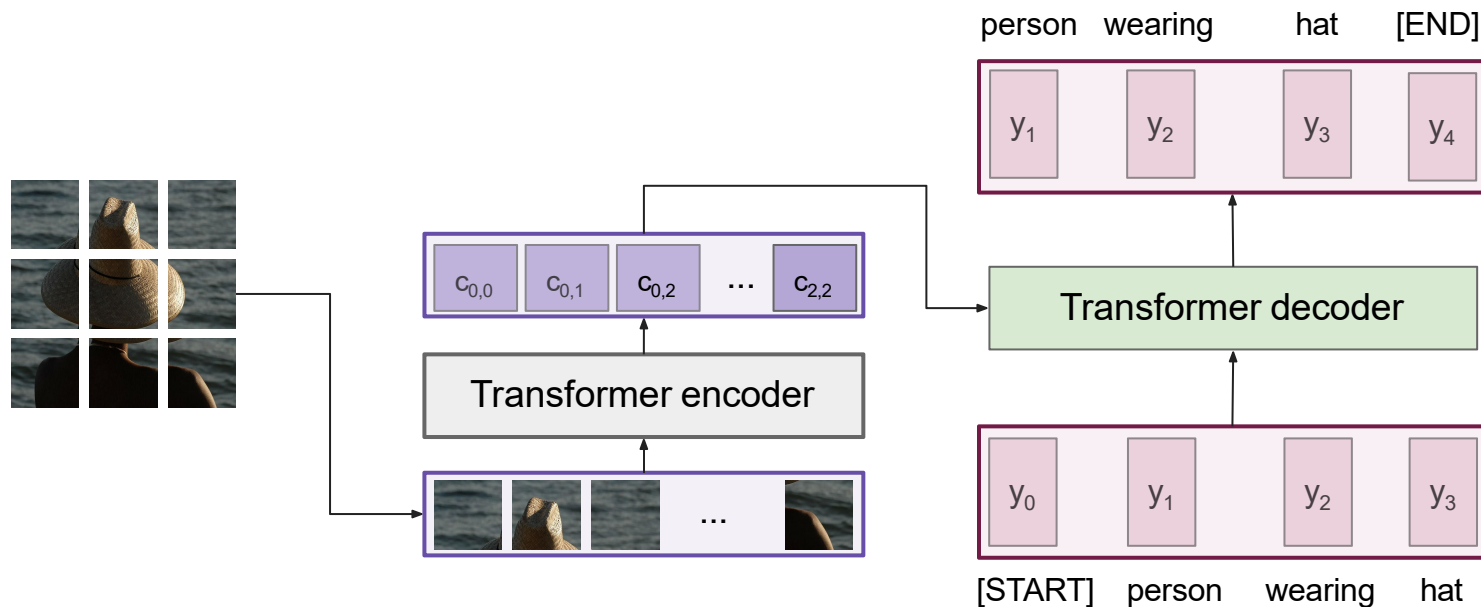


Image Captioning using **ONLY** transformers

- 从像素到语言的Transformer



Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ArXiv 2020
[Colab link](#) to an implementation of vision transformers

Vision Transformers **vs.** ResNets

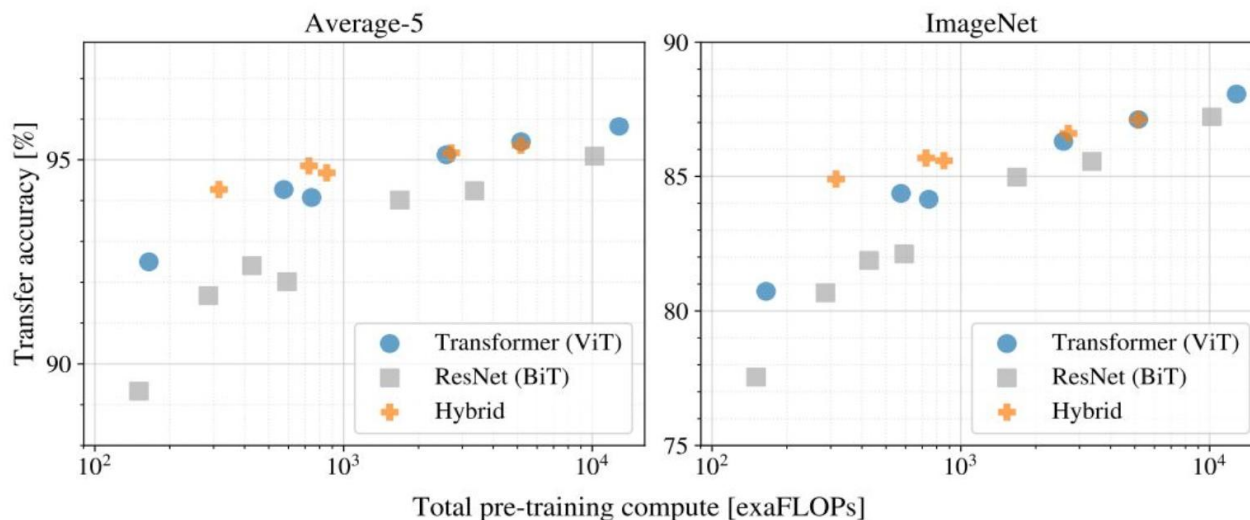
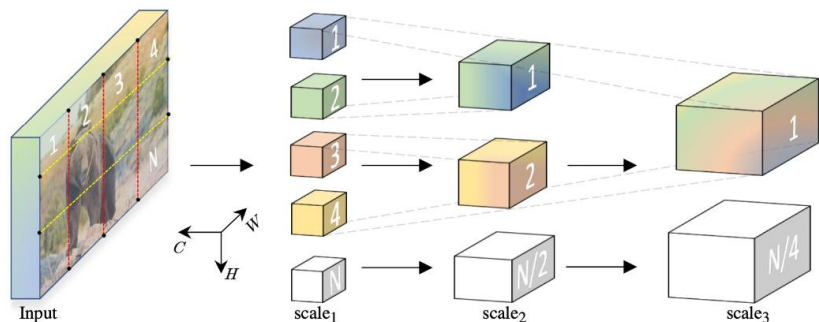
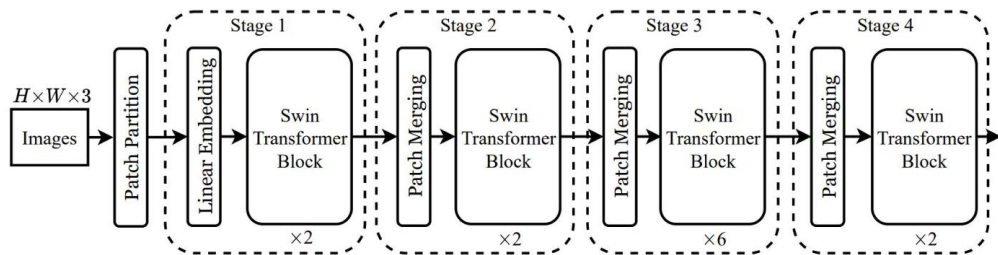


Figure 5: Performance versus cost for different architectures: Vision Transformers, ResNets, and hybrids. Vision Transformers generally outperform ResNets with the same computational budget. Hybrids improve upon pure Transformers for smaller model sizes, but the gap vanishes for larger models.

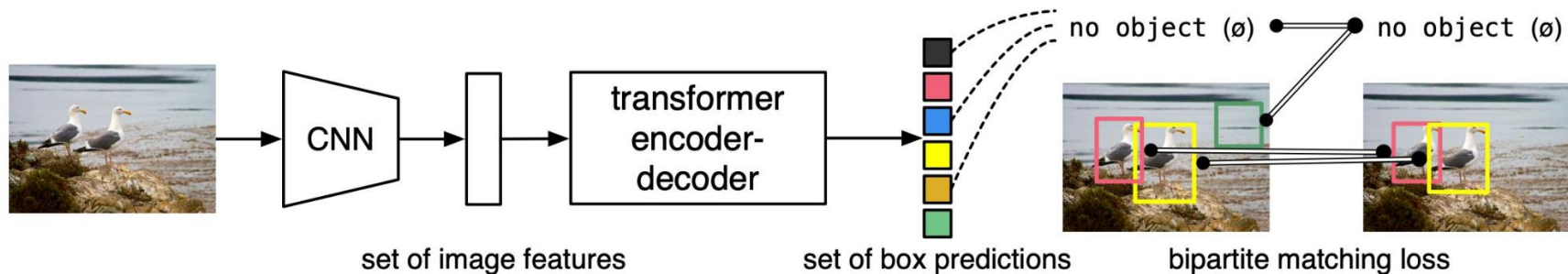
Vision Transformers



Fan et al, "Multiscale Vision Transformers", ICCV 2021



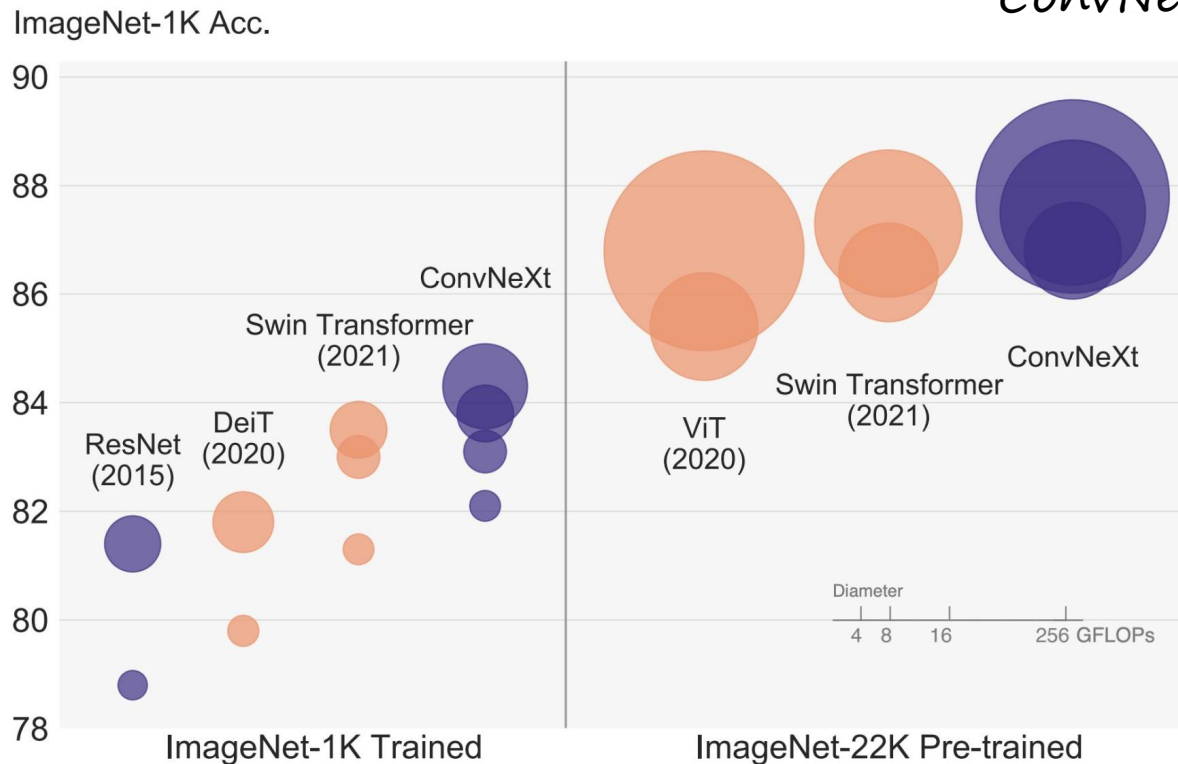
Liu et al, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows", CVPR 2021



Carion et al, "End-to-End Object Detection with Transformers", ECCV 2020

Vision Transformers

ConvNets卷土重来!

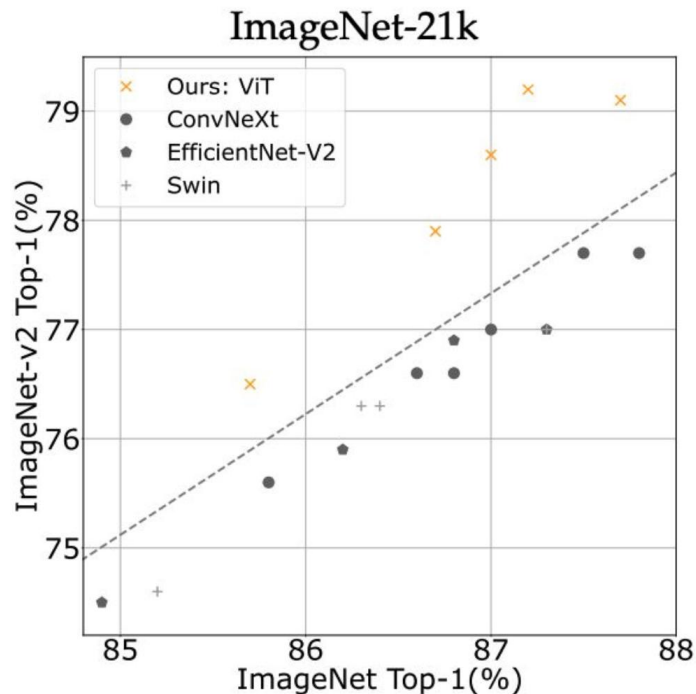
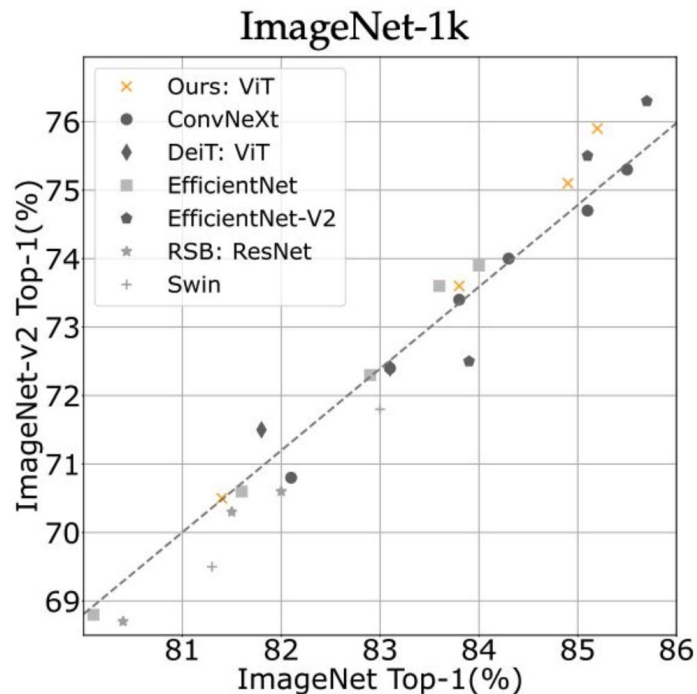


A ConvNet for the 2020s. Liu et al. CVPR 2022

Vision Transformers

DeiT III: Revenge of the ViT

Hugo Touvron^{*,†} Matthieu Cord[†] Hervé Jégou^{*}



总结

- 对 $RNNs$ 引入**注意力机制**可以使它们在每一个时间步“关注”输入的不同部分。
- **The general attention layer**是一种可以用于设计新型神经网络结构的新层。
- **Transformers** 是一种使用**自注意力**和**层归一化**的层。
 - 它具有**高度可扩展性**和**高度可并行性**。
 - 训练**更快**，模型**更大**，在视觉和语言任务中表现**更好**。
 - 它们正在迅速取代 RNN 、 $LSTM$ ，甚至有可能取代卷积操作。

谢谢！